

A survey of machine learning-driven task scheduling approaches for multiprocessor systems

Shaima Rahim^{a,b}, Piyoosh Purushothaman Nair^{a,b}, Rajesh Devaraj^{c,*}

^a Department of Computer Science and Engineering, College of Engineering, Trivandrum, Kerala, India

^b APJ Abdul Kalam Technological University, Thiruvananthapuram, Kerala, India

^c SW-TEGRA, Nvidia Graphics, Bangalore, Karnataka, India

ARTICLE INFO

Keywords:

Multi-core systems
Task models
Scheduling
Energy efficiency
Thermal management
Fault-tolerance
Multi-objective optimization
Machine learning
Deep learning
Reinforcement learning
Deep reinforcement learning

ABSTRACT

Modern embedded computing systems face increasing computational demands due to the complexity of applications in domains such as autonomous vehicles, industrial automation, and healthcare. These systems require real-time responsiveness, energy efficiency, and reliability, necessitating efficient resource scheduling in multiprocessor environments. However, traditional scheduling approaches—ranging from optimal but computationally expensive methods to heuristic-based techniques—often struggle to handle dynamic workloads, multi-objective constraints, and architectural heterogeneity. Machine learning has emerged as a transformative solution, enabling adaptive and data-driven scheduling decisions. This survey provides a comprehensive review of learning-based scheduling techniques in multiprocessor systems, encompassing *Machine Learning*, *Deep Learning*, *Reinforcement Learning*, and *Deep Reinforcement Learning* methods. For this purpose, scheduling approaches are categorized based on *processor architectures* (homogeneous/heterogeneous) and *task models* (independent/dependent), while analyzing key optimization objectives such as *makespan minimization*, *energy efficiency*, *thermal management*, and *fault-tolerance*. Additionally, we examine the datasets employed in learning-based scheduling and discuss their role in evaluating scalability, adaptability, and real-world applicability of proposed methods.

Contents

1.	Introduction	2
2.	Prior surveys	4
2.1.	Task scheduling in multiprocessor real-time systems.....	4
2.2.	Task scheduling in distributed environments.....	5
3.	Research methodology	6
4.	System model	7
4.1.	Platform model	7
4.2.	Task model	8
4.2.1.	Independent task model.....	9
4.2.2.	Dependent task model	9
4.3.	Communication modeling in multiprocessor systems	9
4.4.	Performance objectives model	10
4.4.1.	Makespan	10
4.4.2.	Energy and power model	10
4.4.3.	Thermal model.....	10
4.4.4.	Reliability and fault model.....	10
5.	Scheduling policies and decision strategies.....	11
5.1.	Learning methods.....	11
5.1.1.	Machine learning	11
5.1.2.	Deep learning	11

* Corresponding author.

E-mail addresses: d-tve23jul017@cet.ac.in (S. Rahim), piyooshp@cet.ac.in (P.P. Nair), rdevaraj@nvidia.com (R. Devaraj).

5.1.3.	Reinforcement learning.....	12
5.1.4.	Deep Reinforcement Learning.....	12
6.	Machine learning-based works.....	13
6.1.	Makespan minimization.....	13
6.2.	Energy efficiency.....	14
6.2.1.	Homogeneous platforms.....	14
6.2.2.	Heterogeneous platforms.....	14
6.3.	Multi-objective optimization.....	15
7.	Deep learning based methods.....	15
7.1.	Makespan minimization.....	16
7.2.	Energy optimization.....	17
8.	Reinforcement learning based works.....	17
8.1.	Makespan minimization.....	18
8.2.	Energy efficiency.....	19
8.2.1.	Homogeneous platforms.....	19
8.2.2.	Heterogeneous platforms.....	19
8.3.	Multiobjective optimization.....	19
8.3.1.	Homogeneous platforms.....	19
8.3.2.	Heterogeneous platforms.....	20
9.	Deep reinforcement learning based works.....	21
9.1.	Makespan minimization.....	22
9.1.1.	Homogeneous platforms.....	22
9.1.2.	Heterogeneous platforms.....	22
9.2.	Energy efficiency.....	23
9.2.1.	Homogeneous platforms.....	23
9.2.2.	Heterogeneous platforms.....	23
9.3.	Deadline-constrained scheduling.....	23
9.3.1.	Heterogeneous platforms.....	23
9.4.	Multiobjective optimization.....	24
9.4.1.	Homogeneous systems.....	24
9.4.2.	Heterogeneous systems.....	24
10.	Experimental environments.....	27
10.1.	Datasets used in prior studies.....	27
10.1.1.	Synthetic and benchmark datasets.....	28
10.1.2.	Scientific workflow benchmarks.....	28
10.1.3.	Cloud and production traces.....	30
10.1.4.	Database and simulation benchmarks.....	30
10.1.5.	DS3 simulator-based workflow benchmarks.....	31
10.1.6.	High-performance computing and multi-core benchmarks.....	31
10.1.7.	Thermal, power and fault simulation environments.....	32
10.2.	Performance metrics used for evaluation.....	32
10.3.	Comparative empirical analysis: A case-study.....	34
11.	Challenges and future directions.....	35
11.1.	Well-known challenges.....	35
11.2.	Newly identified challenges.....	35
11.3.	Remedies and emerging directions.....	36
12.	Conclusion.....	36
	CRedit authorship contribution statement.....	37
	Declaration of competing interest.....	37
	Data availability.....	37
	References.....	37

1. Introduction

Embedded computing systems play a vital role in numerous real-world applications, ranging from consumer electronics and industrial automation to critical domains such as healthcare, aerospace, and automotive systems [1]. As these applications grow more complex and intelligent, their computational demands have significantly increased, often requiring real-time responsiveness and energy efficiency. A compelling example is the self-driving automobile, which integrates multiple control functions such as traction control, fuel injection, and anti-lock braking systems. These functionalities must operate concurrently, reliably, and within stringent timing constraints, placing immense pressure on the underlying hardware. To meet these demands, modern embedded platforms often employ heterogeneous architectures comprising a finite number of processing elements with varying capabilities. However, the efficient utilization of such constrained resources

is critical to ensuring optimal system performance. This challenge gives rise to the scheduling problem, which focuses on determining the optimal execution order and resource allocation for a set of applications (hereafter termed *tasks*) while satisfying application-specific constraints such as deadlines, energy limits, and reliability requirements.

The scheduling problem in computing is the strategic assignment of a set of computational tasks to available processors over time, with the goal of optimizing certain performance measures subject to system constraints. Many forms of task scheduling, particularly those involving multiple processors, deadlines, or resource constraints, are NP-hard problems [2], meaning no polynomial-time algorithm is known to solve them optimally in the general case. Tasks, being individual units of computation, can have properties like *execution time*, *levels of priority*, *release times*, *deadlines*, and *inter-task relationships*. Processors, the executing entities, can be *homogeneous* having the same capabilities or *heterogeneous* which varies in performance, power usage, or specialized capabilities. Successful scheduling needs to address

numerous constraints: *makespan minimization* (total time to finish all tasks) [3], *deadline satisfaction* (critical in safety-critical real-time systems) [4], adherence to *task precedence relations* [5], *energy optimization* (important in battery-powered or energy-conscious systems) [6], and fault-tolerance (providing system reliability in the presence of hardware or software faults) [7]. Meeting these complex objectives requires advanced scheduling techniques that can provide efficient and reliable task execution in a variety of computing environments.

The traditional schedulability schemes are broadly divided into *optimal*, *heuristic*, and *meta-heuristic* schedulability approaches. *Optimal* approaches [8] aim to find the best possible solution by systematically exploring the solution space, ensuring completeness and correctness. These methods, such as *Linear Programming* [5], *Dynamic Programming* [9], and *Branch and Bound* [10], are mathematically rigorous and suitable for small to medium-sized problems where computational cost is manageable. However, they often become infeasible for large-scale or NP-hard problems due to exponential time complexity. *Heuristic* methods [11], on the other hand, focus on speed and practicality by employing problem-specific rules or strategies to generate good-enough solutions in a reasonable time frame. Examples include greedy algorithms [12], nearest neighbor in routing problems [13], and rule-based scheduling. While they are efficient, heuristics do not guarantee finding the optimal solution and may perform poorly in some scenarios. *Meta-heuristic* approaches [14,15] provide a more flexible and generalized framework, typically inspired by natural phenomena such as evolution or swarm behavior. Techniques like *Genetic Algorithms* [16], *Simulated Annealing* [17], *Particle Swarm Optimization* [18], and *Ant Colony Optimization* [19] fall into this category. These methods explore the solution space more broadly than heuristics, often incorporating mechanisms to avoid local optima, making them suitable for complex, high-dimensional problems. Although meta-heuristics also do not guarantee optimality, they often achieve near-optimal solutions with better consistency across diverse problem instances.

Scheduling becomes more complex in multiprocessor systems as it is required to control concurrent execution of tasks on several processing units, each of which may have same or different capabilities. The processors' nature (either homogeneous or heterogeneous) affects the scheduling approaches additionally. Heterogeneous systems that consist of different processors with multiple performance characteristics and special functions have extra difficulties for task allocation and require algorithms with the ability to assign tasks adequately according to requirements and capabilities of the processors. Scheduling criteria in such domains can be either single or multi-faceted. Single-criterion scheduling can target the simple objective of minimization of makespan or respecting task deadlines alone. In contrast, multi-objective scheduling aims to simultaneously optimize several goals, such as reducing energy consumption, enhancing fault-tolerance, and managing thermal conditions. Achieving these goals needs adaptive scheduling algorithms that can balance trade-offs and adaptively react to dynamic workloads and system states.

Machine Learning has become a cornerstone of modern computing, offering advanced capabilities for analyzing large datasets and uncovering patterns that traditional algorithms often miss [20]. Its adaptability has led to remarkable progress in domains such as speech recognition, image processing, and autonomous systems. In computing environments, particularly in multiprocessor and heterogeneous systems, ML plays an increasingly vital role in addressing complex challenges like dynamic scheduling, power and thermal management, fault prediction, and workload optimization. As the complexity and dynamism of these systems grow, *conventional scheduling techniques often fall short of delivering optimal or near-optimal performance, especially under fluctuating runtime conditions*. To overcome these limitations, ML has emerged as a powerful, data-driven paradigm capable of predictive and adaptive decision-making. Unlike static heuristics or computationally intensive optimization algorithms, ML techniques can learn from historical data and online system metrics to forecast execution time,

energy consumption, thermal behavior, and fault likelihood, thereby enabling proactive, context-aware scheduling decisions. In heterogeneous architectures with diverse processing elements, ML models such as *classification*, *regression*, and *reinforcement learning* can determine optimal task-to-core mappings based on workload profiles and current system states. Furthermore, ML supports multi-objective optimization by balancing trade-offs between competing constraints. These advancements underscore ML's pivotal role in enabling scalable, intelligent, and energy-aware scheduling solutions in sophisticated, high-performance computing systems.

While learning-based methods have been successfully applied to various scheduling objectives, their integration differs fundamentally between *independent* and *dependent* task models. In independent-task scheduling, learning algorithms primarily focus on predicting task performance and determining optimal task-to-core mappings, often using supervised or regression-based models trained on runtime features such as instruction-level behavior, cache activity, and power consumption. In contrast, dependent-task scheduling involves precedence and communication constraints, requiring the learning framework to determine both *task ordering* and *mapping*, often modeled as a sequential decision-making process using reinforcement learning or graph-based neural networks. Recognizing these methodological differences is crucial for understanding how learning algorithms modify traditional scheduling decisions such as slack computation, priority assignment, and core selection across different task models. This survey thus emphasizes a structured understanding of how learning paradigms influence scheduling design for both independent and dependent task formulations on homogeneous and heterogeneous platforms.

Unlike earlier reviews that broadly discuss ML assisted scheduling techniques, this survey establishes a unified perspective linking the *task model*, *processor architecture*, and *learning mechanism*. By differentiating between independent and dependent task models, this survey highlights how the nature of task dependencies shapes the choice of learning formulation, scheduling objectives, and adaptability. This viewpoint clarifies how data-driven schedulers transition from using predictive models that estimate task or system behavior (for independent tasks) to adaptive decision-making frameworks that learn scheduling policies through interaction and feedback (for dependent tasks). The works reviewed in this survey paper are organized and discussed across several key areas, as illustrated in Fig. 1. This survey investigates recent developments in the use of learning-based approaches, such as machine learning, deep learning, reinforcement learning, and deep reinforcement learning for addressing scheduling problems in multiprocessor systems. Special attention is given to performance objectives and system constraints, including makespan, energy usage, thermal conditions, fault-tolerance, and combinations of these as part of multi-objective optimization strategies. The survey also highlights the current limitations and open challenges that hinder the widespread adoption of intelligent scheduling techniques in real-world multiprocessor platforms, along with insights into future research directions to address these issues. The key contributions of this survey are as follows:

- Introduced a four dimensional taxonomy built from 83 reviewed studies that clarifies how learning paradigms, task models, platform heterogeneity, and objectives interact; this taxonomy enables researchers to place new methods into a consistent framework and identify underexplored intersections.
- Presented a comparative analysis which indicates that lightweight ML suits low-overhead prediction tasks, DL effectively models complex dependencies, and RL/DRL perform best in online, multi-objective trade-off control, but at the cost of increased inference latency and reduced generalization.
- Our analysis across learning paradigms also highlights an emergent research trajectory from supervised prediction for independent tasks to policy learning and GNN/Transformer architectures for dependency-aware schedulers and identifies which problem types should favor which approach.

Section 1: Introduction	Section 2: Prior Surveys	Section 3: Research Methodology
Section 4: System Model		
Platform Model	Task Model	Communication Modelling
Section 5: Scheduling Policies and Decision Strategies		
Section 6: Machine Learning based Works		Section 7: Deep Learning based Works
Section 8: Reinforcement Learning based Works		Section 9: Deep Reinforcement Learning based Works
Section 10: Experimental Environments		
Performance Metrics Used for Evaluation	Datasets used in Prior studies	Comparative Empirical Analysis
Section 11: Challenges and Future Directions		Section 12: Conclusion

Fig. 1. Organization of the survey.

- Systematically compiled tabular summaries that capture each study's core attributes - including learning strategies, execution environments, target objectives, methodology, results and limitations, thereby offering a concise reference for comparative assessment and replication.
- Offered recommendations for experimental standardization (benchmarks, metrics, and reporting practices) to improve cross-paper comparability and accelerate industrial adoption.
- A discussion of major challenges and open research problems associated with applying learning-based methods to scheduling in multiprocessor systems, along with possible future remedies and research directives.
- Through a critical synthesis of existing literature, we outline promising research directions emphasizing hybrid and transferable learning frameworks, cross-platform generalization, and strategies to bridge the gap between simulation-based and real-world implementations.

The remainder of this survey is organized as follows: Section 2 reviews prior surveys on task scheduling, analyzing their scope and limitations, while positioning our work as a comprehensive learning-focused extension. Section 3 describes the research methodology adopted in this study. Section 4 presents the system model, which includes the platform model, task model, communication modeling, and performance objective model. Section 5 provides an overview of the scheduling policies and learning algorithms employed in existing studies. Sections 6, 7, 8, and 9 present a structured review of recent works organized according to machine learning, deep learning, reinforcement learning, and deep reinforcement learning approaches, addressing independent and dependent task models on various hardware platforms. Section 10 discusses experimental evaluation aspects, including a summary of the datasets used in prior studies and the performance metrics commonly adopted for evaluating scheduling techniques. Section 11 highlights the key challenges and outlines potential future research directions, while Section 12 concludes the survey.

2. Prior surveys

The problem of task scheduling on multiprocessor systems has been recognized by the research community for a significant period, resulting in various publications, including comprehensive surveys. However, no existing survey provides a systematic discussion of learning-based scheduling strategies categorized by the *task model*. This distinction is crucial because scheduling decisions such as task ordering, dependency handling, slack computation, and task-to-core mapping differ fundamentally between *independent* and *dependent* task formulations.

Equally important is the consideration of the *hardware platform model*, which characterizes whether the underlying architecture is homogeneous (identical cores) or heterogeneous (cores or accelerators with diverse performance and power characteristics). This distinction directly influences the complexity of task-to-core mapping, load balancing, and the achievable trade-offs between energy efficiency and performance. Learning-based schedulers often adapt differently across these architectural settings, necessitating a unified framework that accounts for both computational and platform heterogeneity. With the increasing prevalence of heterogeneous and energy-constrained computing systems, it becomes necessary to analyze learning-based schedulers in the joint context of both *task-model diversity* and *platform heterogeneity*. Most existing surveys primarily emphasize either dependent workflow management or single-objective optimization (e.g., energy, reliability, makespan, or fault tolerance), thereby overlooking the broader interaction between task models, platform heterogeneity, and learning paradigms. Our survey addresses this gap by providing an integrated and structured perspective on research published over the *past seven years*, encompassing ML, DL, RL, and DRL based scheduling approaches across both independent and dependent task models. Let us briefly review some of the existing surveys on this research topic.

2.1. Task scheduling in multiprocessor real-time systems

Covering hard real-time scheduling and schedulability analysis for homogeneous multiprocessor systems, David and Burns [21] et al. reviewed key results from the late 1960's to late 2009. This survey has outlined fundamental, algorithm-independent findings in this domain, proposed a classification of scheduling methods, and discussed relevant performance comparison metrics. Krishna [22] presented a survey that investigated how to schedule tasks in such systems to maintain deadline guarantees despite processor (permanent or transient) or software failures.

Energy efficiency is as crucial as execution time in computational systems, leading to a trend of trading some performance for lower energy use in diverse environments. Although hardware provides DVFS (Dynamic Voltage and Frequency Scaling), software logic, often within the OS scheduler due to its simplicity, is needed to intelligently apply energy-saving techniques while meeting performance goals. Zhuravlev et al. [23] reviewed the research on energy-aware schedulers, discussing energy-efficient computation methods and the expanding scope of these schedulers to include thermal considerations and heterogeneous multi-core architectures, beyond basic energy minimization. Bambagini et al. [24] surveyed energy-aware scheduling algorithms for real-time systems, reviewing key advancements from the mid-1990's to 2015 and their adaptation to evolving platform features. It first

classified uniprocessor techniques based on energy reduction methods like DVFS and DPM (Dynamic Power Management). Then, it explored how research has addressed the unique problems of energy-aware scheduling on multi-core architectures. Sheikh and Basha et al. [25] surveyed energy-efficient multi-core scheduling algorithms for hard real-time systems, summarizing and classifying existing Partitioned, Semi-Partitioned, and Global approaches for both homogeneous and heterogeneous platforms.

High power density in modern multi-core processors leads to hot spots and reduced reliability. Pagani et al. [26] presented an overview of research efforts that use ML techniques for power and thermal management. This survey explains that traditional methods are often limited because they rely on prior knowledge that is not always available or cannot account for real-world variations. In contrast, ML-based techniques can adapt to changing system conditions by learning from past events, leading to improved management decisions. The survey covers these ML-based approaches for both single-core and multi-core processors, until 2018.

Unlike traditional embedded systems with single criticality, mixed-criticality (MC) systems share platforms among applications of varying importance, leading to significant research in energy-aware scheduling. Zhang and Chen [27] presented a survey on energy-aware scheduling algorithms in MC systems published between 2013 and 2021. It classified these algorithms by system platform, scheduling scheme, and reliability. The survey first examined energy-aware scheduling without reliability, further differentiating between uniprocessor and multiprocessor MC scheduling. Subsequently, it investigated studies that integrate reliability considerations with energy consumption in MC systems.

Recognizing high energy consumption as a major impediment to computing development and the urgency of energy reduction for sustainability, Xie et al. [28] presented a survey focusing on low-energy parallel (dependent tasks) scheduling. It categorized recent advances in this area into energy-efficient, energy-aware, and energy-conscious scheduling styles, encompassing heuristic, meta-heuristic, integer programming, machine learning, and game theory algorithms. Although this work attempts to discuss ML related works, it only covered three such works.

While technology scaling improves processor performance, it also degrades system reliability due to increased susceptibility to transient faults and temperature-induced permanent faults. Fault-tolerance techniques using redundancy are necessary for reliable operation, but their integration complicates timing in real-time embedded systems. Consequently, research has focused on task mapping and scheduling policies that incorporate fault-tolerance while guaranteeing timing and reliability, with advanced techniques also minimizing power and energy. Emerging research now addresses temperature increases caused by fault-tolerance, aiming to satisfy temperature constraints alongside timing and reliability. Safari et al. [29] provided an in-depth survey of these efforts, reviewing and classifying fault-tolerant task mapping and scheduling policies for real-time embedded systems based on their objectives, constraints, fault-tolerance techniques, application models, and hardware models. Reghenzani et al. [30] presented a survey related to current fault-tolerance approaches in real-time systems, identifying key challenges and links between these topics. It also proposed a joint scheduling-failure analysis model to formally illustrate the interplay between software fault-tolerance and timing properties.

An overview of how our survey differs from existing works on task scheduling is presented in Table 1, with a check mark (✓) signifying the inclusion of a particular task model, platform, or performance objective/constraint. The table highlighted that most previous surveys have been narrowly focused on a single aspect, such as a specific task or platform model or particular performance goals, without examining them in the context of ML techniques. To address this limitation, our survey provides a comprehensive and systematic review of diverse task models, platform models, and performance objectives, specifically considering ML-based solutions.

2.2. Task scheduling in distributed environments

In this sub-section, let us review the survey papers that have focused on scheduling applications modeled as workflows, on distributed environments like *Grid*, *Fog*, and *Cloud*.

In [31], Houssein et al. surveyed meta-heuristic algorithms for task scheduling in cloud computing. The article addresses the problem of improper scheduling, which can lead to resource under- or over-utilization. It explains that meta-heuristic algorithms are used to efficiently distribute complex tasks across limited resources. The paper provides a systematic review with a novel taxonomy that classifies these techniques based on the scheduling problem, objective, mapping scheme, and constraints. It discusses the advantages and limitations of popular meta-heuristic methods and presents insights into current challenges and future research directions to assist researchers and practitioners in the field.

Yang and Rahmani [32] provided a systematic literature review on task scheduling techniques in *Fog Computing* (FC), which is presented as an efficient complement to cloud computing for *Internet of Things* (IoT) applications that require low latency. The paper highlighted the need for FC to address the latency issues of cloud-based systems and notes a research gap in task scheduling for this environment. It reviews the advantages and disadvantages of various task scheduling techniques and discusses their main challenges. This survey paper goal is to provide a guide for designing more efficient task schedulers in the future and concludes by offering future research directions.

Medara and Singh [33] surveyed existing energy-efficient techniques for scheduling diverse workflows in the cloud, with a specific focus on methods that minimize energy while maintaining *Quality of Service* (QoS). This study provides a comprehensive overview of current energy-aware workflow scheduling paradigms. Ghafari et al. [34] presented a comparative analysis of 67 task scheduling methods for cloud computing systems, with a primary focus on energy efficiency. This paper highlighted the need for effective task schedulers to meet user QoS requirements and address the key issue of energy consumption in cloud systems. It categorizes the algorithms into three groups: heuristic-based, meta-heuristic-based, and other task scheduling algorithms. For each, it discusses their advantages and disadvantages. The paper aims to help readers choose the best algorithm for optimizing energy consumption and concludes by outlining future research directions in the field.

Abadi et al. [35] reviewed task scheduling algorithms for fog computing, which is presented as an alternative to cloud computing due to the latter's slow response times. The article addresses the challenges of scheduling in a fog environment, where IoT clients require tasks to be executed in a timely, low-cost, and secure manner. It discusses the advantages and limitations of various scheduling algorithms from the literature. Additionally, this survey provides a comparison of different simulation tools to help developers select the most suitable one for their needs and concludes by identifying open issues and promising future research directions in fog task scheduling.

Abadi and Mansouri [36] reviewed fuzzy-based scheduling algorithms in distributed computing, specifically in grid, cloud, and fog environments. The authors highlighted that dynamic tasks and environments pose challenges for traditional schedulers, making fuzzy logic an ideal solution due to its low computational complexity. The survey covers high-quality articles published between 2005 and June 2023, comparing different fuzzy-based schemes based on their strengths, weaknesses, evaluation methods, and simulation environments. It also summarizes various applications of fuzzy logic in distributed systems, explaining the basic concepts of fuzzy inference systems and their role in scheduling, such as in calculating fitness functions, assigning tasks, or clustering resources. The paper concludes by identifying open challenges and future research directions in this field.

In [37], Abadi et al. presented a comprehensive overview of DRL based scheduling algorithms for distributed systems, which are used in

Table 1

Comparison of survey papers on task scheduling in multiprocessor real-time systems.

References	Task model		Platform model		Performance objectives/constraints				ML based solution approaches
	Independent	Dependent	Homogeneous	Heterogeneous	Makespan minimization/meeting deadline	Maximize reliability/fault-tolerance	Minimize energy/power consumption	Temperature control	
David and Burns [21]	✓	×	✓	×	✓	×	×	×	×
Zhuravlev et al. [23]	✓	×	✓	✓	✓	×	✓	✓	×
Krishna [22]	✓	×	✓	×	✓	✓	×	×	×
Bambagini et al. [24]	✓	×	✓	×	✓	×	✓	×	×
Sheikh and Basha [25]	✓	×	✓	✓	✓	×	✓	×	×
Pagani et al. [26]	✓	×	✓	✓	✓	×	✓	✓	✓
Zhang and Chen [27]	✓	×	✓	×	✓	✓	✓	×	×
Xie et al. [28]	×	✓	×	✓	✓	×	✓	×	×
Safari et al. [29]	✓	×	✓	×	✓	✓	✓	×	×
Reghezani et al. [30]	✓	×	✓	×	✓	✓	×	×	×
This survey	✓	✓	✓	✓	✓	✓	✓	✓	✓

fields like astronomy and bio-informatics. The article highlighted that traditional scheduling problems are NP-hard and can be solved using DRL, which combines deep learning and reinforcement learning. The paper reviews and categorizes DRL-based algorithms based on their objectives, QoS parameters, and evaluation environments. The review found that *Q-learning*, a type of reinforcement learning algorithm, is effective for managing scaling and scheduling in a cloud environment. The paper concludes by outlining future research challenges and directions, such as edge intelligence and the development of sustainable AI frameworks.

Hou et al. [38] surveyed DRL-based task scheduling techniques designed to reduce energy consumption in cloud data centers. The article addresses the growing environmental impact of data centers and highlighted DRL as a promising solution for energy-efficient scheduling. It fills a gap in the literature by providing a systematic review and analysis of energy consumption models, DRL algorithms, and the Markov Decision Process (MDP) elements used in research. The paper also summarizes the experimental platforms and datasets employed, identifies current research gaps and challenges, and suggests that DRL-based scheduling can significantly reduce energy consumption.

Mahdizadeh et al. [39] provided a comprehensive exploration of cloud computing and software-defined networks (SDN). The article covers several key areas: It introduced the foundational concepts of cloud computing and software-defined cloud computing. It evaluated various algorithms designed to improve network performance, including IRMTS and reinforcement learning, with the goal of optimizing QoS and data center efficiency. It examined cloud network resource optimization by comparing the performance of different approaches using practical experiments. It also investigated future challenges and scenarios in the domain. In conclusion, the article helps readers choose suitable simulation tools like *Mininet* and *CloudSim* and facilitates a deeper understanding of the architecture, methodologies, and algorithms used to optimize cloud network resources.

DRL is a promising solution for workflow scheduling in dynamic cloud and edge computing environments because of its ability to learn from real-time data. In [40], the authors have analyzed the challenges and opportunities of applying DRL to this problem, including issues with multi-objectivity and curse of dimensionality. Based on these characteristics, the authors proposed a taxonomy for DRL-based workflow scheduling. They used this taxonomy to evaluate the strengths and weaknesses of existing research and to identify future research directions for the field.

All the above survey works have been briefly summarized in Table 2. Previous surveys primarily focused on *workflow scheduling applications* (modeled as DAGs) in *distributed systems* with homogeneous or heterogeneous computing resources. Their main objectives were to minimize energy consumption, satisfy QoS requirements, balance load, and reduce latency, using techniques such as heuristics, meta-heuristics, fuzzy logic, and deep reinforcement learning. While these

surveys primarily focused on workflow (dependent task) scheduling in distributed systems using heuristic, meta-heuristic, or DRL methods, they do not provide a unified analysis of learning-based scheduling across diverse *task-model formulations* and *architectural heterogeneity*. This broader scope motivates our work to consolidate fragmented findings and highlight how different learning paradigms adapt to varying computational abstractions, system architectures, and optimization goals. Such an integrated perspective is essential for identifying open challenges, enabling fair performance comparisons, and guiding the development of next-generation adaptive and energy-efficient schedulers for multicore and distributed systems.

In summary, this survey provides a comprehensive synthesis of learning-based scheduling research across diverse *task models* and *platform architectures*, addressing multiple objectives such as energy efficiency, makespan reduction, reliability, and thermal control in both real-time and distributed environments.

3. Research methodology

This survey follows a structured and systematic literature review process designed to capture the breadth and depth of research on Machine Learning, Deep Learning, and Reinforcement Learning and Deep Reinforcement Learning for task scheduling in real-time and distributed computing environments. The methodology draws inspiration from established review protocols in the systems and task scheduling domains [38,41] but is tailored to address the specific complexities of learning-based scheduling research. To build this survey, we followed a process that went beyond a straightforward checklist of databases and filters. Instead, we treated the search and selection of literature as a layered pipeline that gradually moved from discovery to refinement, and finally to structured analysis. The goal was not only to gather a comprehensive set of works but also to understand how the field of ML, DL, RL, and DRL-based scheduling has evolved across different contexts.

The process began with an extensive search for publications across five established academic databases: *Science Direct*, *IEEE Xplore*, *Springer Link*, *ACM Digital Library*, and *Wiley Online Library*. To make sure no relevant work was overlooked, we also incorporated exploratory searches on *Google Scholar*, which often captures studies that appear in emerging venues. The search covered the period from 2018 to 2025 using keyword combinations connecting scheduling and learning paradigms. Once the initial pool of works was collected, we applied a gradual screening process to filter out papers that fell outside the scope of the survey. Works that were purely hardware-focused or that addressed only peripheral problems such as task offloading without explicit scheduling mechanisms were excluded. Only works that presented explicit scheduling contributions using ML, DL, RL, or DRL techniques were retained.

Table 2
Comparison of survey papers on task scheduling in distributed environments.

Research work	Primary focus	Core problem addressed	Key techniques/Solutions	Platform	Key contributions/Highlights
[31]	Meta-heuristic scheduling	Resource under/over-utilization from improper scheduling	Meta-heuristic algorithms (e.g., swarm, evolutionary, hybrid)	Cloud computing	Provides a novel taxonomy of meta-heuristic techniques, with advantages and limitations.
[32]	Task scheduling in fog computing	High latency in IoT applications	Systematic literature review of available techniques	Fog computing (as companion to cloud)	Fills a research gap by reviewing FC task scheduling, providing future directions.
[33]	Energy-efficient workflow scheduling	Minimizing energy consumption while maintaining QoS	Energy-efficient scheduling techniques	Cloud computing	Provides a comprehensive overview of current energy-aware workflow scheduling paradigms.
[34]	Energy-efficient cloud scheduling	Minimizing energy consumption in cloud systems	Heuristic, Meta-heuristic, and other algorithms	Cloud systems	Comparative analysis of 67 scheduling methods to help readers choose.
[35]	Task scheduling in fog computing	Slow cloud response/latency, cost, and security	Various scheduling algorithms, simulation tools	Fog computing	Reviews algorithms and compares simulation tools for fog developers.
[36]	Fuzzy-based scheduling	Dynamic tasks/environments in distributed computing	Fuzzy-based scheduling algorithms, Fuzzy Logic, Fuzzy Inference Systems	Distributed systems (Grid, Cloud, Fog)	Reviews high-quality articles from 2005–2023, compares schemes based on merits/demerits and evaluation environments, identifies future challenges.
[37]	DRL-based scheduling	NP-hard scheduling problems in distributed systems	DRL, Q-learning	Distributed systems (Cloud)	Provides a comprehensive overview and categorizes DRL algorithms and applications.
[38]	DRL for energy-efficient scheduling	High energy consumption in cloud data centers	DRL, energy consumption models	Cloud data centers	Fills a gap by systematically reviewing DRL for energy reduction. Suggests DRL can significantly reduce energy.
[39]	Heuristic and Meta-heuristic scheduling	Enhancing network performance and resource optimization	Rule-based intelligent meta-heuristic task scheduling, RL, Priority-aware semi-greedy	Cloud computing & Software-defined networks	Guides readers in choosing suitable simulation tools like Mininet and CloudSim.
[40]	DRL-based scheduling	Minimizing makespan and energy consumption	RL, Deep Q-Learning Network (DQN), Double DQN	Cloud computing & Edge networks	Proposes a taxonomy of workflow scheduling with DRL

After this refinement, we obtained a set of 83 reviewed works, which form the core of this survey. To complement these, we included 20 prior survey papers, which serve as valuable comparative references and help to contextualize how the field has evolved over time. Furthermore, 24 dataset and benchmark references (including workflow traces, benchmark suites, and simulation frameworks) were incorporated to situate the evaluation environments used in the reviewed studies and to highlight the empirical foundations on which most scheduling algorithms are tested. This three-way categorization not only strengthens the comprehensiveness of our coverage but also clarifies the boundary between directly reviewed contributions and supporting literature. More importantly, it provides readers with a layered perspective: the reviewed works reflect the current state of methodological innovation, the prior surveys trace the historical and comparative backdrop, and the datasets/benchmarks reveal the experimental grounds upon which the algorithms are validated. Such separation ensures clarity, prevents conflation of contributions, and allows a deeper appreciation of both the novelty and the limitations present in existing research.

To make sense of the reviewed studies, we organized them into a four-dimensional taxonomy. The first dimension captured the *learning paradigm*, ranging from traditional ML methods to advanced DRL frameworks. The second dimension focused on *optimization objectives* such as makespan, energy efficiency, thermal regulation, reliability, and multiobjective trade-offs. The third distinguished between independent task models and dependent workflows typically represented as DAGs, while the fourth dimension classified the execution platform, ranging from homogeneous multi-core processors to heterogeneous multiprocessor systems (e.g., big.LITTLE, edge, and cloud environments). This classification provided a systematic way to compare works and, more importantly, to reveal underexplored areas.

Finally, each study was examined with respect to methodology, learning model, simulation or experimental setup, evaluation metrics, and reported outcomes. The analysis not only identified common design patterns but also highlighted trade-offs, limitations, and emerging directions. Fig. 2 summarizes this process through four complementary visualizations: (a) year-wise publication trends, (b) publisher-wise distribution of reviewed works, (c) distribution of ML, DL, RL, and DRL based studies, and (d) the relative proportions of reviewed works, prior surveys, and dataset references.

4. System model

4.1. Platform model

Generally, there exist two models of hardware processing platforms, namely *multi-core* and *multiprocessor* systems, which differ at the architectural level. A multi-core processor integrates multiple processing cores onto a single chip, allowing them to share resources such as caches, memory controllers, and interconnects. In contrast, multiprocessor systems consist of multiple discrete processors, each with one or more cores, connected through a system bus or high-speed interconnect. While the two concepts are technically different, multi-core emphasizing integration within a chip and multiprocessor encompassing multiple physical processors, in practice they are frequently used interchangeably in scheduling literature [21,25]. This is because both architectures expose the same fundamental challenge: how to allocate and coordinate tasks across multiple concurrent processing units to optimize performance, efficiency, and reliability. With slight abuse of terminology, we use the terms *multi-core* and *multiprocessor* interchangeably in this survey. A multi-core platform is broadly classified as follows:

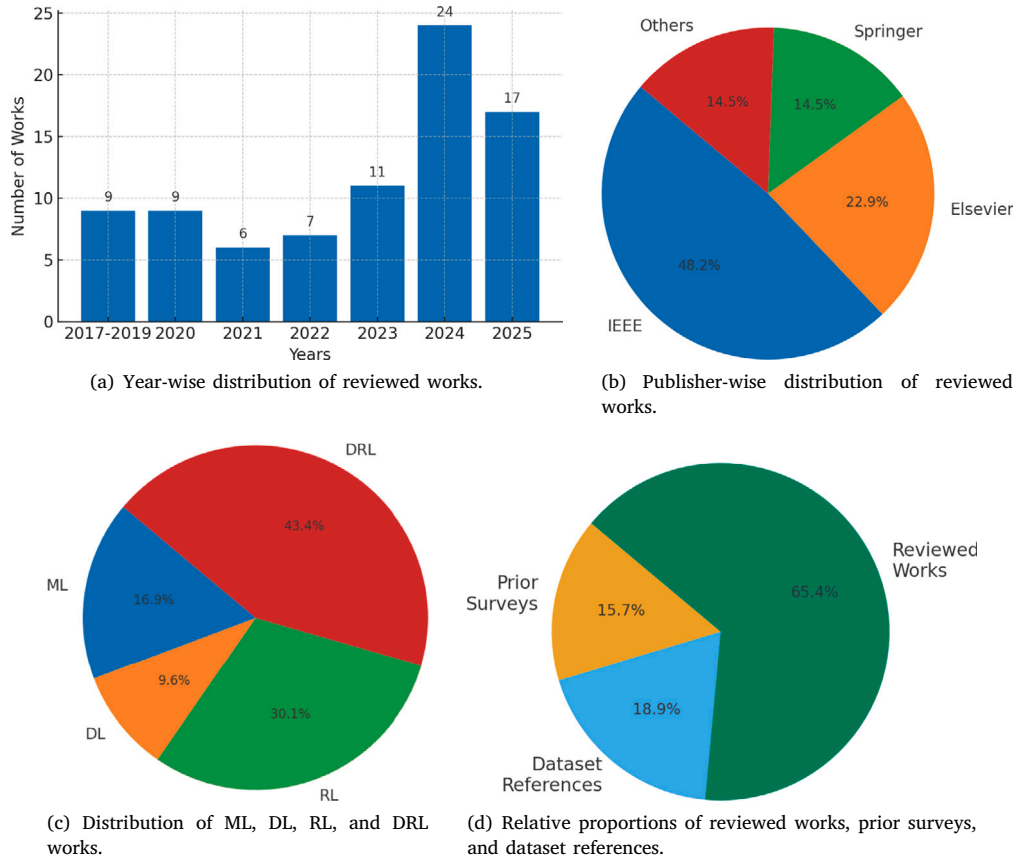


Fig. 2. Overview of surveyed literature year-wise, publisher-wise, learning-method-wise, and distribution of reviewed works, prior surveys, and dataset references.

- A *homogeneous* multi-core architecture is composed of two or more *identical processing cores* on a single chip. These cores share the same micro-architecture, instruction set, clock frequency, and cache hierarchy. Typical examples include symmetric multi-core CPUs such as Intel Core i7 or i5, as well as programming environments like OpenMP that assume identical processing resources.
- In contrast, a *heterogeneous* multi-core architecture integrates a mix of different types of cores on the same chip. These cores are often optimized for different purposes, such as high performance or energy efficiency. A common example is the ARM big.LITTLE architecture, which combines high-performance “big” cores with energy-efficient “little” cores. Other representative cases include CPU+GPU platforms used for compute acceleration and CPU+FPGA systems deployed for domain-specific applications.

From a scheduling perspective, these architectural differences significantly influence the design of ML and RL methods. In homogeneous systems, state and action modeling is relatively straightforward, since all cores are identical and the main challenge lies in balancing the workload. In such cases, objectives like makespan minimization, throughput, or fairness are often studied in isolation. In heterogeneous systems, the same objectives remain relevant, but they are coupled with additional challenges: the state and action spaces must capture device heterogeneity and task-device mappings, and reward functions frequently combine performance goals with trade-offs such as energy efficiency, cost, or thermal constraints. Consequently, the same scheduling approach may behave differently across platforms. Algorithms on homogeneous systems typically converge faster and yield more predictable schedules, while on heterogeneous systems they must

cope with higher decision complexity but can leverage specialized cores to achieve better performance-efficiency balance.

DVFS (*Dynamic Voltage and Frequency Scaling*) and DPM (*Dynamic Power Management*) are two primary techniques used for power and energy management in multi-core processors. DVFS reduces dynamic power consumption by adjusting the voltage and frequency of a processor, and is most effective when the workload is light and the processor can operate at a lower speed without compromising deadlines. DPM, on the other hand, reduces static power consumption by switching idle or underutilized cores into a low-power sleep state. Static power, also known as leakage power, is consumed even when the core is not actively computing. DPM is most effective when a system has long idle intervals, as the energy savings from a sleep state can outweigh the energy cost of the state transition.

Apart from multi-cores, there is a *many-core* platform which is a specialized type of parallel architecture that features a very large number of simple, independent processing cores on a single chip. It is designed to achieve massive parallelism and is distinct from a traditional multi-core CPU. The most common example of a many-core platform is a modern *Graphics Processing Unit* (GPU). While a CPU handles general-purpose tasks efficiently, a GPU is built with a many-core architecture to accelerate tasks like rendering millions of pixels or training a deep neural network. This survey does not cover works related to many-core platforms.

4.2. Task model

Task models define the structure and relationships of workloads in multi-core environments. Based on inter-dependencies, they are

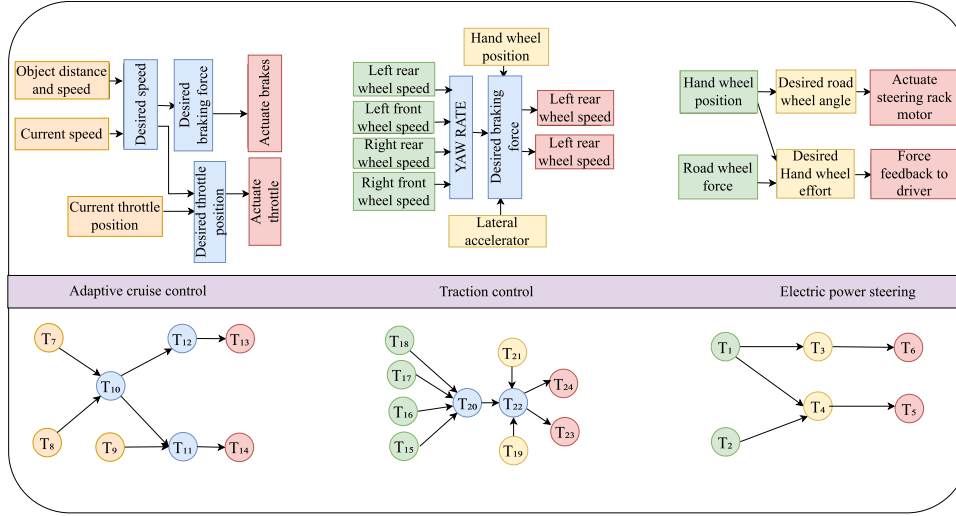


Fig. 3. The three applications in automotive control systems [42] and their DAGs: (1) **Adaptive Cruise Control (ACC)** processes radar/speed inputs to compute throttle/braking commands, (2) **Traction Control (TC)** analyzes wheel speeds and yaw rate to maintain vehicle stability, and (3) **Electric Power Steering (EPS)** converts steering wheel position to road wheel angles.

Table 3

Independent task model: F-16 flight control system.

Task	Execution time (ms)	Period (s)
Guidance	100	10
Control	80	5
Slow navigation	100	10
Fast navigation	60	5
Missile control	500	10

broadly classified into *independent* and *dependent* task models, each with distinct scheduling strategies.

4.2.1. Independent task model

In this model, tasks are self-contained and do not rely on any other task's output. This makes scheduling simpler, as tasks can be assigned to any available processor/core in any order. Independent task models are widely applicable in domains like multimedia processing, data analytics, and scientific simulations. Schedulers for independent tasks aim to optimize throughput, minimize schedule length (also termed *makespan*), and ensure load balancing. Since no execution order is imposed, dynamic strategies can adapt tasks to processors in real-time.

Consider a real-world example, the F-16 Flight Management System (FMS) [43], where modules such as navigation, guidance, and control function independently. This modular design lends itself to parallel execution. Table 3 shows the timing characteristics of different independent functional tasks of the FMS.

4.2.2. Dependent task model

In contrast to the independent models, dependent task models involve tasks connected by precedence constraints. In multiprocessor systems, task dependencies are typically modeled using *Directed Acyclic Graphs* (DAGs), where each node represents a sub task and edges denote precedence and communication constraints. A directed edge from task t_u to t_v implies that t_v cannot commence execution until the output of t_u is available, potentially requiring inter-processor communication. This adds complexity to the scheduler, as execution order and placement must be carefully chosen to minimize idle times and communication delays. If dependent tasks are mapped to different cores, transferring intermediate data becomes necessary, adding communication cost. Such models are typical in automotive control applications [42], where operations must follow a strict sequence. For instance, Fig. 3 depicts DAG representations of Adaptive Cruise Control (ACC), Traction Control

(TC), and Electric Power Steering (EPS) tasks, where control logic is tightly coupled and deadline-sensitive.

4.3. Communication modeling in multiprocessor systems

Many embedded and cyber-physical systems deployed in automotive [1] and industrial applications often use traditional bus-based communication architectures. In these systems, multiple processors or cores share a common communication bus that operates under deterministic real-time protocols such as CAN (Controller Area Network) [44], FlexRay [45], or Time-Triggered Ethernet (TTE) [46]. When tasks are assigned to different cores, the communication delay is influenced by both the inherent characteristics of the chosen protocol and the level of runtime contention on the bus. For example, if task t_u is executed on processor p_i and task t_v on processor p_j , the associated communication cost can be modeled as:

$$comm(t_u, t_v) = \begin{cases} 0 & \text{if } p_i = p_j, \\ D_{bus} + Q_{bus} & \text{if } p_i \neq p_j \end{cases}$$

where D_{bus} represents the fixed delay introduced by the bus protocol, while Q_{bus} captures additional queuing or arbitration delays when multiple tasks contend for the bus at runtime. This simple yet effective model provides insight into how communication overhead and resource contention impact the timing behavior of shared communication infrastructures. In contrast, some scheduling approaches such as the Heterogeneous Earliest Finish Time (HEFT) algorithm [47] assume a fully connected communication topology. Under this abstraction, each pair of processors is directly connected, and the communication delay is primarily a function of data volume and link bandwidth. If task t_u is mapped to processor p_i and t_v to p_j , the cost between tasks is modeled as:

$$comm(t_u, t_v) = \begin{cases} 0, & \text{if } p_i = p_j \\ \frac{data(t_u, t_v)}{bandwidth(p_i, p_j)}, & \text{if } p_i \neq p_j \end{cases}$$

where $data(t_u, t_v)$ is the volume of data to be transmitted (in bits), and $bandwidth(p_i, p_j)$ represents the effective bandwidth between the processors. This model simplifies communication analysis and supports efficient scheduling in performance-oriented heterogeneous systems.

Beyond these abstractions, a more refined communication model is required for heterogeneous systems with non-uniform interconnects. In such systems, communication delay can vary significantly between

processor pairs due to differing interconnect technologies, path latencies, and contention levels [48]. This is particularly relevant in systems where routing, congestion, or architectural heterogeneity affect the communication cost. The generalized delay between t_u on processor p_i and t_v on p_j is expressed as:

$$comm(t_u, t_v) = \begin{cases} 0, & \text{if } p_i = p_j \\ \delta(p_i, p_j, data(t_u, t_v)), & \text{if } p_i \neq p_j \end{cases}$$

where $\delta(p_i, p_j, data)$ is a delay function capturing communication heterogeneity, potentially considering factors like topology-specific delays, dynamic bandwidth sharing, and priority-based arbitration. This comprehensive model enables more accurate scheduling and analysis in modern multi-core and many-core platforms.

4.4. Performance objectives model

In modern multi-core systems, performance is no longer measured solely in terms of speed or throughput. In real-time and embedded domains, meeting *deadline constraints* remains the primary requirement. However, additional objectives such as makespan minimization, energy efficiency, thermal management, and system reliability must also be considered. This subsection discusses the typical models used to quantify and manage these objectives, which in turn guide the design of scheduling and resource management strategies.

4.4.1. Makespan

Makespan represents one of the primary measures in task scheduling, as it captures the *total time required to complete all tasks in a workload* or Directed Acyclic Graph (DAG). It is typically defined as the *maximum finishing time among all tasks*, i.e., the completion time of the exit task in the scheduled DAG [49]. In cloud and workflow scheduling, makespan is used to estimate the overall *schedule length*, and minimizing it ensures that user or system demands are met within the expected deadlines.

By contrast, *response time* is usually defined at the task or job level as the *time interval between the submission of a task and the return of its result* [50]. It consists of waiting time in the queue, communication or transmission delay (if applicable), and processing time. For locally executed tasks (e.g., on a user equipment node), transmission delay is considered zero.

Although makespan is a system-level measure and response time is a task/job-level metric, many studies in the scheduling literature use these terms interchangeably, especially when evaluating workflow completion time. To maintain consistency with prior works and simplify the discussion, we follow this convention in this survey and treat *makespan* and *response time* as interchangeable terms.

4.4.2. Energy and power model

Managing energy and power consumption is essential, particularly in energy-constrained or thermally limited environments. The total energy consumed by a processor depends on both its *dynamic* and *static* power components.

- *Dynamic power* is the result of switching activity in digital circuits and is influenced by factors such as the clock frequency and supply voltage. It can be estimated using:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f$$

where α is the activity factor, C is the effective capacitance, V is the supply voltage, and f is the operating frequency [51].

- *Static (or leakage) power*, on the other hand, is present even when the processor is idle. It can be expressed as:

$$P_{\text{static}} = I_{\text{leak}} \cdot V$$

where I_{leak} is the leakage current and V is the supply voltage.

The *total energy* consumed over a period is the integral of the combined dynamic and static power over time. These models form the basis for techniques like DVFS and DPM, which are crucial for controlling energy usage in modern multi-core platforms [52].

4.4.3. Thermal model

Temperature management is critical to prevent overheating, which can degrade performance, reduce lifespan, and even cause system failure. Thermal models help predict how the processor's temperature evolves in response to its power usage.

A widely used approach is to represent the system as a thermal RC (resistor-capacitor) circuit, where heat flow behaves similarly to electrical current [53]. In this model:

$$C_{\text{th}} \cdot \frac{dT(t)}{dt} = -\frac{T(t) - T_{\text{ambient}}}{R_{\text{th}}} + P(t)$$

Here, $T(t)$ is the temperature at time t , T_{ambient} is the ambient temperature, R_{th} and C_{th} are thermal resistance and capacitance, and $P(t)$ is the instantaneous power consumption.

Recent techniques such as machine learning-based thermal prediction (e.g., using Graph Convolution Networks) are also gaining traction for accurate and real-time thermal modeling in large-scale chips [54].

4.4.4. Reliability and fault model

Reliability in real-time embedded systems is the likelihood that the system still operates normally within a given time frame, provided that it begins fault-free [29]. As semiconductor technology is shrinking and integration densities are growing, contemporary embedded systems are becoming more vulnerable to all kinds of faults, such as transient faults due to radiation or voltage variations, intermittent faults due to fabrication defects or environmental changes, and permanent faults due to wear-out mechanisms and aging effects. These faults can weaken the functional correctness of the system and lead to deadline misses, which are especially vital in safety and mission critical applications.

In multi-cores, individual processing cores may encounter transient failures, which are typically modeled using the *Poisson distribution* [55]. Usually, the *reliability* of an event in unit time t is denoted by $R(t) = e^{-\lambda t}$, where λ is the *constant failure rate per unit time* for a processing core. The reliability of a task T_i assigned onto the processing core P_k over the duration of its execution time $C_{i,k}$ is denoted by

$$R(T_i, P_k, C_{i,k}) = e^{-\lambda_k C_{i,k}}$$

$R(T_i, P_k, C_{i,k})$ denotes the probability of executing T_i on P_k without any failures. From the above equation, it holds that $R(T_i, P_k, C_{i,k}) \in (0, 1]$. The probability that the execution of T_i on P_k fails is $1 - R(T_i, P_k, C_{i,k}) \in (0, 1]$.

To ensure needed levels of reliability, fault-tolerance methods are implemented on several levels of system design. The methods bring about various types of *redundancy*: *Hardware redundancy* duplicates critical elements to hide or detect faults; *Software redundancy* uses techniques like *check pointing* and recovery blocks to regain correct execution; and *Time redundancy* permits re-execution to handle transient errors. Yet, incorporating fault-tolerance mechanisms adds extra execution time, energy, and power overheads, which have to be traded against timing and resource constraints. Reliability modeling hence has to take account of task criticality (differentiating between hard, firm, and soft real-time tasks) as well as the targeted fault and redundancy characteristics of the application.

A more detailed approach to modeling transient faults is provided by *Markov models*, which describe systems transitioning between discrete states. They are particularly effective for transient faults, capturing their time-dependent nature. For example, transitions like healthy $\rightarrow$ faulty $\rightarrow$ healthy. Markov models also allow analysis of system reliability and availability through steady-state probabilities, such as the likelihood of the system being in an available state at any given time. While Markov models characterize fault behavior, *reinforcement learning* can be employed to manage it adaptively. An RL agent learns a policy (a set of rules for selecting actions in each state) to maximize a cumulative reward. In fault-tolerant systems, this translates to maximizing performance and availability while minimizing the cost of fault mitigation.

The reliability models and adaptive management approaches discussed above facilitate the design of resource management and scheduling policies that jointly optimize reliability, performance, power, and thermal objectives in multi-core systems.

5. Scheduling policies and decision strategies

Many forms of task scheduling, particularly those involving multiple processors, deadlines, or resource constraints, are NP-hard problem, since no polynomial-time algorithm exists to solve it optimally. To obtain suboptimal yet practical solutions, various artificial intelligence algorithms have been proposed. Selecting effective scheduling policies is crucial in multi-core systems to ensure timely task execution and adherence to constraints like deadlines. These policies determine the order in which tasks are executed across the system's processing units.

While optimal scheduling algorithms guarantee the best possible schedule, they are computationally expensive and often impractical for large-scale systems [56]. In contrast, heuristic algorithms—although not guaranteed to find the optimal solution—offer faster and more scalable alternatives, making them better suited for dynamic and complex workloads [57]. In simpler systems with well-defined workloads, optimal scheduling may still be feasible. In many cases, hybrid approaches that combine the strengths of both optimal and heuristic methods can yield effective results [58]. The classification of scheduling algorithms is based on the framework provided in [59], which categorizes them into three broad groups: *mathematical optimization*, *heuristic-based*, and *machine learning-based* approaches. Fig. 4 provides a comprehensive overview of this classification for task scheduling algorithms. Mathematical optimization methods such as *Mixed Integer Linear Programming* [5,60], *Branch and Bound* [10,58], and the *Simplex algorithm* [61] provide exact solutions but are often computationally prohibitive. Heuristic-based algorithms are typically categorized into *list scheduling*, *clustering*, and *task duplication* strategies [57]. List scheduling approaches include Levelized-Min Time [62], Dynamic Level Scheduling [63], HEFT [47], IPPTS [64] etc. Clustering-based algorithms such as CMWSL [65], RAC [66], and HYB [67] have also shown promising results. Algorithms like LSTD and TDCA are task duplication algorithms [47,68,69] that provide near-optimal scheduling by considering metrics such as earliest finish time and critical path length. Meta-heuristic methods use population-based or stochastic search to explore large design spaces. Popular examples include Genetic Algorithms (GA) [70], Particle Swarm Optimization (PSO) [18], Ant Colony Optimization (ACO) [19], Simulated Annealing (SA) [17]. Heuristic algorithms are problem-specific rule-based strategies designed to obtain quick, near-optimal solutions for a given scheduling instance, such as Min-Min, Max-Min, or Critical Path methods. In contrast, *meta-heuristic algorithms* such as GA, PSO, ACO, and SA are higher-level, general-purpose frameworks that can be adapted to diverse optimization problems. They rely on stochastic or population-based search mechanisms to balance exploration and exploitation across large design spaces. While heuristics are computationally efficient but task-specific, meta-heuristics provide broader applicability at the cost of increased computation, making them suitable for complex, NP-hard scheduling scenarios in heterogeneous systems.

Machine learning-based methods aim to optimize energy and power efficiency in scheduling by learning patterns in task execution and resource utilization. Classical machine learning techniques, including Random Forest [71], Decision Trees [72,73], Support Vector Machines (SVMs) [74,75], and k-Nearest Neighbors (k-NN) [76,77], have been used for task classification and scheduling decisions. *Deep learning* further enhances this capability by handling complex nonlinear relationships and large datasets [78]. Techniques such as Recurrent Neural Networks (RNNs) [79], Graph Neural Networks (GNNs) [80,81], and Transformers [82,83] are increasingly being applied to model intricate task dependencies and improve scheduling in dynamic environments. *Reinforcement Learning-based* frameworks like Decima [84],

DeepWeave [85], DREAM [86], and Q-Learning [87] enable adaptive scheduling by learning policies from environment feedback. These approaches are particularly effective in heterogeneous and dynamic systems where workloads and resource availability are unpredictable. *Decima* employs a graph neural network to represent job dependencies and a policy network to guide scheduling decisions, aiming to optimize cluster-level job processing. *DeepWeave* accelerates job completion in distributed systems by using deep RL for co-flow scheduling, minimizing network congestion. *DREAM* generalizes stream processing system resource allocation by applying RL to respond to fluctuating workloads. Within this category, *Deep Reinforcement Learning (DRL)* has emerged as a specialized subfield that combines neural network function approximation with RL to address high-dimensional and large-scale scheduling problems. For instance, Mo et al. [88] proposed a GAT-based DRL scheduler for dependent DAG tasks on homogeneous multi-core platforms, capturing inter-task relationships via graph attention. Similarly other DRL-based works leverage Proximal Policy Optimization (PPO) [89], Deep Q-Networks (DQN) [90], and Advantage Actor-Critic (A2C) [91] to dynamically adapt task allocation in multi-core and distributed platforms.

Ultimately, the choice of scheduling policy is influenced by system requirements, task characteristics, and resource availability. A well-suited scheduling strategy can significantly impact the system's overall performance, responsiveness, and reliability.

5.1. Learning methods

Learning-based methods enable multi-core systems to adapt to dynamic execution environments by predicting workload characteristics, optimizing scheduling decisions, and balancing multiple objectives such as makespan, energy, thermal stability, and reliability. These methods provide responsiveness, scalability, and robustness under uncertain and time-varying execution conditions. Based on the availability of labeled data, the structure of input signals, and the degree of environmental uncertainty, learning approaches are broadly categorized into *Machine Learning*, *Deep Learning*, *Reinforcement Learning*, and *Deep Reinforcement Learning*. These categories are described below.

5.1.1. Machine learning

Machine learning (ML) techniques have been widely applied in scheduling for multi-core and heterogeneous systems. They allow systems to learn from data and make predictions that help improve execution time, energy efficiency, and reliability.

Supervised learning [75,96] relies on labeled data to build predictive models. Classical techniques such as linear regression, decision trees, and support vector machines are commonly used to estimate execution time, energy consumption, or resource usage, thereby guiding schedulers to meet deadlines and improve efficiency. They are also applied to reliability analysis, where labeled sensor data enable classifiers to detect and isolate faults in real-time. Unsupervised learning [96], in contrast, relies on unlabeled data. Approaches such as clustering (e.g., k-means) and dimensionality reduction (e.g., PCA, autoencoders) reveal hidden patterns in workload traces, aiding anomaly detection, workload grouping, and system balancing. In scheduling, these methods help classify tasks with similar behavior (e.g., CPU-bound vs. I/O-bound), which reduces contention and improves utilization in heterogeneous platforms. Together, supervised and unsupervised methods provide predictive and structural insights that enhance scheduler adaptability in dynamic multi-core systems.

5.1.2. Deep learning

Deep Learning (DL) extends ML by using multi-layered neural networks that can learn complex patterns from large amounts of data. These methods are especially useful for capturing relationships in system workloads and task dependencies, making them effective on both homogeneous and heterogeneous systems.

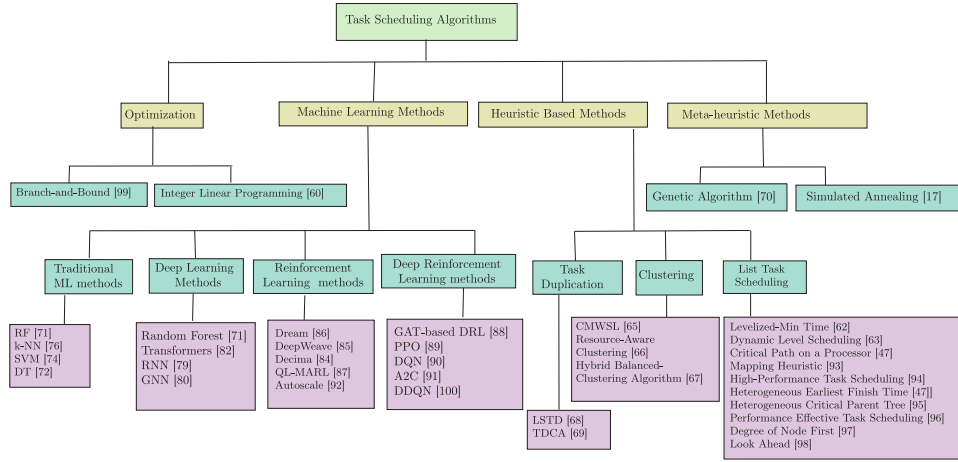


Fig. 4. Taxonomy of task scheduling algorithms. See [92–95].

- **Deep Neural Networks (DNNs):** General multi-layer networks [97] that can capture nonlinear patterns in system data, such as predicting execution time or energy use more accurately than shallow models.
- **Convolution Neural Networks (CNNs):** Originally designed for images, but in scheduling they analyze patterns in performance traces over time and across cores [98].
- **Long Short-Term Memory (LSTM) networks:** A type of recurrent neural network (RNN) that is good at learning from time-series data. For scheduling, LSTMs forecast energy consumption or workload demand over time [99].
- **Transformers:** Sequence models that can analyze system logs or workload sequences for prediction. Recent works also integrate Transformer-based encoders for scheduling; for instance, Wang et al. [100] employed a Set Transformer along with a spatio-temporal GNN to jointly capture DAG task dependencies and heterogeneous cluster states, improving both makespan and load balancing.
- **Graph Neural Networks (GNNs):** Networks designed for graph-structured data. In scheduling, they capture dependencies in task graphs (DAGs) or represent multi-core systems as interconnected nodes, improving scheduling decisions [81].

These deep learning methods improve scheduling by better understanding system behavior, predicting future demands, and grouping similar workloads. However, they require significant computational resources for training and inference, which can be a limitation in real-time or resource-constrained environments.

5.1.3. Reinforcement learning

Reinforcement Learning (RL) [101] enables systems to learn scheduling policies through trial-and-error interaction with their environment. Agents receive rewards for desirable outcomes (e.g., meeting deadlines, reducing power) and penalties for undesirable ones (e.g., missed deadlines, overheating). This makes RL well-suited to dynamic scenarios where system conditions evolve unpredictably.

In scheduling, RL has been widely applied to DVFS, where agents adjust processor voltage and frequency in real-time to balance energy efficiency and performance [102]. Both value-based methods such as Q-learning [103,104] and policy-based approaches like actor-critic [105] are common. A key theoretical foundation is the *Bellman optimality equation*, which recursively defines the optimal state-action value:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right],$$

where $Q^*(s, a)$ is the *optimal action-value function* at state s by taking action a , r is the *immediate reward*, γ the *discount factor*, and s' the *next*

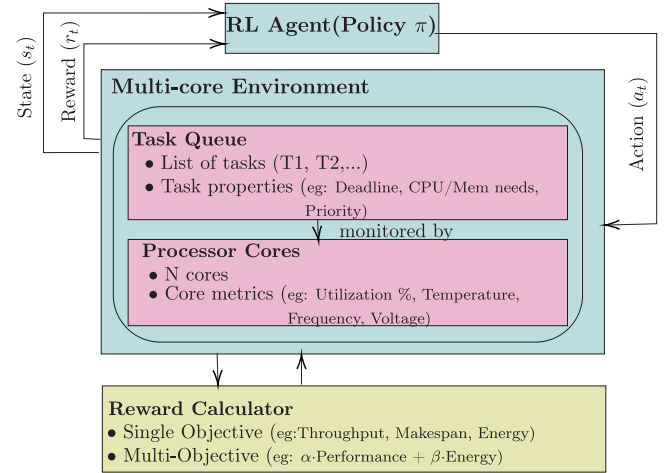


Fig. 5. RL-based scheduling system with agent-environment-reward feedback loops for multi-core processors. The agent observes system state, selects scheduling actions, and receives feedback based on timing constraints and performance metrics.

state. Iterative methods such as Q-learning leverage this formulation to converge toward optimal scheduling policies.

Practical applications include thermal-aware scheduling, where RL agents allocate tasks to prevent hotspots, and multi-agent RL for fault-tolerance, where workloads are redistributed under hardware failures. In cloud environments, RL has been used for energy-efficient auto-scaling and SLA-aware resource allocation [106,107].

An abstract RL scheduling framework is shown in Fig. 5. Here, the agent observes system states such as task requirements, deadlines, utilization, and thermal conditions, then selects actions including task-to-core assignments and DVFS adjustments. Rewards capture objectives like makespan reduction, energy efficiency, or multi-objective trade-offs. Through this closed-loop interaction, the agent iteratively improves its policy to handle the system constraints and dependencies described in Section 4.

5.1.4. Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) combines RL with deep neural networks, enabling agents to handle high-dimensional state and action spaces. Unlike traditional RL, which relies on tabular updates, DRL uses neural approximators to estimate value functions or policies. For

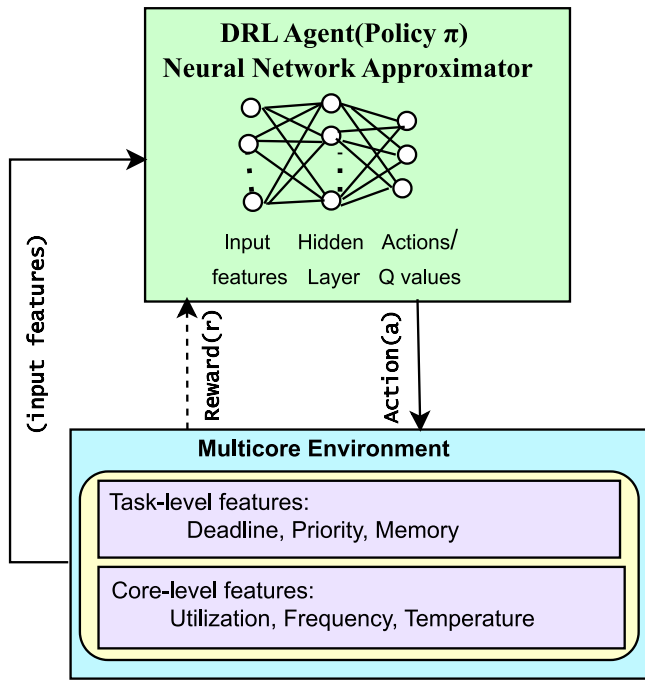


Fig. 6. Deep Reinforcement Learning framework for multi-core task scheduling.

instance, Deep Q-Networks (DQN) learn $Q(s, a)$ values, while actor-critic methods jointly learn policies and value baselines, improving scalability and stability.

Architectures such as DQN [108] and Proximal Policy Optimization (PPO) [89] have been applied to real-time resource management. Applications include DRL-based DVFS for energy–deadline trade-offs in embedded systems [109], attention-based schedulers for DAGs on multi-cores [88,105], and multi-objective DRL for workflow scheduling that balances response time and cost [16]. Surveys such as [110] classify DRL into *value-based*, *policy-based*, and *actor-critic* families, noting their role in load balancing [111], auto-scaling [112], and dynamic provisioning [91] across cloud and edge platforms.

Fig. 6 illustrates the DRL scheduling framework. The agent uses deep neural networks to process task-level features (e.g., deadlines, priorities, memory) and core-level features (e.g., utilization, frequency, temperature). Depending on the architecture, the network approximates either the action-value function $Q(s, a)$ (value-based) or both policy $\pi(a|s)$ and value baseline $V(s)$ (actor-critic). Actions include task-to-core mapping or DVFS control, with rewards reflecting makespan, energy, or thermal balance. Iterative training enables the agent to capture complex dependencies and adapt to dynamic system conditions.

In the following sections, we present the significant contribution of this survey work on analyzing recent works that apply ML, DL, RL, and DRL techniques to task scheduling. These studies address critical performance objectives [113] such as *energy efficiency*, *thermal management*, *fault-tolerance*, and *makespan minimization*. Learning-based approaches are increasingly adopted to balance these often conflicting goals, enabling efficient operation under stringent performance constraints. In particular, multi-objective optimization introduces inherent trade-offs not present in single-objective scheduling. For example, minimizing energy consumption may increase execution time, while strict thermal management can limit system throughput. Independent task models offer greater flexibility in balancing such objectives, whereas dependent tasks introduce additional constraints that make trade-offs more complex. Learning-based approaches, particularly DRL, are well suited to navigate these conflicts, dynamically adjusting scheduling policies to achieve an effective balance between energy, performance, thermal,

and reliability goals. By explicitly analyzing these trade-offs [114, 115], this survey provides guidance on how learning-based schedulers can reconcile competing objectives in heterogeneous and dynamic computing environments.

While the preceding subsections reviewed the major scheduling approaches (mathematical optimization, heuristics/meta-heuristics, ML, DL, RL and DRL) individually, a direct comparison among them highlights their relative applicability under different system characteristics. Table 4 provides a consolidated comparison that summarizes each method's typical application context, suitability to key system factors introduced in Section 4 (task dependencies, communication delays, and platform heterogeneity), and their respective strengths and limitations. This comparative perspective clarifies why recent research trends are shifting from traditional optimization and heuristics toward more adaptive, learning-based scheduling frameworks.

6. Machine learning-based works

Machine learning has emerged as a powerful paradigm for scheduling, offering predictive and adaptive strategies that go beyond static heuristics. By training on system metrics, workload traces, and runtime behaviors, ML-based schedulers can anticipate performance trends and dynamically adjust task-to-core mappings. Such models have been shown to reduce execution time, improve energy efficiency, and enhance thermal and reliability outcomes. However, their effectiveness often comes at the cost of additional training and model maintenance overhead. As compared to more advanced models in DL, RL, and DRL, ML-based approaches offer several advantages. They are generally simpler to design and train, require smaller datasets, and incur lower computational overhead during inference. This makes them particularly suitable for systems with limited hardware resources or real-time constraints, where fast decision-making is critical. Moreover, ML models are easier to interpret, allowing system designers to trace scheduling decisions back to learned patterns, which is often difficult with deep or reinforcement learning methods. To provide clarity, this section organizes ML-based scheduling contributions according to their *optimization objectives*.

6.1. Makespan minimization

In contrast to energy efficiency, thermal management, and reliability objectives, only a limited number of machine learning-based scheduling works have directly addressed the optimization of *makespan* (i.e., minimizing task completion time or improving throughput). Makespan minimization is critical in heterogeneous systems, where reducing overall execution time directly translates to better resource utilization and quality of service. A recent study by [121] introduced a *dynamic task mapping scheme* based on machine learning for *independent tasks on heterogeneous big.LITTLE architectures*. The scheme was specifically designed to minimize *makespan* and improve throughput, particularly in the presence of workload phase variations. To achieve this, the authors designed a three-stage pipeline. In the first stage, a phase-detection model partitioned program execution into distinct intervals, allowing runtime variability to be effectively captured. The second stage employed an IPC (Instructions Per Cycle) based performance prediction model to estimate task execution behavior on both big and LITTLE cores. Finally, the third stage applied a heuristic mapping algorithm that leveraged these predictions to distribute tasks across N big and M LITTLE cores with the goal of maximizing overall performance. Experimental evaluation on a heterogeneous platform demonstrated that this phase-aware and predictive strategy achieved lower completion times and higher throughput when compared to baseline schedulers. The advantage of the approach lies in its ability to dynamically adapt task-to-core mapping in response to workload behavior. However, its reliance on IPC features and training data introduced sensitivity to profiling quality.

Table 4

Comparative summary of scheduling approaches highlighting their applicability to key system characteristics (task dependencies, communication delays, and platform heterogeneity).

Method/Application context	Task model	Communication	Heterogeneity	Key strengths/Limitations
Mathematical optimization [5,10] (used in small-scale or safety-critical systems)	Precisely models task dependencies (DAG); suited for small static workloads	Communication modeled precisely but increases complexity	Explicitly encoded via constraints	Strengths: Optimal or provable solutions. Limitations: Exponential complexity; poor scalability for large/dynamic workloads.
Heuristics [57]/Meta-heuristics [14] (for large DAGs and near-real-time mapping)	Handles large DAGs with approximate ordering	Approximates communication via penalties or heuristic costs	Adaptable via cost/fitness functions	Strengths: Fast, scalable, and practical. Limitations: Requires parameter tuning; weak optimality guarantees.
Supervised ML [116] (used in runtime or energy prediction for hybrid schedulers)	Limited ability to capture DAG dependencies unless explicitly encoded through features	Learns communication effects if present in training data	Heterogeneity represented in input features	Strengths: Fast inference; accurate prediction once trained. Limitations: Needs labeled data; limited generalization to unseen DAGs.
Deep Learning (DL) [99,117] (for complex workload modeling and feature extraction)	Captures implicit dependencies through neural representations	Partially models communication via learned embeddings	Learns platform-specific variations from data	Strengths: Automatically extracts nonlinear task-core relationships; good for large-scale data. Limitations: High training/inference cost; large data requirement; limited interpretability.
Reinforcement [118]/DRL [119] (for dynamic scheduling in heterogeneous systems)	Learns precedence via sequential decision states	Models communication if simulator captures it; training is computationally expensive	Learns heterogeneity handling via experience	Strengths: Adaptive end-to-end policies for dynamic environments. Limitations: High training cost; convergence instability.
Hybrid [120] (for practical adaptive systems combining heuristic and learning-based policies)	Combines heuristic ordering with learned predictions or policies	Integrates learned comm. models with rule-based placement	Merges deterministic and learned decision rules	Strengths: Balanced trade-off between quality and speed. Limitations: Integration complexity; design and tuning effort.

6.2. Energy efficiency

Energy-aware scheduling has become increasingly vital, especially for embedded, mobile, and edge systems where power constraints are critical. ML techniques are extensively applied to design intelligent DVFS/DPM controllers and predict energy-optimal task-core mappings. These models leverage historical workload features, runtime metrics, and hardware counters to guide scheduling or power control actions.

6.2.1. Homogeneous platforms

Hoffmann and Fröhlich [122] proposed the *Online-Learning Energy-Aware multi-core Real-Time Scheduler (OL-EAMRTS)*, a lightweight ANN-based framework for *independent task allocation* on homogeneous multi-core systems. The scheduler integrated DVFS with task migration, relying on low-overhead sampling of *Performance Monitoring Unit (PMU)* counters (hardware registers within the processor that track events such as executed instructions, CPU cycles, and cache behavior). These counters provided real-time utilization features that were fed into a warm-start ANN model pre-trained on synthetic benchmarks. Using these inputs, the ANN predicted post-DVFS utilization, enabling safe frequency reductions without violating deadlines. In parallel, a migration heuristic redistributed tasks to balance load and mitigate resource contention. Experimental results on ARMv7 platforms demonstrated notable energy savings of 30.30% and 14.81% across two case studies. However, the framework showed degraded performance when faced with unseen workloads, where warm-start predictions were inaccurate, leading to missed DVFS opportunities. To address this limitation, the same authors extended their work in [123] by adding a robust unlocking mechanism that reactivated online learning in warm-start failure cases. This enhancement used migration-based re-balancing, safety margin calculation from activity vectors, and controlled one-step DVFS reduction to trigger adaptation. Evaluated on ARMv8 Cortex-A53, it retained low overhead while achieving average energy savings up to 33.30% and successfully recovering from all tested failures. Although effective, the solution still suffered from high initial prediction error

before adaptation, reliance on architecture-specific tuning, and limited evaluation under mixed-criticality workloads. A distinct approach was provided in [124], which introduced a *Sparse Relevance Kernel Machine (SRKM)* model for *independent tasks*. While conceptually related to Support Vector Machines (SVMs) as both are kernel-based learners, SRKM achieved higher efficiency by enforcing sparsity in both the sample and feature space, thereby producing lightweight models suitable for on-chip hardware prediction. Targeting multi-core processors with Heterogeneous Voltage Regulation (HVR), the SRKM leveraged instruction-level workload metrics (e.g., IPC and cache misses) to predict optimal regulator configurations across multiple stages. This fine-grained workload-aware adaptation enabled precise voltage tuning and achieved up to 23.9% energy savings on SPEC CPU2006 benchmarks. However, compared to classical SVMs, which were more mature and widely adopted, SRKM still incurred inference latency and depended heavily on detailed workload statistics, which may limit its scalability in highly dynamic runtime environments.

6.2.2. Heterogeneous platforms

Mandal et al. [125] presented an imitation learning (IL)-based resource management framework for *independent tasks* on heterogeneous mobile platforms. Leveraging the DAGger algorithm, their approach learned policies that dynamically control the number, type (Big/Little), and operating frequencies of active cores, closely approximating near-optimal Oracle strategies. Evaluations on an Odroid-XU3 (Exynos 5422 big.LITTLE SoC) with 19 benchmark applications demonstrated that the IL-based policies nearly doubled the *Performance per Watt (PPW)* relative to the default interactive governor. While highly efficient, the framework depended on offline profiling to construct Oracle policies, which could limit adaptability under unpredictable runtime variations. Kim et al. [126] addressed scheduling for *dependent task graphs* by employing an SVM-based execution time prediction model. The predicted run-times were then used to guide DAG-aware task-to-core mapping under real-time constraints. The proposed method followed a *LITTLE-first assignment policy* i.e., if a task can complete within its deadline

on a LITTLE core, it is scheduled there, otherwise, it is escalated to a big core. This deadline-aware heterogeneous assignment strategy, implemented on a Samsung Galaxy S7 edge (quad Cortex-A53 LITTLE + quad Exynos M1 big), was enhanced by SVM-based completion time prediction that reduced deadline misses and improves mapping accuracy. Leveraging core-specific predictions, their scheduler achieved up to 51.8% energy savings compared to the default ARM big.LITTLE Heterogeneous Multi-Processing (HMP) baselines. A key limitation, however, was that the energy savings diminished under high parallelism degrees, since increased concurrency forced greater reliance on big cores and reduced the comparative benefit over legacy scheduling.

Overall, the distinction between homogeneous and heterogeneous platforms highlights different design considerations. Homogeneous systems often emphasize low-overhead online learning and fine-grained DVFS/DPM strategies, whereas heterogeneous systems explore cross-core coordination, imitation learning, and specialized applications. A recurring challenge across both classes is balancing predictive accuracy with runtime adaptability, particularly under dynamic or mixed-criticality workloads.

6.3. Multi-objective optimization

Many real-world systems must balance multiple conflicting objectives. ML-based multiobjective schedulers model trade-offs across execution time, energy, temperature, and reliability, often by generating a set of candidate solutions where improving one objective (e.g., minimizing makespan) may come at the cost of another (e.g., higher energy use). Such approaches provide flexibility to system designers, who can then select the most suitable compromise based on the current performance priorities and operating constraints.

Zhang et al. [127] proposed *Hot-Trim*, a dynamic thermal and reliability management framework for *independent tasks*. The method employed a Multilayer Perceptron (MLP) trained on Intel performance counter metrics to estimate intra-core power density hot spots, which were then used to guide task mapping and migration. Experiments on an Intel Core i7 quad-core processor showed that, compared to the default Linux scheduler (Completely Fair Scheduler, CFS) and a temperature-based mapping scheme, Hot-Trim reduced core and hot spot temperatures by 1.15–1.31 °C, while enhancing reliability by up to 31.1% for Hot Carrier Injection (HCI), 30.2% for Electro-migration (EM), and 7.0% for Negative Bias Temperature Instability (NBTI). Importantly, these improvements were achieved without additional energy overhead, underscoring its ability to jointly optimize thermal stability and device lifetime. However, reliance on user-space implementation and possible inaccuracies in hot spot estimation may limit its applicability in stringent real-time environments. In another work, Gamatié et al. [75] investigated energy-aware scheduling of *dependent DAG tasks* on multi-core architectures. Their empirical ML methodology applied Support Vector Machines (SVM), AdaBoost, and Artificial Neural Networks (ANNs) to predict execution time and energy consumption across alternative task-to-core mappings. To evaluate prediction quality, the authors introduced the *Percentage of Successful Tendency Prediction (PSTP)* metric, which captured the correctness of relative performance rankings between mappings rather than absolute prediction errors. Applied to the automotive *DemoCar* benchmark, the models achieved PSTP scores of up to 84.8% with AdaBoost and 89.05% with ANN.

Pivezhandi et al. [116] proposed a statistical learning framework for *independent task scheduling* on heterogeneous platforms. Their methodology emphasized feature selection to identify the most influential parameters such as core type, speed, temperature, and memory intensity that drive energy and thermal behavior. A multi-stage pipeline integrated filter-based correlation analysis, wrapper-based backward stepwise regression, and embedded random forest selection techniques to refine predictors for task-to-core allocation. A compressed regression model with bootstrapping achieved efficient environment modeling while lowering complexity. Experimental results on Intel Core i7

(12th Gen, 14 cores), Intel Xeon, and Jetson TX2 platforms using the Barcelona OpenMP Tasks (BOTs) suite demonstrated up to 10% energy savings, a 5 °C reduction in core temperature, and a 61.6% reduction in prediction error compared to prior thermal models. The framework highlighted the promise of lightweight, feature-aware regression methods for balancing energy and thermal objectives across diverse platforms. Rahmani et al. [128] proposed a machine learning-based framework for energy-efficient load balancing in heterogeneous multi-node CPU–GPU clusters. Task execution times were modeled using supervised regressors trained on 18 OpenCL kernel features, extracted through the *Low-Level Virtual Machine (LLVM)* compiler infrastructure (which provides code-level features from intermediate representation) along with regex-based parsing. Using these predictions, the framework defined the *load* of each device as the cumulative predicted execution time of its queued tasks, and scheduled each new task to the Least-Loaded Device (LLD). To evaluate workload distribution, the authors introduced the *IMBALANCE* metric, defined as the total difference in load across all devices, where a near-zero value indicates an evenly balanced system. Experimental results on PolyBench benchmarks across CPU and GPU nodes showed that LLD achieved consistently low imbalance, reduced finishing time by 2.1–6.3× compared to round-robin, weighted round-robin, least-connections, and device-suitability heuristics, and lowered energy consumption by 35%–75% depending on workload type. Although the framework incurred overhead due to per-device execution time prediction, it provided a device-agnostic, online scheduling strategy that jointly optimizes energy efficiency, response time, and load balancing in distributed heterogeneous clusters.

Table 5 provides a comparative summary of the ML-based works, categorized by task model, hardware platform, optimization objective, methodology, results, and limitations. Dependent-task scheduling on heterogeneous platforms is still underexplored in ML-based multiobjective research, as most works emphasize independent tasks. While ML approaches have improved energy efficiency and balanced trade-offs across thermal stability, reliability, and execution speed, they remain limited by reliance on offline training, handcrafted features, and weak adaptability at runtime. In contrast, recent DL, RL, and DRL approaches directly addressed these gaps. DL automatically learns complex workload patterns and DAG dependencies without manual feature engineering. RL supports sequential decision-making for dynamic and uncertain environments whereas DRL integrates both to scale across high-dimensional state–action spaces. Collectively, these paradigms overcome the static nature of classical ML by enabling online adaptation, cross-platform scalability, and direct optimization of latency-sensitive objectives, motivating their growing role in scheduling research.

7. Deep learning based methods

Deep Learning techniques, particularly deep neural networks, are well-suited for addressing complex scheduling problems due to their ability to capture high-dimensional, nonlinear relationships and to automatically extract meaningful patterns from workload and system data. Unlike traditional heuristics or machine learning models, DL approaches can generalize across diverse execution contexts and handle both independent and dependent task models on homogeneous as well as heterogeneous platforms with minimal manual feature engineering. In scheduling research, DL has been employed to map task and system features directly to scheduling decisions, supporting a variety of optimization goals. Recent advances in neural architectures further broaden the applicability of DL in scheduling. For instance, Convolution Neural Networks effectively learn from workload traces and system metrics, while Graph Neural Networks capture structural dependencies in directed acyclic graphs (DAGs). This section reviews representative DL-based scheduling methods, organized by their optimization objectives. Most DL-based scheduling contributions target heterogeneous systems, where the diversity of resources highlights the representational power of deep models.

Table 5

Summary of machine learning-based works grouped by task model with objectives: Energy (E), Temperature (T), Fault and Reliability (F), Makespan (M), and Deadline (D).

Ref.	Learning type	Hardware platform	E	T	F	M	D	Methodology summary	Evaluation & Results	Limitations
Independent tasks										
[122]	ANN (Online learning)	Homogeneous – ARMv7 multi-core	✓	×	×	×	×	Online-Learning Energy-Aware multi-core Real-Time Scheduler (OL-EAMRTS); uses ANN to predict post-DVFS utilization and combines with task migration	Energy savings of 30.30% and 14.81% with DVFS + migration	Warm-start mis-predictions under unseen workloads; limited generalization
[123]	ANN (Online learning, Adaptive)	Homogeneous – ARMv8 Cortex-A53 multi-core	✓	×	×	×	×	Extension of OL-EAMRTS with unlocking mechanism; safety margin + migration-based re-balancing to re-enable online learning after warm-start failures	Average energy savings of 17.30% (up to 33.30%); successful recovery from all failure cases	High initial prediction error; requires architecture-specific tuning; limited evaluation under mixed-criticality
[124]	Sparse relevance kernel machine	Homogeneous – multi-core with Heterogeneous Voltage Regulation (HVR)	✓	×	×	×	×	Predicts optimal voltage supply config using instruction-level workload metrics	Up to 23.9% energy saving on SPEC CPU2006	Inference latency, requires fine-grained workload stats
[127]	MLP	Homogeneous – Commercial multi-core (Intel)	×	✓	✓	×	×	Predicts intra-core power density; thermal-aware task migration	1.15–1.31 °C temp drop; reliability improved up to 31.1%	User-space impl., estimation errors, no RT constraints handling
[125]	Imitation learning (Dagger)	Heterogeneous – ARM big.LITTLE mobile (e.g., 4 big + 4 LITTLE cores)	✓	×	×	×	×	Learns energy-optimal execution policies; adjusts core type/ number/ frequency dynamically	101% PPW improvement over Android DVFS governor	Offline training, workload-specific profiling, runtime variability sensitivity
[128]	Extra trees, Gradient boosting	Heterogeneous – CPU-GPU clusters (multi-node)	✓	×	×	✓	×	LLD scheduler predicts per-device execution time from OpenCL kernel features; introduces IMBALANCE metric for load distribution	Shortest finishing time (2–6× faster) and lowest energy (35%–75% reduction) vs. baselines	Offline-trained models; focuses on efficiency, not hard real-time deadlines
[116]	Compressed regression	Heterogeneous – Intel Core i7 12th Gen (8P + 6E cores), validated also on Intel Xeon & Jetson TX2	✓	✓	×	×	×	Predicts energy/thermal-efficient core mappings using feature selection (correlation, backward stepwise, RF) and compressed regression	10% energy saving, 5 °C thermal drop; 61.6% MSE reduction vs. SOTA	Limited to offline prediction, no real-time adaptation
[121]	IPC-based Prediction + Heuristic Mapping	Heterogeneous – ARM big.LITTLE multi-core	×	×	×	✓	×	Phase-aware task mapping: (1) phase detection, (2) IPC-based core performance prediction, (3) heuristic mapping across big/LITTLE cores	Reduced completion time and improved throughput vs. baseline schedulers	Dependence on IPC features and training data; no energy/thermal consideration
Dependent tasks										
[126]	SVM	Heterogeneous – ARM big.LITTLE (e.g., 4 big + 4 LITTLE cores)	✓	×	×	×	×	Predicts core-specific execution time for DAG tasks; maps tasks to cores for energy savings	Up to 51.8% energy saving vs. HMP baselines	Offline profiling, limited adaptability to new workloads/hardware
[75]	SVM, AdaBoost, ANN	Homogeneous – multi-core (automotive DemoCar)	✓	×	×	✓	×	Models mapping performance; PSTP metric for ranking correctness	PSTP up to 89.05% (ANN), accurate mapping performance	High training cost, app-specific graph structure

7.1. Makespan minimization

DL approaches addressing *makespan minimization* typically map task characteristics to scheduling decisions that reduce overall execution time. Common architectures include CNNs and GNNs, which effectively capture structured task dependencies. Notably, most DL-based makespan optimization works target *heterogeneous platforms*, where resource diversity highlights the advantages of neural models, while little progress has been reported for homogeneous systems.

Neelakantan and Yadav [98] proposed the Whale-based Convolution Neural Framework (WbCNF) for job scheduling in *independent task* scenarios on heterogeneous cloud platforms, targeting the minimization of *makespan*. The framework integrated the Whale Optimization Algorithm (WOA) into the classification layer of a Convolution Neural Framework (CNF), where tasks were prioritized based

on predicted deadlines (short-deadline tasks receive higher priority). Experimental evaluations demonstrated substantial makespan reductions, with WbCNF completing 500–3500 tasks in only 80–120 s, compared to 150–700 s under heuristic schedulers such as TaSLTS [129], LB-AD [130], PSO [18], and IWDO [131]. These results confirmed the framework's effectiveness in reducing overall completion time by coupling meta-heuristic optimization with neural modeling.

Wu et al. made two notable contributions toward optimizing scheduling performance in heterogeneous computing systems. In [120], they proposed the *End-Edge-Cloud Heterogeneous Resource Scheduling Method (EHRSM)*, which combines an RNN-based adaptive batching mechanism with PSO-based global scheduling. This hybrid framework converts online task arrivals into offline batches, reducing total completion and waiting times across CPU-GPU-NPU resources and achieving faster response with lightweight computation. In a related

direction, Wu et al. [132] introduced an *Edge Server-Oriented Deep Neuro-Fuzzy System (ESODNFS)* that employs parallel ANFIS units and a DNN-based weighting module for accurate task runtime estimation under heterogeneous edge server environments. Although primarily designed for runtime prediction, ESODNFS provides essential input for schedulers like EHRSM, enabling more informed and adaptive task allocation decisions that ultimately contribute to minimizing makespan and improving resource utilization.

Johnston et al. [81] extended DL methods to *dependent* workloads by presenting *IRIS-GNN*, a GNN-based hybrid scheduling policy integrated into the Intelligent Runtime System (IRIS). IRIS-GNN performed graph classification of DAGs into categories such as *locality*, *concurrency*, and *mixed*, using synthetic workloads generated by DAGGER. Based on classification, it dynamically selected the most suitable scheduling policy (e.g., Depend, Round Robin, First-to-Finish), eliminating the need for manual policy selection. Evaluation on a 7-GPU AMD EPYC Milan2 system demonstrated 96.4% *classification accuracy* and performance competitive with the best dynamic policies. Further extending GNNs, Alshaer et al. [114] proposed a meta-scheduling framework for adaptive time-triggered multi-core architectures (ATMA). A Genetic Algorithm was first used offline to generate multi-schedule graphs (MSGs), which then trained a supervised GNN to predict temporal task priorities at runtime. This enabled schedule reconstruction under slack, failure, or mode changes, while preserving precedence and safety constraints. Results showed near-GA performance for makespan and significant improvements over list scheduling, offering a practical trade-off between adaptability and efficiency for real-time heterogeneous systems. A related perspective was provided by Lua et al. [133], who framed scheduling as a link prediction problem in time-triggered systems. Jobs, processors, and switches were modeled as nodes in a heterogeneous graph, with dependencies and allocations as edges. Trained on GA-generated schedules, the GNN predicted missing job-processor links to reconstruct feasible schedules at runtime. On 10- and 40-job models, prediction accuracy reached 41%, with larger graphs showing improved results. Although accuracy lagged GA baselines, the framework demonstrated runtime adaptability without requiring pre-stored schedules, highlighting the promise of GNN-based allocation in dynamic environments.

These works demonstrated how DL-based scheduling for heterogeneous platforms has evolved from CNN-based hybrids for independent workloads to increasingly sophisticated GNN-based schedulers for dependent DAGs. CNNs excel at prediction-optimization integration for cloud scheduling, while GNNs capture structural dependencies and enable adaptive real-time scheduling. However, challenges remain in runtime efficiency and inference overhead, underscoring the need for further refinement.

7.2. Energy optimization

DL frameworks have also been employed for energy-efficient scheduling by modeling system power characteristics and guiding dynamic allocation decisions. Ayub et al. [117] investigated a DL-based Adaptive Deadline-First (A-DF) scheduler on a *homogeneous Intel PXA270 multiprocessor*. Their approach integrated DPM with full task migration, allowing tasks to pause and resume across cores to balance deadlines and energy use. The DL scheduler outperformed SVM, CNN, and ANN baselines in accuracy and precision, achieving a 6.3% *energy reduction* when evaluated using the *STORM* [134] simulation framework (a flexible, open simulation tool for real-time multiprocessor scheduling). Additionally, it maintained robust performance across different clock frequencies. While migration integration with DL-guided decisions allows finer control, offline-trained models and frequent migrations may limit adaptability and incur overhead. In contrast, Gomatheeshwari and Selvakumar [97] designed a *lightweight deep neural network (LW-DNN)* for *independent task scheduling* on *heterogeneous ARM big.LITTLE* platforms. Workload characterization was

performed through embedded benchmarks (MiBench, IoMT, CoreMark) on ODRROID XU3, with hardware performance counters reduced via principal component analysis. The 20-layer LW-DNN predicted optimal core cluster and voltage-frequency pairs, targeting minimal Energy Delay Product (EDP). Results showed up to 97% prediction accuracy and reductions of 33% in energy, 35% in EDP, and 33% in execution time compared to heuristic and ML baselines. However, reliance on offline benchmark training and performance counters limited adaptability to unseen workloads, and inference overhead may be significant in resource-constrained embedded settings. Together, these studies show that DL can model and optimize energy use in both homogeneous and heterogeneous systems. While results demonstrated strong predictive accuracy, both approaches share limitations in offline training requirements and runtime adaptability.

The work by Balasekaran et al. in [135] extended DL models to *dependent-task models* in safety-critical autonomous vehicle systems. A DL-powered scheduler for autonomous vehicle systems that targeted *makespan reduction* while operating under strict real-time constraints was proposed. Deadline satisfaction was evaluated through the *task miss rate*, defined as the proportion of tasks whose response times exceed their deadlines. Using this measure, the scheduler reduced deadline violations by 32%–45%. Their framework integrated a Single Layer Feedforward Neural Network (SLFN) for resource prediction with an Earliest Hyperperiod First (EHF) scheduling algorithm. Implemented on Odroid Xu4 and Nvidia Jetson Xavier platforms with CAV and IoMT benchmarks, the scheduler shortened makespan by 20%–27% and achieved ~98% prediction accuracy, significantly outperforming EDF and RM baselines. While highly effective for autonomous workloads, reliance on domain-specific training data constrained its generalizability to broader application classes. Overall, this work demonstrated the capability of DL to reduce execution time in safety-critical heterogeneous systems, while ensuring deadlines are met and resources are efficiently utilized.

Table 6 provides a comparative summary of the DL-based works, categorized by task model, hardware platform, optimization objective, methodology, results, and limitations. Compared with ML, DL offers stronger representational power and predictive accuracy, especially when handling complex task dependencies in heterogeneous environments. However, DL models generally require extensive offline training, large datasets, and platform-specific tuning, which reduce their adaptability to unseen or highly dynamic workloads. By contrast, RL and DRL naturally support online learning and adaptability, making them more suitable for real-time responsiveness. This distinction highlights the potential of hybrid DL–RL approaches that combine DL's predictive strength with RL/DRL's adaptability to meet the demands of future multi-core scheduling.

8. Reinforcement learning based works

Reinforcement Learning (RL) has emerged as a practical and adaptive approach for task scheduling, particularly in environments where system conditions such as workload intensity, resource availability, and thermal behavior change dynamically over time. Unlike supervised and unsupervised learning approaches that rely on offline datasets or static models, RL learns directly through interaction with the system environment, enabling it to cope with uncertainty and variability in task arrivals, execution times, and resource performance. By framing scheduling as a sequential decision-making process, RL methods can dynamically adapt to both the task model (independent versus dependent) and the execution platform (homogeneous versus heterogeneous). Classical RL techniques, including Q-learning, Double Q-learning, and SARSA have proven especially powerful because they adjust scheduling policies from system feedback without requiring large training datasets or detailed workload models. These approaches flexibly balance multiple objectives such as *makespan*, *energy efficiency*, *thermal safety*, *fault-tolerance*, and *reliability*, all while maintaining relatively lightweight runtime overhead. In the following subsections, we review RL-based scheduling studies organized according to their optimization objectives.

Table 6

Summary of deep learning-based works grouped by task model with objectives: Energy (E), Temperature (T), Fault and Reliability (F), Makespan (M), and Deadline (D).

Ref.	Learning type	Hardware platform	E	T	F	M	D	Methodology summary	Evaluation & Results	Limitations
Independent tasks										
[98]	CNN + Whale Optimization Algorithm (WOA)	Heterogeneous – Cloud platform	×	×	×	✓	×	Integrates WOA into CNN to jointly optimize deadline prediction, task prioritization, and resource allocation; assigns short-deadline jobs higher priority to reduce completion time	98.5% deadline prediction accuracy; reduced execution time, resource usage, and makespan vs. IWDO, TaSLTS, LB-AD, and PSO	Offline model training; dependency on accurate profiling; potential scalability issues for large-scale dynamic workloads
[97]	Lightweight Deep Neural Network (LW-DNN)	Heterogeneous – ARM big.LITTLE (ODROID XU3, Exynos 5422 SoC)	✓	×	×	×	×	Runtime workload characterization using embedded benchmarks; PCA for feature reduction; 20-layer feed-forward network predicts optimal core cluster and VF pair to minimize EDP	Up to 97% core prediction accuracy; avg. 33% less energy, 35% lower EDP, 33% faster execution vs. heuristics/ML predictors	Benchmark-driven offline training; reliance on hardware performance counters; possible inference overhead in resource-constrained devices
[117]	Adaptive Deadline-First (A-DF) deep learning framework	Homogeneous – Intel PXA270-based multiprocessor	✓	×	×	×	×	Combines full task migration with DPM in DL-guided A-DF scheduler; monitors utilization and deadlines to decide migrations	6.3% energy saving; +2.1% accuracy; +2.4% precision; maintains performance across multiple clock frequencies	Offline-trained model; platform-specific tuning; full migrations may incur context-switch/cache overhead
Dependent tasks										
[81]	GNN	Heterogeneous – CPUs, GPUs, FPGAs, DSPs	×	×	×	✓	×	Classifies DAGs into locality/concurrency/mixed using GNN; selects optimal dynamic scheduling policy (Depend, RR, First-to-Finish)	96.4% classification accuracy; execution times competitive with best policies; validated on DAGGER workloads (GEMM, GETRF)	Runtime inference overhead due to Python; limited scalability without optimized deployment
[114]	GNN-based meta-scheduler (with GA pre-training)	Heterogeneous – Adaptive Time-Triggered multi-core Architecture (ATMA)	×	×	×	✓	×	Uses GA to generate multi-schedule graphs; trains supervised GNN to predict temporal task priorities and adapt to context events (slack, failure, mode changes)	Close-to-GA makespan; outperforms list scheduling with large modifiable task sets	GA training overhead; approximation accuracy depends on diversity of pre-generated schedules
[133]	GNN for link prediction in job allocation	Heterogeneous – Time-triggered systems (jobs, processors, switches)	×	×	×	✓	×	Represents jobs/processors as heterogeneous graph; predicts missing job–processor edges to reconstruct feasible schedules	Up to 41% prediction accuracy (better on larger graphs); reduces need for pre-stored schedules	Accuracy still below GA baselines; limited runtime generalization for unseen workloads
[135]	Single Layer Feedforward Neural Network (SLFN) + Earliest Hyperperiod First (EHF)	Heterogeneous – Odroid Xu4 & Nvidia Jetson Xavier	×	×	×	✓	✓	SLFN predicts optimal core cluster based on real-time features; EHF schedules tasks to reduce deadline violations	32%–45% task miss rate reduction; 20%–27% lower makespan; ~98% prediction accuracy vs. EDF and RM	Trained/evaluated only on IoT-based autonomous vehicle workloads; limited generalizability to other domains

8.1. Makespan minimization

For independent tasks, Shen et al. [136] introduced *EECRL*, a reinforcement learning scheduler for heterogeneous End–Edge–Cloud (EEC) systems incorporating CPUs, GPUs, and NPUs. The framework employed a Q-learning/DQN agent to jointly minimize *average task completion time* and *average waiting time*. The state encoded node resources, task execution options, and queue states, while the action space included both immediate assignment and deferred scheduling to mitigate congestion. Implemented on a real EEC testbed with Kubernetes, K3S, and KubeEdge across cloud servers, Atlas 200 DK edge devices, and Raspberry Pi end devices, EECRL achieved an average completion time of 24.29 s and up to 271% *improvement* over the First-Fit baseline under bursty workloads. The real-world validation highlighted the scalability and practicality of EECRL, though the optimization scope remains limited to time-related objectives.

The study in [83] presented TRATSS, a Transformer-based Task Scheduling System designed for autonomous vehicle operations in graph-based environments. It addressed a single-agent task scheduling problem formulated similarly to the Traveling Repairman Problem (TRP), where the agent must visit each area once with minimal

cost (e.g., distance or time). TRATSS integrated reinforcement learning (REINFORCE policy gradient) with a GNN-based encoder and a multi-head attention-based transformer decoder consisting of three sub-networks: (1) area selection, (2) entry point selection, and (3) pattern assignment. This architecture enabled TRATSS to optimize task sequences dynamically, accounting for spatial configurations and area-specific movement patterns. Experimental results on benchmark datasets and search-and-rescue scenarios demonstrated lower total costs and strong generalization across larger problem instances. The framework supported online decision-making and was implemented in Gym-compatible environments for real-world deployment. Within the family of hybrid RL approaches, Zhao et al. [118] proposed QL-HEFT for dependent tasks. This scheduling framework integrated Q-learning with the traditional HEFT algorithm [47] for dependent tasks in heterogeneous cloud environments. Q-learning was used to learn an optimal task ordering, where the HEFT-based upward rank ($rank_u$) served as the reward signal. Task-to-processor assignment was then carried out using the standard EFT-based HEFT mapping strategy. Simulation results using WorkflowSim and standard scientific DAG benchmarks showed that QL-HEFT outperformed HEFT and CPOP in terms of makespan, speedup and efficiency, especially under high CCR and complex task graphs. Nevertheless, scalability remains a concern due to the growing

Q-table size, motivating the authors to explore deep reinforcement learning in future work.

8.2. Energy efficiency

Reinforcement learning has also been applied to energy optimization, where the goal is to lower processor power consumption through different mechanisms without violating real-time or performance constraints. The specific strategies differ depending on the task model (independent vs. dependent), the hardware platform (homogeneous vs. heterogeneous), and the control mechanism employed (e.g., DVFS policy selection, task migration, power capping).

8.2.1. Homogeneous platforms

Islam et al. [137] introduced a hybrid RL-driven DVFS framework for *independent real-time tasks*. Their method relied on Q-learning to dynamically select both voltage–frequency settings and the most suitable DVFS policy. Two modules were incorporated to drive this process: the *Frequency Selector*, which set per-core operating frequencies based on CPU intensity, core ID, and previous frequency; and the *Technique Selector*, which choose among policies such as Cycle Conserving (CC) [138], Dynamic Reclaiming Algorithm (DRA) [139], Look Ahead (LA) [138], and Aggressive Speed Reduction (AGR) [139]. Operating under EDF scheduling, the framework guaranteed deadlines while saving energy. Results on synthetic workloads and real benchmarks reported up to 33% energy savings compared to traditional DVFS methods. Wang et al. [140] proposed an online power management framework for *independent tasks* on multi-core clusters. Their approach combined chip-level power capping with uncore frequency scaling, guided by decentralized RL agents based on double Q-learning. Each node adapted its power settings in response to runtime indicators such as IPC and memory access intensity. Unlike earlier supervised methods, the policy was refined continuously from feedback, enabling adaptation to new workloads. Experiments on two clusters showed up to 17% energy savings and a 5% performance boost, with an average of 12% savings and less than 3% slowdown. The decentralized nature, however, makes it less suited for workloads with strong inter-dependencies across nodes. A collaborative angle is explored in [103], where multiple mobile devices apply Q-learning for DVFS control and share their learned policies through a central server. Instead of exchanging full Q-tables, devices uploaded only compact policies, which fused into a global model that guides voltage/frequency selection. This cooperative setup improved adaptability to unseen states and accelerates convergence: learning speed was increased by up to 7×, and energy usage was cut by about 10% compared to single-device RL. Still, the method must contend with large state–action spaces, inter-core dependencies, and occasional instability during exploration.

Uddin et al. [141] addressed energy efficiency through a *dynamic slack-sharing* (DSS) mechanism for *independent periodic real-time tasks*. Here, Q-learning was used to determine at runtime how much slack to allocate to the next task and how much to reserve for later ones. Unlike prior approaches that treated slack allocation as static, DSS adapted it on the fly, recalculating frequencies when preemption occur and updating unfinished workloads accordingly. Tests on both simulated systems and a real Intel Core i5-8250U platform showed that DSS consistently outperformed classical DVFS schemes such as cycle conserving, look-ahead, and Lawitzky's method. Importantly, DSS was also compared with the *hybrid double Q-learning* (HDQ) approach and observed that HDQ learned which DVFS technique to select at runtime whereas DSS focused on learning the optimal slack-sharing rate. Results demonstrated that DSS achieved greater energy savings than HDQ across CPU-intensive, memory-intensive, and mixed workloads, particularly by allocating more slack to memory-intensive tasks. However, DSS convergence weakened in scenarios with highly variable execution times (low BCET/WCET ratios), where HDQ remained more stable.

8.2.2. Heterogeneous platforms

In the context of independent tasks on heterogeneous many-core processors, [104] proposed a Chip-Specific Power Co-Management (CSPM) strategy. Distributed Q-learning agents managed both DVFS levels and the on/off states of voltage regulators in each power domain. By incorporating inter-domain dependencies and process variation into a modular learning scheme, CSPM achieved more fine-grained energy control. Reported results showed a 60.1% reduction in the system's energy-delay product (EDP) compared to conventional DVFS methods. The main drawback is the added area and power overhead required to embed RL agents. Wang et al. [142] addressed real-time scheduling on big.LITTLE architectures. Their two-layer BLA-DM framework first used a SARSA-based binding phase to map independent periodic tasks to either big or LITTLE cores, balancing performance and energy. A second, dynamic migration phase then redistributed tasks within clusters to refine utilization. Independent periodic tasks were locally scheduled using EDF. Experiments on an ARM ZU4EV-based simulator showed that BLA-DM outperformed Linux CFS, heuristic TSA, and other baselines in both energy efficiency and schedule length. The approach, however, depends on accurate workload profiling and assumes that each cluster remains internally homogeneous, which may not hold in irregular or rapidly changing systems.

Overall, RL-based energy optimization on homogeneous systems has centered around DVFS control, slack sharing, and cooperative learning, with strong results in real-time and embedded contexts. On heterogeneous platforms, research has expanded toward chip-specific power domain management and big.LITTLE task mapping, achieving substantial energy savings but often at the cost of added design complexity or profiling requirements.

8.3. Multiobjective optimization

Reinforcement Learning has been applied to multiobjective scheduling, jointly optimizing conflicting metrics such as energy, thermal stability, makespan, and reliability in dynamic, resource-constrained environments.

8.3.1. Homogeneous platforms

A notable contribution in this direction was presented in [143], which proposed a Q-learning-based framework for dynamic thermal and reliability management (DRM) of *independent tasks* on *homogeneous multi-core processors*. The method integrated Q-learning with DVFS and operated under a phase-driven paradigm, where program segments exhibiting similar runtime behavior were grouped into distinct states in the Q-table. At each decision point, the agent selected the optimal working mode defined by a combination of active cores and frequency levels that maximizes throughput while adhering to reliability constraints shaped by aging-related failure mechanisms. To improve scalability and adaptation, the framework introduced three supporting modules. The *Thermal-aware Relearning Module* employed an auto-regressive model to predict future core temperatures and adjust learning behavior in anticipation of thermal shifts. The *Failure-rate Update Module* dynamically refined reliability bounds based on real-time feedback from on-chip aging sensors, accounting for cumulative degradation. The *Online Clustering Module* compressed the Q-table by grouping phases with similar performance and wear-out profiles, significantly reducing memory overhead. Experiments conducted on a 3D DRAM-on-Logic CPU platform using task phases derived from SPLASH-II and PARSEC benchmark suites demonstrated the method's effectiveness, showing over 1.36× performance improvement and 60% memory reduction compared to prior Q-learning-based DRM techniques. While DVFS indirectly impacts energy consumption, the core focus of this approach remains on temperature regulation and long-term system dependability. Its reliance on specific hardware-level sensors and models may limit generalizability in energy-constrained or platform-agnostic environments. Hossein-Khani and Akbari [144] presented a

Q-learning-based task mapping strategy to enhance reliability in many-core systems executing *independent tasks*. Their approach operated in two levels: (i) a DBSCAN-based clustering step that grouped cores into thermally similar bins, and (ii) Q-learning agents for task-to-bin and task-to-core mapping. By considering temperature profiles along with aging mechanisms such as thermal cycling (TC), and electromigration (EM), the framework dynamically updated mean time to failure (MTTF) without relying on offline profiling. Experiments on 16-, 32-, and 64-core platforms using SPLASH2, PARSEC, and a Sobel filter demonstrated up to 27% average MTTF improvement and more uniform thermal distribution compared to state-of-the-art mapping techniques. The method is limited to ring-based interconnects, does not explicitly optimize inter-task communication, and remains simulation-based.

Mandal et al. [145] proposed a PPO-driven RL framework for *independent-task* scheduling with DVFS, targeting joint thermal and energy optimization under deadlines. The environment modeled cores with discrete power modes (OFF/LOW/HIGH), switching overheads, and a lumped thermal model, while the agent observed per-core temperature, mode, and occupancy along with the next task's features and current time, and then selected (core, DVFS level) actions. A temperature-aware reward function balanced thermal headroom against energy and deadline penalties, shifting priority toward thermal safety as cores heat up, and a deterministic "emergency" fallback was invoked if the peak temperature approached the safety bound. On simulated task sets, the framework reduced peak temperature compared to an energy-only RL policy and achieved $\approx 6.6\%$ lower energy consumption than a greedy thermal heuristic, though at the cost of $\approx 3.5\%$ higher energy compared to the energy-only baseline. Key limitations include reliance on a simplified thermal model, restriction to non-preemptive tasks, and evaluation limited to simulation. Maurer et al. [115] proposed a QoS-aware dynamic frequency scaling governor for mixed-critical embedded systems, where the RL agent run in hardware and was complemented by a preemptive shield to enforce power constraints while meeting deadline-driven QoS and minimizing energy. The agent was designed as an LCT-based RL controller augmented with eligibility traces [SARSA (λ)] to accelerate credit assignment under sparse rewards, while the shield restricted actions near a power "margin" to preempt violations. Evaluated in a hardware-in-the-loop setup (LLVM SingleSource benchmarks on a SparcV8 (LEON3) core on Xilinx Artix-7), the shielded SARSA (λ) achieves nearly 100% deadline success, lowest power/energy, and zero power-constraint violations, outperforming Q-learning and SARSA baselines without eligibility traces or shielding. This work targeted independent tasks on a single-core platform, emphasizing energy/QoS trade-offs under hard power ceilings.

For independent tasks on homogeneous multi-core systems, CAR-TAD [146] introduced a Compiler-Assisted RL framework for *thermal-aware task scheduling and DVFS*. The approach uniquely leveraged compiler-generated LLVM Intermediate Representation (LLVM-IR) to extract application-level features (e.g., instruction count) to enrich RL state modeling. A Q-learning agent concurrently selected both the target core and operating frequency for each incoming task, aiming to minimize peak temperature while satisfying latency constraints. The reward function balanced latency satisfaction with thermal mitigation. Results on PARSEC benchmarks showed up to 16 °C reduction in peak temperature and 10 °C reduction in average temperature, while maintaining low latency violation rates and strong adaptability to unseen workloads.

8.3.2. Heterogeneous platforms

Siyadatadeh et al. [147] proposed *RL-TIME*, a Q-learning based runtime replication and mapping framework for periodic *independent* real-time tasks on embedded platforms. RL-TIME observed per-core Thermal Safe Power (TSP), task reliability (via voltage/frequency-dependent failure models), and EDF schedulability, then determined the number of replicas and their core mappings online. A discretized

state encoded the current reliability target and normalized TSP; the action specified replica counts. The reward jointly drive the system toward reliability targets while respecting thermal constraints. On PARSEC workloads, RL-TIME reduced average power by 63%, improved schedulability by 53%, and met TSP 72% more often compared to static and heuristic baselines (EM [148], ReMap [149], TMR [150]). Key limitations include reliance on state discretization, cold-start pre-training, and evaluation restricted to simulation. Weber et al. [151] proposed the *FIT-aware Learning Heuristic for Application Allocation (FLEA)*, a Q-learning framework for dependent tasks on many-core platforms. FLEA integrated a Thermal Estimation Accelerator (TEA) and FIT estimator, enabling allocation and migration that minimized hotspots and extended system lifetime. Results showed lower peak/average temperatures, improved thermal distribution, and higher MTTF than state-of-the-art thermal/reliability methods. Overheads from Q-table training and migration were mitigated by compact state representations and efficient heuristics. Maity et al. [152] developed a self-learning resource manager for heterogeneous CPU-GPU platforms executing OpenCL DAG kernels. Their framework employed model-free Q-learning/DQN together with an offline Sparse Variational Gaussian Process (SVGP) surrogate of temperature-latency dynamics to balance exploration and exploitation safely, while co-tuning DVFS states and kernel partitioning from sensor feedback. The results on Odroid XU4 showed that exploration enabled the discovery of efficient mappings that reduced peak temperatures, while controlled exploitation ensured that deadlines were preserved.

A self-optimizing computing system (SOSPCS) was proposed in [153] for reconfigurable heterogeneous platforms with dependent tasks. At compile-time, applications were mapped into instruction dependency graphs (IDGs) and analyzed using NN-based classifiers and community detection. At runtime, a decentralized Q-learning scheduler dynamically assigned tasks to heterogeneous processing elements. The system leveraged software-defined hardware (SDH) and domain-specific SoCs to improve programmability, performance, and energy efficiency. Limitations included reliance on static classifiers, exploration overhead, and occasional misclassifications that misallocate GPU-intensive tasks. Afshari and Abdi [154] proposed QSMix, an offline Q-learning scheduler for dependent mixed-critical tasks. It simultaneously optimized execution time, energy, temperature, and reliability using a weighted reward with penalties for violations. QSMix constructed Q-tables via simulated mappings under DVFS settings. Experiments on synthetic and real DAGs showed up to 9% average improvement over heuristic, meta-heuristic, and DRL baselines, particularly in large-scale or thermally constrained environments.

Shi et al. [155] addressed scheduling in autonomous driving systems (ADS), where periodic sensing tasks (LiDAR, radar, camera) form a dependent DAG. They introduced Age of Information (AoI) as a central scheduling metric, showing that optimizing AoI improves data freshness, reduces worst-case response time (WCRT), and increases throughput. They formulated the AoI-centric scheduling problem (NP-hard) and proposed (i) a 4-approximation algorithm (RT-Sch) minimizing WCRT, and (ii) RL-Sch, a DDQN-based approach learning near-optimal schedules. On Apollo ADS simulations, RL-Sch achieves lower maximum AoI and WCRT on 4 cores than Apollo's scheduler on 8 cores, demonstrating safety and efficiency gains. Garfiet al. [112] proposed a Q-learning-based auto-scaling strategy for scientific workflows represented as DAGs in heterogeneous cloud systems. The scaling problem was modeled as a Markov Decision Process (MDP), where actions determined the number and type of VMs provisioned, and the reward balances *makespan* and *cost* through a tunable parameter δ . Validated on CyberShake, Montage, Inspiral, and SIPHT benchmarks, the approach achieved 25%–55% improvements in the joint makespan–cost metric compared to a state-of-the-art RL baseline, and in some cases approached the performance of heuristic near-ideal strategies. In the service domain, [156] introduced COMS, a customer-oriented multi-task scheduling model tailored for cloud manufacturing platforms. The

Table 7

Summary of reinforcement learning-based works for independent tasks with objectives: Energy (E), Temperature (T), Fault and Reliability (F), Makespan (M), and Deadline (D).

Ref.	Learning type	Hardware platform	E	T	F	M	D	Methodology summary	Evaluation & Results	Limitations
[83]	REINFORCE policy gradient with Transformer + GNN encoder-decoder	Homogeneous – single-agent UAV scheduling	×	×	×	✓	×	Transformer-based task scheduling system (TRATSS) with 3-stage decoder: area selection, entry point, and movement pattern assignment	Outperformed Concorde solver with ~10% lower cost; strong generalization to larger unseen instances	Limited to single-agent homogeneous scenarios; no explicit multi-agent coordination or multiobjective optimization
[137]	Q-learning for DVFS (hybrid policy + frequency selector)	Homogeneous – multi-core real-time systems	✓	×	×	×	×	Dynamically selects DVFS policies and voltage–frequency settings under EDF	Up to 33% energy savings on CLab, CNC, MiBench	Limited scalability of Q-table; depends on EDF scheduling
[140]	Double Q-learning for decentralized power management	Homogeneous – multi-core clusters	✓	×	×	×	×	Agents adapt chip-level power cap and uncore frequency from IPC/memory feedback	12% avg. energy savings; 5% performance boost	Less effective for workloads with inter-node dependencies
[103]	Cooperative Q-learning (policy sharing)	Homogeneous – mobile devices + server	✓	×	×	×	×	Devices share compact policies to accelerate convergence	7× faster learning; ~10% energy saving	Instability during exploration; large state–action spaces
[141]	Q-learning for Dynamic Slack Sharing (DSS)	Homogeneous – periodic real-time tasks	✓	×	×	×	×	Adaptive slack allocation with preemption	Outperforms DVFS baselines on Intel i5-8250U	Convergence weak under variable execution times
[104]	Distributed Q-learning (CSPM)	Heterogeneous – many-core processors	✓	×	×	×	×	Manages DVFS + regulators with inter-domain dependencies	60.1% EDP reduction vs. DVFS	Hardware overhead from embedded RL agents
[142]	SARSA-based BLA-DM	Heterogeneous – ARM big.LITTLE	✓	×	×	×	×	Two-phase mapping: SARSA + dynamic migration	Better energy + schedule length vs. Linux CFS, TSA	Needs accurate profiling; assumes cluster homogeneity
[143]	Q-learning with DRM modules	Homogeneous – multi-core CPU + 3D DRAM	×	✓	✓	×	×	Thermal/reliability-aware DVFS scheduling	1.36× perf. gain; 60% memory reduction	Needs hardware sensors; platform-specific
[145]	PPO-based RL	Homogeneous – multi-core DVFS	✓	✓	×	×	×	Joint thermal–energy optimization with fallback	6.6% less energy vs. heuristic; safe temps	Simplified thermal model; sim only
[144]	Q-learning + DBSCAN thermal clustering	Homogeneous – many-core	×	✓	✓	×	×	Two-level mapping: thermal clustering + Q-learning	27% MTTF improvement; balanced thermal distribution	Limited to ring interconnect; sim only
[115]	SARSA (λ) + shield	Homogeneous – LEON3 (Artix-7 FPGA)	✓	×	×	×	✓	RL agent in hardware with preemptive shield	Near-100% deadlines; lowest energy	Single-core; shield logic overhead
[146]	Q-learning + LLVM-IR features	Homogeneous – multi-core CPUs	×	✓	×	×	✓	Compiler-assisted RL for thermal + latency-aware DVFS	Up to 16 °C peak temp reduction; low latency violations	Evaluated only on PARSEC; limited generalization

framework jointly addressed several objectives: customer satisfaction (modeled via soft constraints on time, cost, and quality), service feasibility (hard constraints on availability windows), energy consumption, and computational efficiency. A set of dependent tasks decomposed from orders are mapped to services while balancing soft non-functional expectations and strict availability requirements. To solve this multi-objective problem, the authors designed an Improved Hyper-Heuristic Algorithm (IHHA) that integrates three custom low-level heuristics with a Q-learning-based high-level controller for adaptive heuristic selection. Experimental results confirmed IHHA's superiority over baseline algorithms (NSGA-II, MOEA/D, PSO, SPEA2, and TLBO), delivering up to 47.52% reduction in computational time. This demonstrated its effectiveness in optimizing energy while simultaneously improving customer satisfaction.

As summarized in Tables 7 and 8, existing RL studies span both homogeneous and heterogeneous systems, covering various objectives, along with their hardware platforms, optimization objectives, methodologies, results, and limitations. These capabilities position RL as a strong alternative to static heuristics, while also complementing other learning paradigms like supervised ML or deep learning in environments where workloads and resources fluctuate frequently. Homogeneous systems emphasize thermal–energy trade-offs, slack/QoS management, and latency control, while heterogeneous works broaden into complex trade-offs spanning makespan, energy, reliability, and cost.

At the same time, classical RL still faces scalability barriers. Q-tables and discretized state–action spaces become impractical as systems grow more complex, particularly in large heterogeneous platforms where state dimensionality increases rapidly. To overcome these challenges, researchers are turning toward DRL. By using neural approximators, DRL can generalize across high-dimensional states, capture dependencies between tasks, and handle continuous action spaces. This makes DRL a natural evolution of RL-based scheduling, better suited to the dynamic, large-scale heterogeneous computing systems expected in the future.

9. Deep reinforcement learning based works

Deep Reinforcement Learning combines reinforcement learning with deep neural networks, enabling agents to operate effectively in large and high-dimensional state–action spaces. Unlike classical RL approaches, which are constrained by the scalability of Q-tables or handcrafted features, DRL leverages function approximation to learn directly from complex state representations. This makes DRL particularly effective in heterogeneous and dynamic scheduling environments, where workloads evolve rapidly and system conditions change frequently.

DRL-based schedulers have demonstrated strong adaptability and robustness in optimizing diverse objectives such as *makespan*, *energy*

Table 8

Summary of reinforcement learning-based works for dependent tasks with objectives: Energy (E), Temperature (T), Fault and Reliability (F), Makespan (M), and Deadline (D).

Ref.	Learning type	Hardware platform	E	T	F	M	D	Methodology summary	Evaluation & Results	Limitations
[118]	Q-learning + HEFT (QL-HEFT)	Heterogeneous – cloud DAG	×	×	×	✓	×	Learns task ordering; HEFT maps EFT	Outperforms HEFT/CPOP on scientific DAGs	Q-table scalability issues
[147]	Q-learning (RL-TIME)	Heterogeneous – embedded multi-core	✓	✓	✓	×	×	Runtime replication + mapping under EDF/TSP/reliability	63% less power, 53% better schedulability	Discretization; simulation only
[136]	Q-learning/DQN (EECRL)	Heterogeneous – End-Edge-Cloud	×	×	×	✓	×	Minimizes completion + waiting time via deferral	271% faster vs. First-Fit; Kubernetes testbed	Energy not directly optimized
[155]	Double DQN (RL-Sch)	Homogeneous – ADS DAG	×	×	×	×	✓	AoI-driven RL scheduling	Lower AoI + WCRT vs. Apollo schedulers	ADS-specific; sim only
[151]	Q-learning (FLEA)	Homogeneous – RISC-V many-core	×	✓	✓	×	×	Offline Q-table; runtime guided by TEA + FIT	Better thermal + reliability than DRM	Overhead from migration/training
[153]	Q-learning (SOSPCS)	Heterogeneous – reconfigurable SoC	✓	×	×	✓	×	Compile-time NN + community detection + RL	Energy + perf gains via SDH/SoCs	Exploration overhead; NN misclassifications
[156]	Q-learning + hyper-heuristic (IHHA)	Heterogeneous – cloud manufacturing	✓	×	×	✓	×	RL selects heuristics dynamically	47% better tradeoff boundary vs. NSGA-II	Overhead; limited to cloud manufacturing
[152]	Q-learning + DQN (thermal-aware)	Heterogeneous – Odroid XU4 (CPU-GPU)	×	✓	×	×	✓	RL agent adjusts DVFS + partitioning; SVGP model for safety	Lower peak temp; deadline compliant	Limited CNN DAG workloads; sim
[112]	Q-learning (auto-scaling)	Heterogeneous – cloud workflows	×	×	×	✓	×	Auto-scaling as MDP; balances makespan + cost	25%–55% perf gain vs. baselines	Focus on DAG workflows
[154]	Q-learning (QSMix)	Heterogeneous – mixed-critical multi-core	✓	✓	✓	✓	×	Weighted reward across MS, E, T, F	9% avg. improvement vs. baselines	Offline + sim only

efficiency, thermal management, and fault-tolerance. These approaches typically employ architectures such as DQN, Proximal Policy Optimization (PPO), or hybrid actor-critic models, and are increasingly extended with GNNs and Transformer encoders for structured task representations. Moreover, both single-agent and multi-agent DRL methods have been applied, covering discrete and continuous control spaces, as well as federated and hybrid frameworks. In this section, we classify DRL-based scheduling works according to their optimization objectives.

9.1. Makespan minimization

Deep reinforcement learning has been increasingly adopted to minimize the *makespan*, i.e., the overall completion time of a workflow, under dependent task models [59,88,119]. Recent works demonstrated that the choice of scheduling strategy is strongly influenced by the underlying hardware platform. To reflect this distinction, the following discussion is organized into *homogeneous* and *heterogeneous* multi-core systems.

9.1.1. Homogeneous platforms

Mo et al. [88] proposed a *GAT-based DRL scheduler* for *dependent real-time tasks* modeled as DAGs on *homogeneous multi-core* platforms. Task dependencies were captured using a *Graph Attention Network (GAT)*, which enhanced representation learning by aggregating predecessor-successor relationships with a multi-head attention mechanism. On top of this representation, a *Soft Actor-Critic (SAC)* policy optimized task-to-core mappings to minimize *makespan* while satisfying deadline and dependency constraints. Unlike classical heuristics such as HEFT or CPOP, which relied on fixed priority rules and local task ranking, GAT explicitly model complex DAG interactions, enabling the RL agent to make globally informed scheduling decisions that adapt to varying DAG structures and dynamic workloads. Experiments on randomly generated DAG workloads showed that the proposed method outperformed heuristics (HEFT, CPOP), meta-heuristics (GA, PSO), and DRL baselines (DQN, A2C, PPO), achieving on average a 6%–10% *makespan reduction*, lower *Schedule Length Ratio (SLR)*,

and *higher speedup*. However, integrating GAT with entropy-driven SAC introduced significantly higher training/runtime overheads (up to ~3.8× compared to DQN), limiting scalability in resource-constrained real-time systems.

9.1.2. Heterogeneous platforms

Zhadan et al. [59] introduced a multi-agent reinforcement learning (MARL) approach for static DAG scheduling in *heterogeneous* systems. Each task type in the DAG was treated as a separate agent, and Proximal Policy Optimization (PPO) [89] was used to adaptively select prioritization heuristics at each scheduling step. The framework featured a two-level architecture: the upper level employed PPO agents to choose among nine graph-based metrics for prioritizing tasks, while the lower level used the Earliest Finish Time (EFT) rule for task-to-core mapping. This dynamic dual-layer structure enabled adaptation to diverse DAG topologies. Experimental results across different DAG sizes and heterogeneous executor configurations (i.e., setups where the available executors such as CPU cores, GPUs, or VMs differ in performance, power consumption, or specialization) revealed up to 8.96% *makespan improvement* over DONF [157] and up to 27.58% over HPS [158] and PETS [159], with proximity of 92.03% to MILP-based optimal solutions [60].

To further improve adaptability, an online DRL-based scheduler is proposed in [119]. Here, an online DRL framework used R-DQN with a deep neural network (DNN) and L2 regularization to prevent overfitting. The system state was represented through both task and resource features, and training stability is ensured via experience replay and fixed target networks. This scheduler dynamically allocated tasks to virtual machines (VMs) according to current system conditions. Evaluated on real-world workflow benchmarks (Epigenomics, Inspiral, Montage, Sipht) using a Python-based WorkflowSim platform, the method demonstrated significant *makespan reduction* and improved robustness compared to heuristic and meta-heuristic baselines such as MAXMIN, DHEFT, and ACS. Tan et al. [160] proposed a deep reinforcement learning-based scheduling algorithm for handling DAG tasks on heterogeneous multi-core processors. The scheduling problem was

framed as a Markov Decision Process (MDP), where task-node selection was handled by a deep neural network, and processor allocation was carried out via a heuristic algorithm inspired by earliest finish time (EFT). A novel transformer-enhanced graph convolution network was employed for DAG feature extraction. The PPO algorithm, combined with an Advantage Actor-Critic (A2C) network and a dynamically decaying clipping factor, optimized the scheduling policy. Simulation results showed a 10.89% improvement in scheduling time compared to baseline methods like QL-HEFT, IPPTS, FCFS-HTDG, Smart-mDAG-EFT, and OCT-HEFT. The primary optimization objective was to minimize makespan, and the experimental setup focused on batch-mode DAG arrivals in heterogeneous multi-core systems.

9.2. Energy efficiency

Deep Reinforcement Learning has been increasingly adopted for energy-aware scheduling, where agents learn directly from interaction with the environment to optimize processor power consumption while respecting timing and performance constraints. Compared to classical RL, DRL scales better to large state spaces and continuous actions, making it particularly suited for modern heterogeneous platforms where energy is a major design parameter.

9.2.1. Homogeneous platforms

Imtiyaz et al. [161] proposed RL-TS, an agent-based DRL scheduling framework for large-scale cloud environments. RL-TS learned from runtime feedback to make dynamic scheduling decisions. Experiments on workloads of 1000–5000 jobs showed up to 81% success rate and 27% gains in *energy efficiency* compared to heuristics. Although finish times were slightly longer, the improvements in success rate and energy savings justified the trade-off for cloud-scale workloads.

Zhou and Lin [109] proposed a model-free DVFS governor for soft-deadline periodic real-time workloads on embedded devices. Their method was implemented within the Linux CPUFreq framework, the kernel subsystem that managed CPU dynamic voltage and frequency scaling (DVFS) through pluggable governors. Unlike the heuristic-based approaches, their governor employed a temporal-encoding Double Deep Q-Network (DDQN). The system state incorporated frequency/utilization history, average utilization, normalized time progress, and an “allocated performance matrix” (a compact temporal-encoding feature that records how execution time has been distributed across frequency levels under varying utilization intervals). Inference was performed in-kernel by a lightweight quantized neural network with 150 parameters, while training with prioritized replay occurred in user space. Although effective for energy savings, the approach remained constrained by its reliance on periodic workloads, per-task training, and evaluation on a single platform without explicit thermal modeling.

Peng et al. [105] addressed dependent workloads with DRL-GAT, a dynamic scheduling algorithm for energy-efficient DAG scheduling on multi-core platforms. The framework combined heuristic task-to-core mapping with an Actor-Critic RL model that selected DVFS levels. Graph Attention Networks (GAT) captured task dependencies in the state space, enabling the RL agent to consider critical paths and task interactions when making scheduling decisions. Compared to MLP-based baselines, which treated inputs as flat vectors and ignored graph structure, GAT provided a richer, dependency-aware representation, leading to more accurate energy- and deadline-aware scheduling. Trained offline and deployed online, DRL-GAT achieved near-optimal energy use (within 10%) with reduced computation time, outperforming MLP-based and heuristic baselines in real-time schedulability and energy. Limitations include fixed-profile offline training, GAT/Actor-Critic overhead, and the separation of mapping and frequency decisions into distinct stages.

9.2.2. Heterogeneous platforms

Gupta et al. [162] proposed a DQN-based power management framework for independent tasks on heterogeneous ARM big.LITTLE

SoCs. The agent jointly controls the number and frequency of big and LITTLE cores, with states defined by utilization, power, and hardware performance counters. A tailored reward optimizes performance per watt (PPW), and prioritized experience replay reduces memory cost. On the Odroid XU3 platform, the method achieves PPW within 1% of an *Oracle* (an offline exhaustive search baseline) and 91.2% action accuracy. Main drawbacks include high model complexity, offline training cost, and sensitivity to reward design.

Gupta et al. [163] proposed HFDRRL, a hybrid fuzzy-based deep remora RL scheduler for dependent tasks on heterogeneous multi-core processors. DRL classified tasks as short/long, fuzzy logic assigns processors, and the Remora Optimization Algorithm (ROA) balanced load through migration. The task model explicitly considered precedence and critical sections. On TGFF benchmarks, HFDRRL reduced execution time and energy compared to GA [70], PSO [18], IM-GA [164], and BABP [165]. Limitations include the computational overhead of ROA, especially for large task sets.

9.3. Deadline-constrained scheduling

In this category, DRL agents are trained to minimize task deadline violations, an objective that is particularly critical in real-time and interactive systems. The surveyed methods adopt DRL techniques tailored to dynamic workload arrivals, resource heterogeneity, and even trust-aware edge scenarios.

9.3.1. Heterogeneous platforms

An early contribution to deadline-constrained scheduling in cloud settings was DDQN-TS, proposed by Tong et al. [166]. Designed for *independent tasks* on virtual machines, the method employed a Double Deep Q-Network (DDQN) to address Q-value overestimation common in DQN-based task schedulers. The framework comprised two stages: an offline training phase where scheduling policies were learned, and an online module that performed real-time task assignment as tasks arrive. A custom reward function balanced average deadline satisfaction against task completion rate, with incentives favoring faster VMs that meet SLA constraints. Evaluation using Random, Google, and Alibaba cloud workloads demonstrated that DDQN-TS achieved higher completion rates and faster convergence compared to Random, Round Robin, and conventional DQN strategies.

Li et al. [167] presented an energy-efficient DVFS governor for a homogeneous quad-core ARM Jetson Nano. Their method employed a temporal-encoding Double Deep Q-Network (DDQN), where the system state was derived from utilization and per-task timing signals. Two encoder variants were used: (i) a Gated Recurrent Unit (GRU) combined with a Multilayer Perceptron (MLP) (a standard feedforward neural network with multiple fully connected layers) and (ii) a pure MLP. The agent selected CPU frequencies directly within the Linux kernel, achieving inference latency below 0.2 ms, while training occurred in user space. The reward function balanced deadline adherence with energy use. Experiments on MiBench workloads showed 3%–10% power savings compared to standard Linux governors, with minimal deadline misses. Strengths of the approach include its low-latency kernel-resident inference and explicit deadline-awareness. However, some limitations remain. The evaluation was restricted to a single SoC with only two available frequency levels. The method also relied on sparse rewards and employed floating-point arithmetic in the kernel, which raised efficiency and safety concerns.

Expanding beyond performance metrics, the BD-TTS framework [168] introduced a trust-aware scheduling mechanism in edge computing, where reliability and latency concerns were compounded by the potential presence of malicious nodes. BD-TTS integrated blockchain technology with deep reinforcement learning to provide decentralized trust evaluation and secure scheduling. A blockchain-based consensus mechanism assessed the trustworthiness of edge servers, while a DQN-based DRL scheduler dynamically allocated tasks based on trust values

and resource availability. Empirical results showed that BD-TTS not only improved deadline satisfaction but also increased task success rates and lowered the probability of assigning tasks to untrusted servers by over 64.4%, enhancing both performance and trustworthiness in edge environments.

Zhang et al. [16] addressed *real-time workflow scheduling* with precedence-constrained DAGs in cloud environments through a hybrid GA-DQN framework. The method integrated a Genetic Algorithm with Deep Q-Network, dividing scheduling into two coordinated stages. First, GA generated execution schemes for workflow sub-tasks on VM cores, capturing precedence constraints and reducing the combinatorial search space. These GA-derived schemes were then used as workflow features for the DQN agent, which performed real-time task-to-VM allocation decisions by sensing both workflow characteristics and dynamic cloud states. The reward function was shaped by *deadline satisfaction* as the primary objective, while execution cost was incorporated as a secondary optimization goal. Evaluation on real-world scientific workflows (LIGO, Epigenomics, CyberShake) modeled in Pegasus and executed under varying workload arrival rates, workflow types (transaction-intensive and compute-intensive), and VM configurations demonstrated that GA-DQN consistently outperformed baseline strategies (Random, Round Robin, Earliest, NOSF). Results highlighted significant gains in success rate (i.e., proportion of workflows meeting QoS deadlines), average response time, and overall cost-efficiency. Importantly, GA-DQN achieved this with reduced computational overhead compared to standalone GA approaches, enabling its applicability to real-time scheduling scenarios. However, scalability to very large deployments remained limited by the reliance on a single DRL agent which in turn affects performance in highly dynamic or distributed systems.

In summary, DRL-based deadline-constrained scheduling has been explored extensively in cloud and edge platforms, addressing both independent and dependent task models as well as trust-aware scenarios. By contrast, no significant DRL studies were identified for homogeneous multi-core platforms in this objective category, pointing to an open research opportunity.

9.4. Multiobjective optimization

In multiobjective scheduling, DRL has been applied to balance *makespan*, *energy*, *thermal stability*, and *reliability*. The following works are categorized into *homogeneous* and *heterogeneous* system settings.

9.4.1. Homogeneous systems

Mohammadi and Beitollahi [169] proposed *Q-scheduler*, a Deep Q-Learning-based task scheduler for *independent periodic tasks* on *homogeneous real-time multiprocessor embedded systems*. The method aimed to jointly minimize *energy consumption* and *processor temperature* without relying on traditional DVFS/DPM techniques. The scheduler was trained offline using synthetic task data characterized by task utilization and temperature, along with processor utilization and thermal state. A deep neural network approximated the Q-values for task-to-core assignments using a state matrix encoding task and processor attributes, with continuous online updates for adaptability. During execution, Q-scheduler ensured schedulability, temperature compliance, and power constraints by reassigning tasks that violate any condition. The system used EDF scheduling at the core level with non-migrating jobs. Evaluation results showed up to 15% energy savings and 10% reduction in processor temperature compared to state-of-the-art methods, demonstrating its effectiveness in thermal-aware and energy-efficient scheduling in real-time systems.

9.4.2. Heterogeneous systems

While only limited work has explored DRL-based multiobjective scheduling in homogeneous platforms, the majority of research tar-

gets *heterogeneous systems*. This emphasis arises because heterogeneity introduces diverse processing capabilities, resource costs, and fault behaviors, making multiobjective trade-offs particularly challenging. Accordingly, most DRL contributions in this area addressed cloud, edge, and mobile computing environments, as detailed below.

On mobile platforms, where thermal constraints can impair performance and accuracy, a notable recent work [170] focused on *thermal-aware scheduling of independent DNN tasks* on *heterogeneous mobile platforms* equipped with GPU and NPU. The study aimed to avoid device overheating while maintaining a balance between *processing speed*, *inference accuracy*, and *energy consumption*. Two scheduling algorithms were proposed: Heuristic-Based Scheduling (HBS), which used historical data to predict temperature changes and make greedy decisions based on current state; and Deep Reinforcement Learning Scheduling (DRLS), which employed deep Q-learning to account for the long-term impact of scheduling decisions. The DRLS framework integrated a temperature-sensitive reward function and a tailored replay buffer to mitigate catastrophic forgetting. Evaluations conducted on HUAWEI Mate 10 Pro using SqueezeNet and GoogleNet showed that DRLS outperformed heuristic, GPU-only, NPU-only, and DVFS-based strategies, particularly under confined environments such as VR headsets. This work highlighted the efficacy of DRL in improving thermal resilience and sustained performance for mobile deep learning workloads.

In cloud datacenter, where reliability and efficiency were critical, the Deep Reinforcement Learning Fault-Tolerant Aware Task Scheduler (DRFTSA) [171] utilized a Deep Q-Network model to optimize scheduling decisions in cloud computing environments. The model operated on *independent tasks* and targeted *heterogeneous VMs* distributed across multiple datacenters. DRFTSA assigned priorities to both tasks and VMs based on factors such as task length and the unit electricity cost at the VM's datacenter, which allowed for energy-efficient and fault-aware mapping. The system was trained using reinforcement learning with a reward function that jointly minimized *makespan*, *rate of failure*, and *energy consumption*. It leveraged a real-world dataset-Google Cloud Jobs-and was implemented in the CloudSim simulator. In extensive evaluations, DRFTSA was benchmarked against classical meta-heuristics such as Particle Swarm Optimization (PSO) [172], Ant Colony Optimization (ACO) [19], and Genetic Algorithm (GA) [70]. The results showed that DRFTSA reduced makespan by up to 37.12%, failure rate by 46.19%, and energy consumption by 28.8%, highlighting its effectiveness for real-time scheduling in large-scale cloud systems.

To further address scheduling in multi-cloud environments, a multi-objective prioritized task scheduling framework named *MOPTSA3C* was presented by Mangalampalli et al. [91]. This work targeted *independent tasks* in a *heterogeneous multi-cloud environment*, utilizing an improved Asynchronous Advantage Actor-Critic (A3C) algorithm. The scheduling objectives included minimization of *makespan*, reduction of *resource cost*, improved *resource utilization*, and enhanced *reliability*. Task priorities were derived from size and runtime requirements, while VM priorities were determined by unit electricity cost, enabling the scheduler to dynamically provision resources across multiple clouds. By adaptively mapping tasks to cost-efficient and available VMs, the framework achieved effective load balancing and cost-aware provisioning. Extensive evaluations in CloudSim with both synthetic and real-world datasets (NASA, HPC2N) demonstrated that MOPTSA3C consistently outperformed DQN, A2C, and MOABCQ baselines, delivering superior performance across all stated objectives.

Building on this, a recent study by the same authors [173] proposed a Hybrid Deep Deterministic Policy Gradient Task Scheduler (HDDPGTS) tailored to High-Performance Computing (HPC) and High-Throughput Computing (HTC) workloads in cloud environments. The scheduler consisted of three phases: (1) task classification based on task length and runtime to distinguish between HPC and HTC tasks; (2) VM resource tracking to match task types with appropriate VMs; and (3) priority-based task-to-VM mapping using unit electricity cost as a

constraint. The underlying actor-critic framework used the Deep Deterministic Policy Gradient (DDPG) algorithm to learn optimal scheduling policies in continuous action spaces. CloudSim-based experiments with both real and synthetic workloads confirmed that HDDPGTS significantly outperformed DQN and A2C in terms of *makespan*, *energy efficiency*, *scheduling overhead*, and *scalability*.

Task offloading in hybrid cloud-edge environments was addressed by Zhao et al. [50], where the Self-Learning Task Offloading and Resource Allocation (SLTRA) algorithm was proposed. This method considered *independent tasks* across heterogeneous execution nodes such as UEs, MEC servers, and cloud servers. It employed a Deep Q-Network to learn optimal task-to-node mappings under constrained computing and reliability conditions in a *real-time scheduling* context. The system was modeled as an MDP, with a reward function that jointly optimized *deadline satisfaction* and *energy consumption*, balanced using entropy-based weights. The agent adapted online during runtime to dynamic workload profiles, user mobility, and varying node reliability. Experimental results demonstrated that SLTRA consistently outperformed traditional baselines (e.g., QL, RR, WRR, Random) across multiple scenarios, achieving notable improvements in both deadline compliance and energy efficiency while maintaining strong adaptability to scale and variability.

In a similar setting, Tong et al. [174] presented DTORA, a deep reinforcement learning framework for dynamic task offloading and resource allocation in mobile edge computing (MEC). The system modeled *independent tasks* as a Poisson arrival process and accounts for user mobility between base stations in a heterogeneous MEC environment. Using a Deep Q-Network, DTORA performed real-time offloading decisions with the goal of maximizing *deadline satisfaction* while simultaneously reducing overall *energy consumption*. A key contribution is its entropy-weighted reward function that adaptively balanced these competing objectives. Experimental evaluation showed that DTORA consistently outperformed Q-learning, Weighted Round Robin, and Random strategies, particularly under scenarios with high UE mobility and dynamic task generation, underscoring its suitability for *real-time MEC scheduling*.

In highly dynamic cloud environments, the study in [175] integrated Interval-Valued Pythagorean Fuzzy Sets (IVPFS) with an enhanced Dyna Q+ agent to address scheduling under uncertainty. The IVPFS handled vague system conditions (e.g., security, pricing, traffic), while Dyna Q+ incorporated exploration bonuses to improve learning efficiency. The combined scheduler targeted *independent tasks* on *heterogeneous cloud systems* and was evaluated using CloudSim 3.3 with random and GOCJ datasets. Results demonstrated substantial improvements: 90% reduction in execution time and *makespan*, 50% drop in *cost*, and 95% increase in *resource utilization*, outperforming traditional Dyna-Q and fuzzy-logic-based schedulers.

Extending reinforcement learning to distributed cloud environments, Farooq et al. [176] proposed FR-EAHTS, a Federated Reinforced Energy-Aware Heterogeneous Task Scheduling framework for *independent tasks* on *heterogeneous multi-core systems*. The method integrated transformer-based state-action encoding (TransformX) and Adaptive Proximal Policy Optimization (APPO) within a federated reinforcement learning (FRL) paradigm. The architecture enabled both global and local hierarchical load balancing while leveraging Dynamic Voltage and Frequency Scaling (DVFS) to reduce *energy consumption*. Additional components included a Dynamic Learning Rate (DLR) adjustment strategy, Parameter Dynamic Adjustment (PDA), and Dynamic Heuristic Load Distribution (DHLD) to enhance learning stability and scheduling responsiveness. Experimental results using PARSEC 3.0 benchmarks revealed that FR-EAHTS reduced *makespan* by 30%, *energy* by 35%, and *scheduling overhead* by 40%, significantly outperforming traditional and centralized RL schedulers.

For IoT-edge computing scenarios, Ghahari-Bidgoli et al. [177] introduced JCOARS, a joint computation offloading and autonomous resource scaling framework designed to minimize *latency*, *energy usage*,

and *cost*. The system operated on *independent IoT tasks* in a *heterogeneous edge environment*, using Deep Q-Reinforcement Learning (DQRL) to coordinate three modules: an offloading manager for routing delay-sensitive and non-delay-sensitive tasks to edge or cloud nodes, a scaling manager that executed horizontal scaling based on cost, energy, and distance constraints, and a learning processor that updated scheduling policy online. Evaluation results showed a 25.7% reduction in average latency, 8.9% cost reduction, and marginal energy improvement (0.9%) compared to baseline strategies. This model-free RL approach supports dynamic workload adaptation and robust edge resource management.

In the context of hierarchical scheduling for complex task dependencies, Zhang et al. [178] proposed MD-HRL, a Multi-agent Deep Hierarchical Reinforcement Learning framework designed for *dependent tasks* on *heterogeneous computing clusters*. The architecture featured a two-level agent system: a high-level agent (H-Agent) utilizes multi-hop attention graph neural networks (MAGNA) and 1D convolution networks to encode task and resource features, while a low-level agent (L-Agent) employed a Double Deep Q-Network (DDQN) to determine optimal task-to-resource mappings. Kolmogorov–Arnold Networks (KAN) were used to assign subtask priorities. The reward model jointly optimized *makespan* and *energy* while promoting *load balance*. Experimental results demonstrated that MD-HRL achieved significant improvements over conventional heuristics and DRL models, especially in large-scale task graphs.

A more structured approach for DAG-based task scheduling was proposed by Wang et al. [100] in a framework called Multiobjective Optimization Deep Reinforcement Learning (MODRL). The method explicitly combined a *Spatio-Temporal Graph Attention Network (STGAT)* to capture DAG task dependencies and execution order with a *Set Transformer* to encode cluster states in a permutation-invariant manner, thus uniting GNN-based relational modeling with transformer-based scalability. The model targeted *dependent tasks* on *heterogeneous high-performance systems*, where a Dueling Double Deep Q-Network (D3QN) assigned tasks while optimizing both *makespan* and *load balancing*. To explore Pareto trade-offs, parameters were transferred between agents trained on different weighted reward configurations. Experimental evaluation on synthetic and Alibaba-trace DAGs confirmed that MODRL achieved faster convergence and superior scalability compared to heuristic and DRL-only baselines.

Complementing this, Zhou et al. [179] proposed GG-DRL, a DRL-based scheduler for *dependent DAG tasks* on *heterogeneous multi-core systems*. The system used a graph preprocessing module combining GAT and GRU to capture task dependencies and workload patterns. Training was conducted using an Actor-Critic (A2C) strategy with dual objectives: minimizing *makespan* and maximizing processor *load balance*. Experiments demonstrated that GG-DRL outperformed traditional methods (HEFT, GA) and baseline DRL models in terms of convergence speed and scheduling efficiency.

Li et al. in [181] addressed the challenge of ensuring fault-tolerance in *dependent DAG workflows* executed on *heterogeneous MEC servers*, where frequent node failures can compromise workflow completion. Traditional reliability-oriented schedulers often relied on blanket replication or task resubmission, which significantly inflated *makespan* and resource usage. To overcome these limitations, the authors designed a DRL framework that integrated *graph attention coding* with an Actor-Critic learning paradigm. Specifically, a Graph Attention Network (GAT) was used to encode workflow DAGs and capture task dependencies, while a two-layer LSTM Actor jointly decided both the replication degree of each task and its placement on heterogeneous servers. Invalid scheduling actions were masked to respect dependency and placement constraints, and the Critic guided policy updates with reliability-aware rewards that penalize violations. Simulation results on synthetic MEC platforms demonstrated that the proposed method sustained workflow completion ratios close to 100% under varying server failure rates, while also reducing average *makespan* and server active time compared to baseline schemes such as Greedy, GSMS, and FTAW. By selectively

Table 9

Summary of deep reinforcement learning-based works for independent tasks with objectives: Energy (E), Temperature (T), Fault and Reliability (F), Makespan (M), and Deadline (D).

Ref.	Learning type	Hardware platform	E	T	F	M	D	Methodology summary	Evaluation & Results	Limitations
[161]	DQN (RL-TS)	Homogeneous – Cloud clusters	✓	×	×	×	×	Learns scheduling from runtime rewards for large workloads	81% success, 27% energy saving on 1000–5000 jobs	Slightly higher finish time
[109]	DDQN DVFS governor	Homogeneous – Jetson Nano (ARM A57)	✓	×	×	×	×	Kernel-side DDQN agent with temporal encoding for soft-deadline tasks	3%–14% energy savings vs. Linux governors	Limited to periodic tasks; per-workload training needed
[167]	DDQN (DVFS governor)	Homogeneous – Jetson Nano quad-core	✓	×	×	×	✓	GRU/MLP encoders with DDQN for deadline-aware energy-efficient DVFS	3%–10% power savings; low deadline misses	Only one SoC; sparse rewards; floating-point in kernel
[162]	DQN for PPW optimization	Heterogeneous – ARM big.LITTLE (Odroid XU3)	✓	×	×	×	×	Controls active big/LITTLE cores and frequency; prioritized replay	PPW within 1% of Oracle; 91% action accuracy	Costly offline training; sensitive to reward design
[166]	Double DQN (DDQN-TS)	Heterogeneous – Cloud VMs	×	×	×	×	✓	Offline-trained DDQN for SLA-aware scheduling	Higher completion rates; faster convergence vs. DQN, RR	Inference overhead for large workloads
[168]	Blockchain + DQN (BD-TTS)	Heterogeneous – Edge computing	×	×	×	×	✓	Trust-aware DRL with blockchain-based server reliability	64% fewer untrusted allocations; lower response time	Blockchain consensus overhead
[169]	DQN (Q-scheduler)	Homogeneous – real-time multiprocessors	✓	✓	×	×	×	DQN-based EDF scheduler minimizing energy and temp	15% energy, 10% thermal savings vs. heuristics	Offline training; non-migrating tasks
[170]	DQN (DRLS)	Heterogeneous – Mobile (GPU+NPU)	✓	✓	×	×	×	Thermal-aware scheduling of DNN tasks with replay buffer	Lower temps, higher accuracy vs. GPU/NPU-only	Heavier runtime overhead; fixed workloads
[171]	DQN (DRFTSA)	Heterogeneous – Cloud datacenters	✓	×	✓	✓	×	Fault-tolerant + energy-aware scheduler using DQN	37% makespan, 46% failure, 29% energy reduction	Training cost; CloudSim only
[91]	A3C (MOPTSA3C)	Heterogeneous – Multi-cloud	×	×	✓	✓	×	A3C-based multi-objective scheduler for independent tasks	Better makespan, cost, utilization, reliability	Scalability challenges for large-scale clouds
[173]	DDPG (HDDPGTS)	Heterogeneous – Cloud HPC/HTC	✓	×	×	✓	×	DDPG-based actor-critic for HPC/HTC tasks	Improves makespan, energy, overhead, scalability	Higher inference latency; large workloads
[50]	DQN (SLTRA)	Heterogeneous – Cloud/Edge	✓	×	×	×	✓	DQN offloading + allocation with entropy reward	Reduced latency + energy vs. QL, WRR	Adaptability requires tuning
[174]	DQN (DTORA)	Heterogeneous – MEC with mobility	✓	×	×	×	✓	DQN-based MEC scheduler optimizing RT+energy	Better latency/energy under UE mobility	Limited to Poisson arrivals; MEC-specific
[175]	Dyna-Q+ with IVPFS	Heterogeneous – Cloud systems	×	×	×	✓	×	Fuzzy Dyna-Q+ for scheduling under uncertainty	90% less makespan, 50% lower cost	Fuzzy sets complex to tune
[176]	APPO + Transformers (FR-EAHTS)	Heterogeneous – Cloud multi-core	✓	×	×	✓	×	Federated DRL with TransformX + APPO	30% MS, 35% energy, 40% overhead improvement	Federated complexity; heavy training cost
[177]	DQRL (JCOARS)	Heterogeneous – IoT Edge/Cloud	✓	×	×	×	×	Joint offloading + resource scaling via DQRL	25.7% latency, 8.9% cost, 0.9% energy gains	Small energy improvement; complexity
[180]	D3QN + PER (EETS)	Heterogeneous – Cloud VMs	✓	×	×	×	✓	D3QN with prioritized replay for cloud tasks	Better energy, RT vs. DQN/DDQN	Scalability to huge clouds limited

replicating tasks according to DAG structure, the approach achieved strong *reliability* guarantees while avoiding unnecessary overhead.

Wang et al. [77] presented a DRL framework for *continuous workflow scheduling in heterogeneous cloud environments*, addressing the limitations of heuristics and existing DRL schedulers in handling concurrent workflow arrivals. The framework incorporated a *Graph Attention Network (GAT)* to capture task dependencies within each workflow and a *self-attention multilayer perceptron (MLP)* to encode dynamic features of ready tasks and heterogeneous processors. To manage the dynamic and large action space, a *k-dimensional tree (KD-Tree)* combined with a *k-nearest neighbors (kNN)* mapping algorithm was employed, enabling efficient translation of continuous policy outputs into discrete task–processor assignments. The reward function was explicitly multiobjective, jointly minimizing *average makespan* and improving *CPU/memory load balancing* across processors. Extensive experiments

on the *TPC-H benchmark* and *Alibaba Trace dataset* demonstrated that the proposed method converged faster and outperformed state-of-the-art DRL approaches (e.g., Decima [84], Dream [86]) and heuristics (Dynamic extensions of HEFT and CPOP [47]), achieving up to 45.1% makespan reduction while enhancing load balance efficiency. However, the method exhibited scalability challenges for very large workflow sequences and incurred higher inference latency than heuristic baselines due to neural network computation overheads.

Focusing on energy-efficient scheduling under batch workloads, the *EETS* algorithm [180] introduced a Deep Reinforcement Learning approach for cloud task scheduling using an enhanced D3QN with Prioritized Experience Replay (PER). Designed for *independent tasks* on *heterogeneous virtual machines*, the model balanced *execution time*, *queue waiting time*, and *energy consumption* through a carefully constructed reward function. Evaluation results showed that EETS significantly

Table 10

Summary of deep reinforcement learning-based works for dependent tasks with objectives: Energy (E), Temperature (T), Fault and Reliability (F), Makespan (M), and Deadline (D).

Ref.	Learning type	Hardware platform	E	T	F	M	D	Methodology summary	Evaluation & Results	Limitations
[59]	PPO MARL	Heterogeneous – Static DAG	×	×	×	✓	×	PPO agents pick task priorities; EFT mapping	9%–28% MS improvement vs. HPS, PETS	Limited to static DAGs
[119]	R-DQN	Heterogeneous – Cloud workflows	×	×	×	✓	×	DNN with replay/target net for workflow scheduling	Lower makespan; robust vs. MAXMIN, HEFT, ACS	NN inference overhead; replay buffer costs
[105]	Actor–Critic + GAT (DRL-GAT)	Homogeneous – DAG multi-core	✓	×	×	×	×	GAT encodes task deps; DRL sets DVFS levels	Energy within 10% of optimal; good schedulability	Needs offline training; heuristic task mapping
[160]	Transformer-GCN + PPO/A2C for DAG scheduling	Heterogeneous – multi-core processors	×	×	×	✓	×	Splits task-node selection (DNN) and processor allocation (heuristic EFT); Transformer-enhanced GCN extracts DAG features	Reduces average makespan by 10.89% vs. QL-HEFT, FCFS-HTDG, Smart-mDAG-EFT, OCT-HEFT, IPPTS	Limited to batch-mode DAG arrivals; does not consider task priority scenarios
[163]	Hybrid fuzzy + DRL + ROA (HFDRL)	Heterogeneous – multi-core	✓	×	×	×	×	DRL+fuzzy+ROA for DAG scheduling	Lower exec time + energy vs. GA, PSO	High complexity for large DAGs
[16]	GA + DQN	Heterogeneous – Cloud DAGs	×	×	×	×	✓	GA explores mappings; DQN refines VM assignment	Lower response time + cost vs. heuristics/DRL	Scalability issues; single agent
[178]	Hierarchical DRL (MD-HRL)	Heterogeneous – DAG clusters	✓	×	×	✓	×	Multi-hop GNN + DDQN hierarchical agents	Better MS, energy, load balance	Training overhead; complexity
[100]	MODRL (STGAT + Set Transformer + D3QN)	Heterogeneous – HPC DAGs	×	×	×	✓	×	Optimizes MS + load balance via multiobjective D3QN	Superior convergence + scalability vs. DRL/heuristics	Computationally heavy
[179]	GG-DRL (GAT + GRU + A2C)	Heterogeneous – DAG multi-core	×	×	×	✓	×	Actor–Critic with GAT+GRU encoders	Faster convergence; better MS vs. HEFT, GA	Limited eval; sim only
[181]	GAT + Actor–Critic	Heterogeneous – MEC DAGs	×	×	✓	✓	×	Reliability-aware DRL with selective replication	100% workflow completion under failures	Overhead from coding + replication
[77]	GAT + Self-attention MLP + DRL	Heterogeneous – Cloud DAGs	×	×	×	✓	×	DRL with KD-tree mapping for large workflows	45% MS reduction vs. Decima, DREAM	Higher inference latency
[182]	POMDP + Active Inference	Heterogeneous – multi-core DAGs	×	×	✓	×	✓	Reliability-aware with partial observability	Better deadlines + reliability on Jetson Orin	Complex; higher compute overhead

outperformed baseline DQN and DDQN approaches in terms of *energy savings*, improved *deadline satisfaction*, and enhanced learning stability particularly under high task loads.

Finally, addressing uncertain execution environments, the framework by Zou and Gray [182] combined Partially Observable Markov Decision Processes (POMDP) with Active Inference to optimize *dependent task scheduling* on *heterogeneous multi-core systems*. Unlike kernel-level solutions, this application-layer approach incorporated task criticality and partial observability, using multivariate Gaussian models for performance feature estimation. A free energy minimization principle guided belief updates and action selection, enabling probabilistic adaptation to unseen workloads. Experiments on Jetson Orin platforms using CoreMark-Pro benchmarks showed notable improvements in *deadline adherence*, *load balancing*, and overall *reliability* compared to ML and heuristic baselines.

As summarized in Tables 9 and 10, DRL-based schedulers extend classical RL solutions by enabling function approximation over high-dimensional and structured task states. The reviewed literature reveals that in *homogeneous settings*, research remains relatively sparse, with works such as Q-scheduler primarily targeting energy and thermal objectives. By contrast, the majority of contributions addressed *heterogeneous environments*, where DRL demonstrated strong adaptability in balancing multiple objectives across cloud, edge, and embedded platforms.

Across surveyed works, DRL has proven effective in capturing workload dependencies (via GNNs, Transformers), handling continuous control spaces (via DDPG, PPO), and integrating auxiliary techniques such as federated learning, fuzzy logic, and blockchain for trust management. These innovations have enabled notable gains in *makespan reduction*, *energy conservation*, *reliability*, and *adaptability*. However, DRL

approaches continue to face practical challenges. Neural inference overhead can undermine real-time applicability, reward engineering remains task- and platform-specific, and generalization across unseen workloads or hardware architectures is limited. Future directions may include lightweight DRL models optimized for on-device inference, hybrid learning frameworks that combine DRL with meta-heuristics, and standardized benchmarks for fair cross-platform evaluation. Additionally, closer integration with emerging paradigms such as federated reinforcement learning, GNN/Transformer-based end-to-end schedulers, and reliability-aware DRL will be essential to meet the demands of next-generation heterogeneous computing systems.

10. Experimental environments

To evaluate the performance of task scheduling algorithms, a well-defined experimental environment is essential. This section describes the datasets used for experimentation and the performance metrics employed to assess the algorithms. The comprehensive overview and comparative assessment of datasets used in learning-based scheduling techniques, focusing on data characteristics, and workload diversity, is one of the unique contributions of this survey paper.

10.1. Datasets used in prior studies

A variety of datasets have been used in prior studies to evaluate learning-based scheduling, power management, thermal modeling, and performance optimization techniques. These datasets include synthetic, benchmark, and real-world data, providing comprehensive testbeds for comparing algorithmic performance under diverse conditions. This section summarizes the most common datasets and their applications.

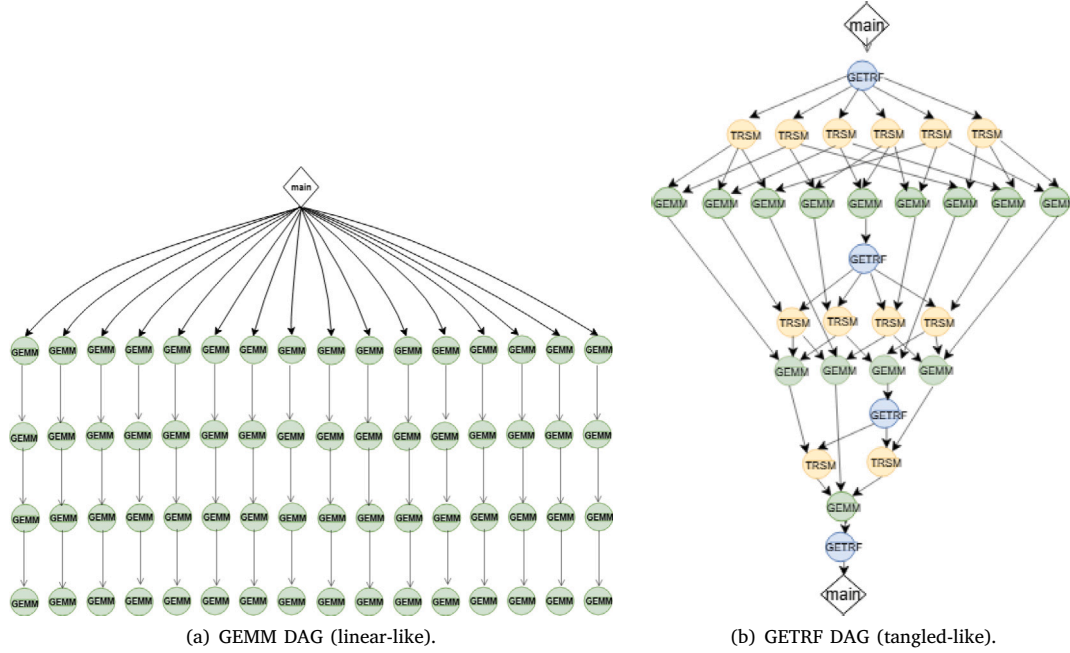


Fig. 7. Task-graph structures of (a) GEMM and (b) GETRF workloads from the MatRIS library, illustrating their similarity to synthetic linear and tangled DAGs generated by DAGGER [81].

10.1.1. Synthetic and benchmark datasets

Synthetic datasets provide controlled environments to systematically assess scheduling algorithms. Researchers often generate synthetic workflows of varying sizes (tiny, small, medium, large) to evaluate execution times and resource usage. Custom simulators implemented in C++ are also used to produce synthetic task sets, enabling flexible testing under diverse workload conditions. In contrast, benchmark suites such as MiBench [183] supply embedded applications that reflect real-world, resource-constrained scenarios. For multi-threaded settings, artificial workloads may be created by combining actual multi-threaded programs, thereby introducing heterogeneous phase diversity and effectively stress-testing dynamic resource management (DRM) techniques. Similarly, the *Task Graphs For Free* (TGFF) benchmark [184] enables the generation of randomized task graphs for standardized evaluation of scheduling and allocation methods. A major strength of synthetic datasets lies in the ability to control workload size, dependency density, and noise levels in task execution times. By tuning these parameters, researchers can isolate the impact of scheduling algorithms on metrics such as *makespan*, and *deadline miss rate*. Small, noise-free task graphs are useful for validating theoretical schedulers, whereas larger graphs with injected variability offer insights into robustness under real-world uncertainties. TGFF-generated task graphs can also be adapted for diverse domains, ranging from embedded workloads to cloud-scale DAGs, although their generality limits direct applicability to highly domain-specific cases (e.g., avionics or automotive control). Beyond generic evaluation, recent studies highlight structural similarities between synthetic DAGs and real workloads. For instance, IRIS-GNN [81] demonstrated that the DAG of *General Matrix Multiplication* (GEMM) closely resembles a linear chain, while LU factorization (GETRF) maps to tangled DAG structures (Fig. 7). Such correspondences validate the representativeness of synthetic DAG generators like TGFF or DAGGER, since they reproduce dependency characteristics observed in mathematical kernels. Consequently, synthetic DAGs not only provide scalable testbeds under controlled conditions but also yield realistic insights into scheduling performance for scientific and high-performance computing workloads.

10.1.2. Scientific workflow benchmarks

Scientific workflows are widely used to evaluate scheduling under data-intensive and compute-intensive conditions. They capture domain-specific DAG structures that can be scaled to larger instances while preserving inherent dataflow and dependency characteristics. Closed-form scaling relations for representative workflows such as *Epigenomics*, *Laplace*, *Stencil*, and *Gaussian Elimination* are available in the literature [5], highlighting how problem size directly influences DAG growth. The suitability of these benchmarks also depends on their adaptability to different application domains, dataset sizes, and performance sensitivities. Fig. 8 shows the commonly used scientific workflow benchmarks in scheduling. Representative examples include:

- *Montage* (astronomical image mosaicking) [187]: generates science-grade mosaics of the sky from FITS-format images. Its workflow consists of multiple stages, including reprojection (mProjectPP), difference computation (mDiffFit), background modeling and correction (mBgModel, mBackground), and final mosaic assembly (mAdd). Additional steps such as metadata aggregation (mImgTbl), image shrinking (mShrink), and format conversion (mJPEG) highlight its modular and scalable design (Fig. 9). Montage is highly I/O-intensive and suitable for large-scale data-intensive scheduling and bandwidth/storage evaluation. However, it is less representative of latency-critical domains such as autonomous driving or industrial control, where smaller DAGs with variability and noise are preferred for robustness and deadline adherence. For Pegasus-based execution, Montage can be scaled by increasing domain-specific parameters such as the mosaic dimension.
- *CyberShake* (seismic hazard analysis) [186] and *Broadband* (seismic waveform modeling): represent large-scale earthquake simulation workflows. These workflows stress inter-task communication, synchronization, and dependency management under massive DAGs but without real-time constraints. They are excellent for scalability and communication-intensive studies, though less suited for embedded real-time systems. In Pegasus, scaling is

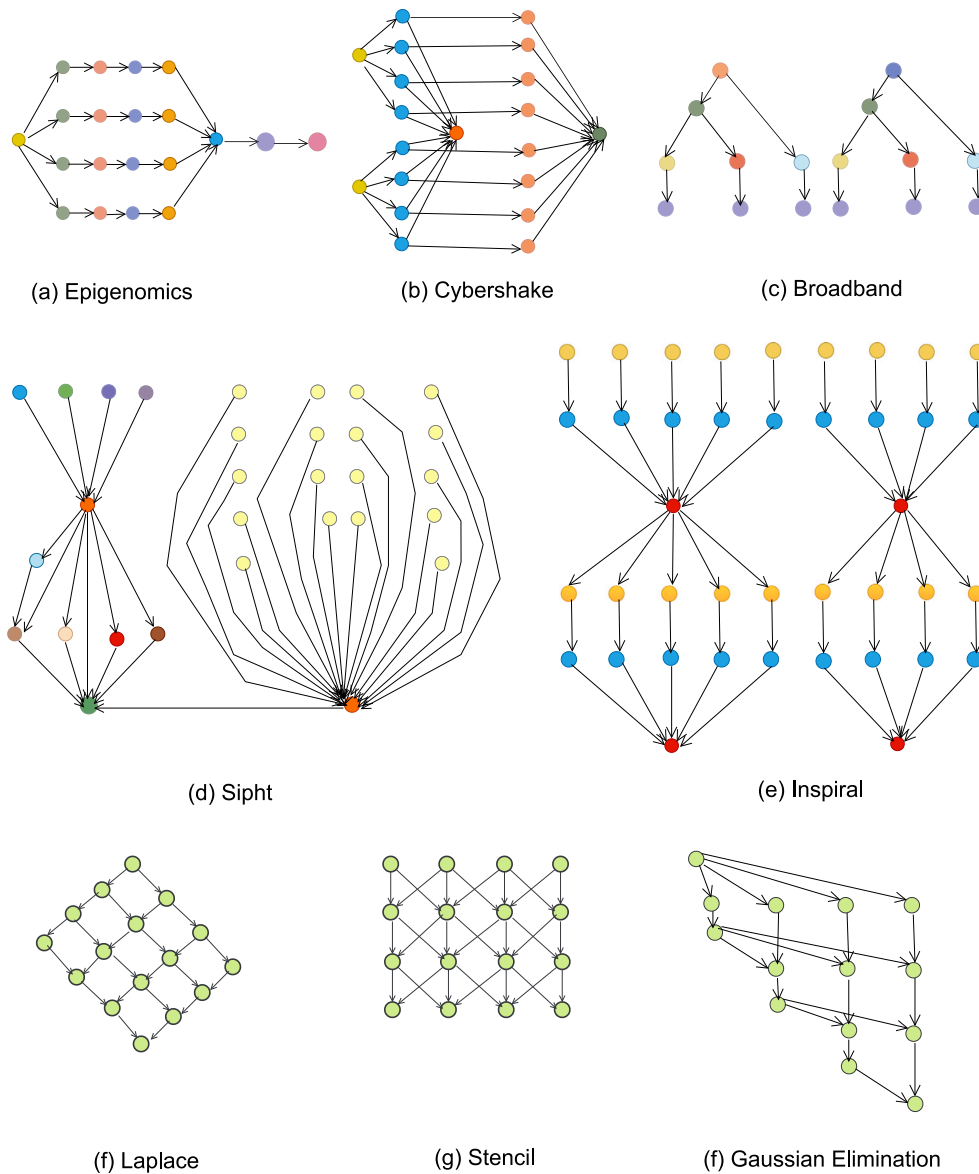


Fig. 8. Scientific workflows: (a) Epigenomics [185], (b) CyberShake [186], (c) Broadband [185], (d) SIPHT [187], (e) Inspiral [185], (f) Laplace [5], (g) Stencil [5], (h) Gaussian Elimination [188].

typically achieved through the number of seismic stations multiplied by rupture events, which directly expands DAG size and communication overhead.

- *Epigenomics* (genomic sequencing pipelines) [185] and *SIPHT* (RNA motif discovery) [187]: capture large-scale bioinformatics workloads. Epigenomics emphasizes pipeline parallelization and memory efficiency in genome sequencing, while SIPHT models genome-wide search tasks with heavy data dependencies. Both are valuable for evaluating data-parallel schedulers, where dataset size directly influences robustness, resource utilization, and fault recovery. In terms of scaling, Epigenomics produces $4(\gamma + 1)$ tasks and $5\gamma + 2$ messages for γ genome fragments, demonstrating a predictable growth pattern with input size.
- *Inspiral* (gravitational wave signal detection) [185]: models astrophysical workflows for gravitational physics. It is compute-intensive and dominated by floating-point operations, making it valuable for evaluating CPU-bound scheduling strategies. Larger instances stress scalability, while smaller DAGs can be used to study resilience under noisy conditions. Pegasus-based scaling is

typically controlled by the number of sequence files or event traces.

- *Laplace* (finite-difference PDE solver) and *Stencil* (iterative numerical kernels) [5,58]: capture iterative PDE-based computations characteristic of scientific HPC workloads. They stress repeated structured updates on multidimensional grids and highlight scalability across large problem sizes. Specifically, a Laplace grid of size φ generates φ^2 tasks and $2\varphi(\varphi - 1)$ messages, while a Stencil workload with λ levels and ξ nodes per level produces $\lambda\xi$ tasks and $(\lambda - 1)\{3(\xi - 2) + 4\}$ messages. However, they are less representative of irregular DAGs in embedded or control-oriented domains, where smaller datasets and non-uniform patterns are more realistic.
- *Gaussian Elimination* (dense linear algebra solver) [58,188]: focuses on dense matrix factorization kernels, emphasizing computational intensity rather than data transfer overheads. It exhibits quadratic-cubic scaling behavior, where a matrix of size χ yields $\frac{\chi^2 + \chi - 2}{2}$ tasks and $\chi^2 - \chi - 1$ messages. This makes it highly suitable for stressing computational limits in schedulers. Dataset size strongly affects scalability, making it useful for stressing CPU

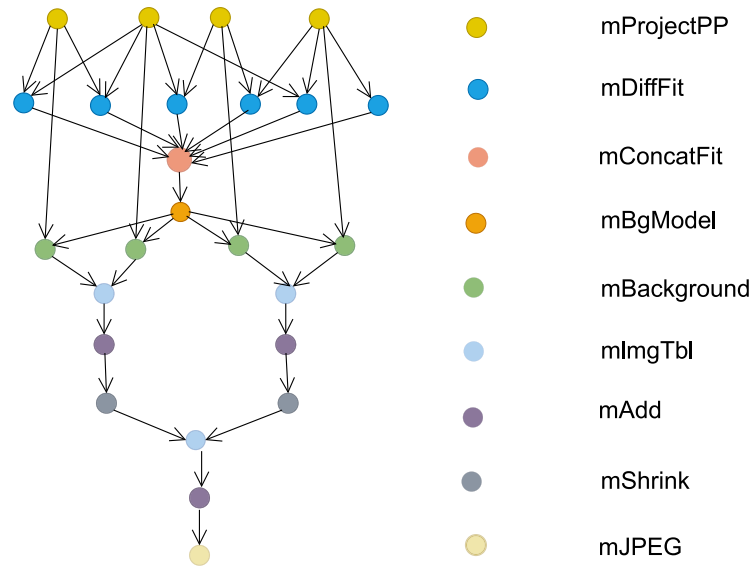


Fig. 9. Example Montage workflow [187] DAG illustrating key processing stages for generating astronomical image mosaics.

utilization but less relevant for evaluating communication-heavy scheduling.

- *FFT (radix-2)* (canonical signal-processing DAG) [47]: a highly parallelizable kernel central to *Digital Signal Processing (DSP)* and *Orthogonal Frequency Division Multiplexing (OFDM)* communication systems. A radix-2 FFT of size N generates $\frac{N}{2} \log_2 N$ tasks and $N \log_2 N$ messages distributed across $\log_2 N$ stages, highlighting its logarithmic scaling properties. This makes it valuable for evaluating parallel scheduling performance. However, as a highly regular DAG, FFT is less representative of irregular, noisy control workloads often encountered in real-time embedded systems.

These workflows are commonly managed and generated using the *Pegasus workflow management system* [187], which provides tools to define, map, and execute complex applications as DAGs across distributed computing platforms. Pegasus can also generate synthetic traces that approximate real execution patterns, while task arrivals are often modeled using *Poisson processes* to reflect stochastic workload behavior. To evaluate scheduling strategies, simulation environments such as *CloudSim* [189] and its extension *WorkflowSim* [190] are widely used. CloudSim provides a flexible platform for modeling cloud data centers, virtual machines, heterogeneous computing resources, and scheduling policies. WorkflowSim enhances CloudSim by supporting DAG-based workflow modeling, task dependencies, and inter-task communication delays, making it well-suited for developing, training, and testing machine learning-based scheduling algorithms under realistic cloud execution conditions.

Consequently, the selection of a benchmark strongly influences the performance metrics that matter most in evaluation. Scientific workflows generally highlight metrics such as *makespan* and *resource utilization*, whereas embedded and control-oriented applications (e.g., adaptive cruise control, see Fig. 3 in Section 4) place greater emphasis on *deadline satisfaction rates* and *fault recovery times*. A detailed discussion of performance metrics is presented in Section 10.2.

10.1.3. Cloud and production traces

Real-world production traces are increasingly used to evaluate scheduling techniques in large-scale, heterogeneous cloud environments, as they capture realistic arrival patterns, workload variability, and resource heterogeneity that synthetic datasets cannot. The *Alibaba Cluster Trace* [191] contains logs from thousands of machines in Alibaba's datacenters, including batch jobs, service workloads, and AI training tasks. It has become a standard benchmark for ML-, DL-,

and RL-based scheduling studies. Similarly, the *Google Cluster Trace (GCT)* [192], collected over 29 days from more than 12,000 machines in Google's production clusters, includes detailed job, task, and machine events, and has been widely used for large-scale scheduling, failure analysis, and resource modeling. In contrast, the *GoCJ dataset* [193] is a synthetic dataset generated via Monte Carlo simulation based on statistical characteristics of the Google Cluster Trace. It provides job submissions, resource requests, and execution logs in a lightweight format suitable for algorithm prototyping and comparative evaluation. Together, these traces offer diverse, realistic, and complementary benchmarks for assessing the scalability, adaptability, and robustness of scheduling approaches.

Beyond these widely adopted traces, Liu et al. presented a comprehensive review of publicly available cloud datasets [194], in which 42 datasets were systematically analyzed and categorized across 11 dimensions such as data content, scale, and application domain. Representative examples include the *Azure LLM Inference Dataset* (Microsoft, 2024), which records latency, throughput, memory, and power profiles of large language model inference workloads; the *Alibaba GPU Traces* (2020–2023), capturing hybrid training and inference jobs across 6,500 GPUs; and the *SURFsara Dataset* (2019), which provides multi-domain scientific workloads with detailed CPU/GPU utilization and energy metrics. Other host-cluster traces, such as the *Google Power Data* (2024) and *Helios* (SenseTime, 2020), include explicit power and temperature measurements, making them valuable for energy- and thermally-aware scheduling research. Incorporating such public cloud datasets into benchmark studies improves *reproducibility*, by enabling consistent and transparent evaluation across research efforts, and enhances *realism*, by reflecting the diverse, dynamic, and energy-aware characteristics of modern large-scale cloud environments.

10.1.4. Database and simulation benchmarks

In addition to production traces, standardized benchmarks and simulation frameworks are widely employed to evaluate scheduling under controlled conditions. The *TPC-H benchmark* [195] is a decision-support workload consisting of 22 parameterized SQL queries executed over synthetic databases with scale factors ranging from gigabytes to terabytes. Query execution plans derived from TPC-H are often modeled as DAGs, enabling their integration into workflow scheduling studies. The *CloudSim 3.3 toolkit* [189] provides a configurable simulation environment for modeling virtual machine provisioning, job scheduling, and energy-aware resource management in cloud datacenters. While TPC-H offers reproducibility in database-centric scenarios, CloudSim enables

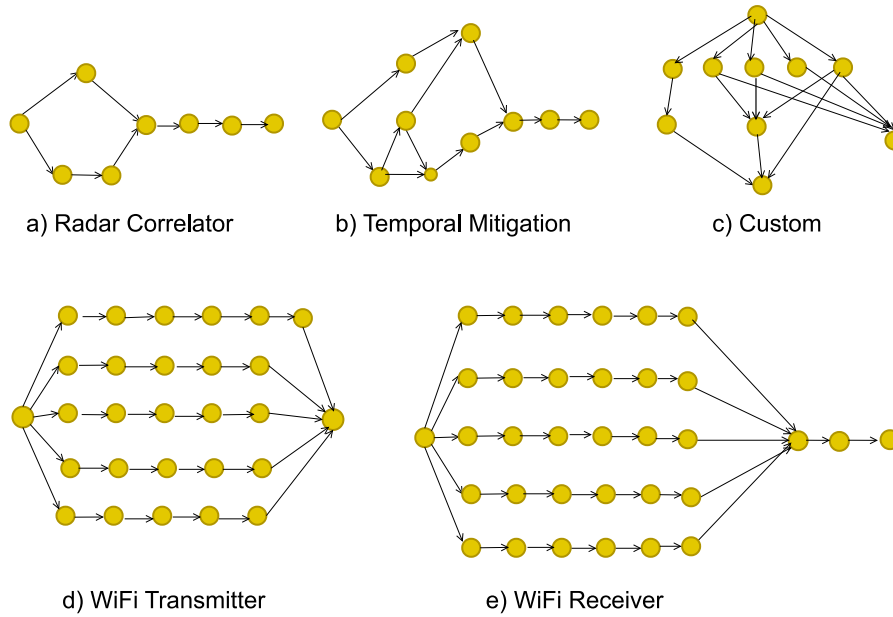


Fig. 10. Structures of four real-world workflows and one custom workflow used in the DS3 simulator.

flexible, large-scale experimentation without relying on production traces.

10.1.5. DS3 simulator-based workflow benchmarks

While scientific workflows primarily represent large-scale, data-intensive domains, the DS3 simulator [196] is tailored toward *embedded and communication-driven workloads*. It provides a high-level evaluation platform for workflow scheduling on heterogeneous systems, incorporating diverse cores (e.g., ARM Cortex-A7, A15, A53, A72) alongside specialized accelerators such as FFT units, matrix multipliers, and Viterbi decoders. DS3 includes five representative workflows: *Radar Correlator*, *Temporal Mitigation*, *WiFi Transmitter*, *WiFi Receiver*, and a Custom workflow covering a broad spectrum of DAG complexities, task counts, depths, and computational intensities (Fig. 10). This makes DS3 particularly effective for analyzing scheduling performance under heterogeneous hardware–software interactions that are often not captured by synthetic DAG generators.

A key advantage of DS3 lies in its ability to balance communication- and computation-driven workload characteristics. For example, the *Radar Correlator* and *WiFi* workflows model communication-intensive embedded applications, whereas the *Temporal Mitigation* workflow emphasizes computation-heavy phases. By varying DAG size and depth, DS3 supports scalability studies, while controlled injection of noise into task execution times enables robustness analysis under non-deterministic runtime conditions.

In terms of evaluation metrics, DS3 benchmarks extend beyond classical *makespan* and *energy consumption* by enabling the study of real-time objectives such as *deadline satisfaction rates* and robustness indicators like *fault recovery time* under task replication or rescheduling. These features make DS3 well-suited for embedded signal processing and wireless communication domains, where both performance and reliability are critical. However, compared to domain-specific real workloads, DS3 workflows remain abstractions; hence, their scenario coverage should be interpreted with caution when generalizing results to industrial systems.

10.1.6. High-performance computing and multi-core benchmarks

In contrast to workflow and simulator-based datasets, high-performance computing (HPC) and multi-core benchmarks emphasize computational intensity, memory behavior, and synchronization. These

suites are particularly valuable for evaluating scalability, parallel efficiency, and load balancing across both shared-memory and distributed architectures. For classical HPC workloads, the NAS Parallel Benchmarks [197] provide a collection of CPU- and memory-bound kernels implemented in serial, OpenMP, and MPI versions. Their adaptability stems from multiple problem size classes (A–E), which scale computational intensity and dataset size, making them well-suited for studying how scheduling algorithms respond to workload scaling on clusters and supercomputers.

Similarly, PARSEC-3.0 [198] and SPLASH-2 [199] are widely adopted for shared-memory multi-core systems. These suites include multithreaded applications that stress cache coherence, synchronization, and data sharing. They also provide tunable input sets (*simsmall*, *simmedium*, *simlarge*) that enable systematic exploration of how workload size and data-sharing patterns influence scheduling performance and scalability. This makes them particularly effective for studying dynamic scheduling and load-balancing strategies. The LINPACK benchmark [200], primarily designed to measure floating-point throughput in dense linear algebra problems, remains the de facto standard for ranking HPC systems on the TOP 500 list. Although it primarily targets peak throughput evaluation rather than scheduling efficiency, it still serves as a stress test for dense numerical workloads. For micro-architectural and energy-focused studies on low-power multi-core processors, benchmarks such as COSMIC [201] and CAP Bench [202] provide fine-grained control over cache behavior and replacement policies. These benchmarks facilitate evaluation of both performance and energy consumption, while also highlighting robustness under cache stress, a property especially relevant for embedded and real-time systems where predictable memory access is critical.

Over the years, there were multiple research works focused on developing tools for simulating scheduling algorithms in multi-core platforms. Examples include *STORM* [134], *YARTISS* [203], *SimSo* [204]. These simulation tools designed to evaluate and compare multiprocessor scheduling policies. The ultimate goal of these tools is to enable reproducible experiments, helping to either confirm or invalidate existing research results. The JADE simulator [205] further complements these efforts by modeling detailed multi-core architectures with integrated power models, thereby supporting analysis of deadline satisfaction and fault recovery when scheduling policies are tested under varying runtime perturbations. Finally, to validate simulation-based

findings, researchers often conduct experiments on commodity multi-core CPUs such as Intel quad-core i7 processors. Such real hardware evaluations capture variability due to operating system noise, thermal throttling, and dynamic power management, thus bridging the gap between simulation results and realistic execution environments.

Together, these HPC and multi-core benchmarks support the evaluation of classical performance metrics such as *makespan*, *throughput*, and *energy consumption*, while also enabling the study of domain-specific indicators including *deadline satisfaction rates* and *robustness* in embedded and low-power contexts. As such, they complement workflow-based benchmarks and provide a more holistic perspective on scheduling algorithm performance across diverse computing environments.

10.1.7. Thermal, power and fault simulation environments

Beyond performance-centric benchmarks, evaluating scheduling under energy, thermal, and reliability constraints requires specialized traces and models. Such datasets are typically collected or modeled using tools like Intel's Performance Counter Monitor (IPCM), the HotSpot simulator, and SPICE-based models, each providing unique insights into micro-architectural events, thermal profiles, and fault behaviors that directly affect robustness and real-time predictability. IPCM [206] provides real-time measurements of events such as core frequency, cache hit/miss rates, temperature, and power states. This makes it particularly useful for evaluating scheduling policies on commodity multi-core platforms under dynamic and noisy operating conditions, such as workload interference or operating system jitter. Dataset size and variability in IPCM traces directly affect how robustly a model generalizes across workloads. Gem5 [207] is a simulation infrastructure that combines the best features of the M5 and GEMS simulators. M5 provides a highly configurable framework with various CPU models and instruction set architectures (ISAs), while GEMS contributes a detailed memory system with support for different cache coherence protocols and interconnect models. Currently, gem5 supports most commercial ISAs, and can even boot the Linux operating system on three of them (ARM, ALPHA, and x86). Given its collaborative nature, the success of its predecessors, and its liberal, BSD-like license, gem5 is considered a valuable tool for full-system simulation. HotSpot [208], a compact thermal modeling tool, estimates temperature profiles across processors given floorplans and power traces. It is widely used in embedded and mobile computing research where thermal hotspots can cause deadline misses due to throttling. By scaling input power traces, HotSpot enables testing of scheduler adaptability across low-power and high-power regimes.

McPAT [209] is a framework for modeling the power, area, and timing of multi-core and many-core processors. It supports a wide range of process technologies, from 90 nm down to 22 nm and beyond. At the microarchitectural level, McPAT can model key chip components like processor cores (both in-order and out-of-order), on-chip networks, shared caches, and integrated system components such as memory and Ethernet controllers. The framework also supports detailed circuit-level analysis for critical path timing, area, and power, and it can model device types based on the ITRS roadmap. It uses a flexible XML interface to integrate with various performance simulators, enabling a wide range of design space explorations. SPICE models [210,211] are commonly applied for estimating Soft Error Rates (SER), enabling analysis of fault-tolerance under voltage scaling and transient error conditions. These datasets are especially relevant for safety-critical domains such as avionics or automotive control, where robustness and fault recovery time are as important as energy efficiency.

Beyond raw measurements of temperature or energy, these datasets support evaluation of domain-specific metrics such as *deadline satisfaction rate* (e.g., tasks meeting deadlines despite thermal throttling) and *fault recovery time* (e.g., recovery from soft errors induced under scaled voltages). Together, IPCM, Gem5, HotSpot, McPAT, and SPICE provide essential support for analyzing how learning-based scheduling algorithms balance performance, energy efficiency, thermal stability,

and system reliability in both general-purpose and domain-specific environments.

Table 11 provides an overview of commonly used benchmarks, detailing their domains, dataset sizes, input characteristics, and whether they are computation or memory intensive. The choice of datasets has a decisive impact on the conclusions drawn from scheduling evaluations. Scientific workflow benchmarks such as Montage, CyberShake, or Epigenomics are well suited for studying data-intensive and compute-heavy domains, but they capture few of the timing and robustness constraints that arise in embedded or autonomous systems. By contrast, embedded-oriented simulators like DS3 offer latency-sensitive and communication-driven workloads, yet their abstracted nature limits direct transfer to cloud or HPC environments. Dataset size and variability are equally influential: small, noise-free DAGs are valuable for validating algorithmic correctness, while larger task graphs with injected noise provide deeper insights into scalability, resilience, and fault recovery under realistic perturbations. Therefore, meaningful comparison of learning-based schedulers requires careful alignment between dataset characteristics and the intended application domain; otherwise, performance results risk being misleading or lacking generalizability.

10.2. Performance metrics used for evaluation

To measure the efficiency of task scheduling algorithms, a collection of performance metrics is utilized. These metrics offer quantitative estimates of the degree to which an algorithm achieves particular goals, for example, minimizing makespan, maximizing resource utilization, or providing fairness. Now, let us discuss some of the widely used evaluation metrics.

- *Makespan (Schedule Length)* [47] represents the total completion time of all tasks in a workload, reflecting the overall efficiency of a scheduler in completing an entire DAG or task set. Lower value of this metric implies faster task accomplishment and improved system performance.
- *Schedule Length Ratio (SLR)* [47] is the ratio of the makespan produced by the scheduling algorithm to the length of the critical path (i.e., the minimum possible execution time on the fastest processor). It evaluates how close the achieved schedule is to the theoretical lower bound.
- *Speedup* [47] defined as the ratio of the execution time of the application on a single fastest processor to the execution time achieved by the scheduling algorithm on a multiprocessor system. It reflects the relative performance improvement gained from parallel execution.
- *Deadline miss rate* [161,168] measures the percentage of tasks failing to complete within their assigned deadlines. Similarly, *Schedulability Ratio* [224] is the ratio between the number of tasks successfully completed within their deadlines and the total number of tasks submitted for execution on the platform. This is commonly used as an indicator for the evaluation of schedulability analysis.
- *Resource Usage (Efficiency)* [47,225] examines how computing resources such as CPU, memory, and bandwidth are consumed. Higher utilization indicates fewer wasted resources and reduced cost.
- *Throughput* [173] is defined as the number of tasks completed per unit time, provides a broad indicator of scalability and system efficiency.
- *Waiting time* [136] is the time a task remains in the queue prior to execution, and *turnaround time*, i.e., the total duration from task submission to completion, are also key performance indicators.
- *Load balancing* [110,226,227] is essential for fair distribution of work among available resources to avoid bottlenecks and enhance system stability.

Table 11

Overview of representative datasets and benchmarks used in prior studies, including domain, size, input characteristics, and workload type. The columns CI and MI denote Computation-Intensive and Memory-Intensive benchmarks, respectively. A ✓ indicates the presence of the property.

Benchmark	Domain	Size	Input samples/Metrics	Workload type	
				CI	MI
MNIST [212]	Computer vision	21 MB	60k training, 10k test images	✓	×
KITTI [213]	Computer vision	175 GB	7.5k training/test images	✓	×
PARSEC [198]	Parallel computing	8–12 GB	Multithreaded workloads	✓	×
NAS Parallel [197]	HPC	Few MB–TB	Parallel kernels (LU, CG)	✓	✓
LINPACK [200]	HPC	Varies	Linear algebra problems	✓	×
CoreMark [214]	Embedded systems	Few MB	CPU metrics (IPS)	✓	×
CoreMark Pro [215]	Embedded systems	Few MB	CPU metrics	✓	✓
SPEC CPU2006 [216]	Embedded systems	8 GB	CPU metrics	✓	×
MiBench [183]	Embedded systems	Varies	Embedded workloads	✓	×
Dhrystone [217]	Embedded systems	Few KB	Integer metrics	✓	×
Whetstone [218]	Embedded systems	Few KB	Floating-point metrics	✓	×
TGFF [184]	Task scheduling	Synthetic	Task graphs	✓	✓
E3S [219]	Embedded DAGs	50–200 MB	Automotive/avionics tasks	✓	×
SDF ³ [220]	Streaming DAGs	Varies	Signal processing graphs	×	✓
WSN-DS [221]	Network security	50 MB	374k traffic samples, 19 features	✓	✓
NSL-KDD [222]	Network security	150 MB	126k samples, 21 intrusion attacks, 41 features	✓	×
DREBIN [223]	Mobile security	~5.5 GB	129k Android apps	×	✓
COSMIC [201]	Architecture	Varies	Cache metrics	✓	✓
CyberShake [186]	Scientific workflows	Varies	Earthquake workflows	✓	✓
Epigenomics [185]	Scientific workflows	>100 GB	Genomic workflows	✓	✓
Montage [187]	Scientific workflows	Varies	Astronomical workflows	×	✓
FFT (radix-2)	Scientific workflows	Varies	DSP/OFDM, spectral analysis; highly parallel, batched transforms	✓	×
Alibaba Cluster Trace [191]	Cloud workloads	2–8 TB	Batch and service workloads from 1.3k–4k machines; job DAGs, task/machine usage logs	✓	✓
Google Cluster Trace [192]	Cloud workloads	~41 GB (compressed)	29-day trace from ~12k machines; job events, task events, and machine usage statistics	✓	✓
GoCJ Dataset [193]	Cloud workloads	Text files (KB–MB)	Synthetic job-size dataset (21 files) generated via Monte Carlo simulation using Google cluster trace statistics	✓	✓
TPC-H [195]	Database/Decision support	GB–TB	22 SQL query templates for on-line analytic processing (OLAP); query plans often modeled as DAGs	✓	✓
CloudSim [189]	Cloud simulation	Synthetic	Simulation of VM allocation, job scheduling, energy-aware cloud environments	✓	✓

- *Energy consumption* [103,163] remains a crucial metric, particularly in embedded and mobile systems where power is limited. In cloud computing environments, *cost* [91,228], representing the monetary expense of executing workloads, is a paramount consideration.
- *Scalability* [173] is the ability of the system to accommodate more tasks or users without degradation, is measured by observing performance under increasing loads.
- *Communication overhead* [48] is the additional time and effort required for data transfers between tasks and resources, reflects efficiency in distributed and heterogeneous platforms.
- *Fault recovery time* [22] is the system's ability to recover from failures [73] (e.g., through task re-execution, replication, or migration). This is typically evaluated in safety-critical domains such as autonomous driving [83,155], industrial control, and healthcare workflows, where reliability directly affects system dependability.
- *Running time* [57] determines how quickly the scheduling algorithm can find a solution. This is especially important for real-time and dynamic systems where a fast response is critical.
- *Convergence time* [229,230] is the duration it takes for a learning algorithm to reach a stable and acceptable level of performance. It signifies the point at which the model has “learned” and further training does not yield significant improvement. The definition of “stable performance” varies depending on the type of machine learning approach.

- With respect to ML approaches, convergence is typically observed by plotting the loss function over the number of training epochs. When the loss curve flattens out and the validation loss no longer significantly decreases, the model has converged.
- In RL, the focus is on how the agent's performance improves with experience. Convergence can be seen when Q-values (in Q-learning) stop changing much from one update to the next, or when the cumulative reward steadily rises and eventually plateaus over many episodes. This tells us the agent has found a reasonably stable policy.
- To deal with this instability, researchers often rely on techniques such as experience replay (to smooth out the learning signal) [119,180] and target networks (to prevent feedback loops) [119].
- *Prediction Accuracy* [133,135] is widely adopted when performance prediction or scheduling is formulated as a classification problem. It represents the proportion of correctly predicted mappings or scheduling outcomes relative to the total. In multi-core task scheduling, this metric is useful for evaluating how reliably a model assigns tasks to appropriate performance classes such as high-performance or energy-efficient configurations.
- *Precision* [117] measures the fraction of true positive predictions among all positive predictions. It is particularly relevant in multi-core scheduling contexts where false positives (e.g., misclassifying a poor mapping as efficient or deadline-feasible) may lead to

severe consequences such as deadline misses, bottlenecks, or unnecessary energy consumption. Thus, precision highlights the reliability of positive scheduling decisions.

- **Percentage of Successful Tendency Prediction (PSTP)** is a trend-oriented metric introduced in [75] that evaluates whether the relative ordering between two mappings is preserved between actual and predicted performance values. This is especially important in multi-core scheduling, where the scheduler's primary objective is often to select the better configuration among alternatives rather than to achieve exact numerical predictions. PSTP therefore complements error-based metrics by emphasizing decision quality in task-to-core mappings and resource allocation.

To measure the above performance metrics, the following input parameters will be varied:

- **Number of tasks and Utilization** [5,47]: The number of tasks is a fundamental parameter that represents the total number of computational jobs or processes ready for execution. It directly affects system utilization which is calculated as the ratio of total computational demand to available processing capacity. An increase in the number of tasks increases utilization, assuming task execution demands remain constant.
- **Number of Processing Elements (PEs)** [5,47]: It refers to the total computational units (e.g., cores in a multi-core processor or distinct machines in a distributed computing environment) available for execution. A higher number of PEs generally allows more tasks to run simultaneously, which can reduce the makespan. However, simply adding more PEs does not always guarantee a proportional decrease in makespan due to factors like communication overhead and task dependencies.
- **Number of Communication Resources** [5,47]: It refers to the total number of channels, links, or pathways available for data exchange between processing elements in a distributed system. This is a critical factor in task scheduling, as it directly impacts the communication overhead. A higher number of communication resources can lead to: (i) reduced latency, (ii) increased throughput, and (iii) lower contention.
- **Communication-to-Computation Ratio (CCR)** [5,47]: Used to measure the balance between a task's communication overhead and its computational workload. It is calculated as the ratio of the average time spent on communication to the average time spent on computation. A *High CCR* (>1) indicates communication-intensive workloads, best suited for a single, powerful machine to avoid the overhead of a slow network. A *Low CCR* (<1) denotes computation-intensive workloads, which can be efficiently distributed across a large cluster.
- **Heterogeneity factor** [47]: Used to quantify the variability in execution time of a single task when it is run on different processing elements within a heterogeneous system. In simpler terms, it measures how "skewed" the performance of different processors is for a given task. High heterogeneity factor implies that the execution time for a task varies significantly across the different processors. Low heterogeneity factor means the task's execution time is relatively uniform across all processors.
- **Fault Arrival Rate** [22]: The average number of faults that occur per unit of time. A higher fault arrival rate means that faults are more frequent, and an algorithm's ability to tolerate them is put to a more rigorous test. The common way to model it is using statistical distributions such as *exponential*, *Poisson*, or *Weibull distribution*. By varying the fault arrival rate in their experiments, researchers can determine the algorithm's robustness and scalability in different, realistic scenarios, from relatively stable systems to highly unreliable ones.

Through this broad set of performance measures and input parameters, researchers can comprehensively assess the efficiency, reliability, scalability, and domain-specific adaptability of scheduling algorithms.

Table 12

Average response time (ms) during training phase [226].

Iteration	NSGA-II	NSGA-III	Q-Learning	DQN	DRLIS
10	15 100	16 400	14 400	15 100	13 800
20	14 800	15 000	13 000	14 300	12 800
30	14 850	14 850	12 700	13 700	11 800
40	14 860	14 860	12 400	13 200	11 400
50	14 860	14 860	11 600	12 800	11 250
60	14 860	14 860	11 200	12 700	11 250
70	14 860	14 860	11 200	11 800	11 200
80	14 860	14 900	11 200	11 300	11 280
90	14 860	15 000	11 200	11 200	11 200
100	14 870	15 200	11 250	11 250	11 200

Table 13

Average response time (ms) during evaluation phase [226].

Iteration	NSGA-II	NSGA-III	Q-Learning	DQN	DRLIS
10	860	890	840	860	860
20	850	850	820	850	815
30	850	845	820	840	785
40	855	840	805	840	775
50	860	835	785	840	770
60	865	840	770	820	770
70	860	840	770	790	765
80	860	840	780	780	765
90	855	845	780	770	770
100	855	850	785	770	770

Table 14

Average scheduling overhead with 95% confidence interval [226].

Algorithm	Overhead (ms)
NSGA-III	9.8
NSGA-II	9.4
Q-Learning	1.8
DQN	6.0
DRLIS	3.8

10.3. Comparative empirical analysis: A case-study

In this section, we discuss the comparative performance analysis of learning-based scheduling approaches over traditional methodologies using the latest state-of-the-art.

To illustrate this comparison, we examine the performance evaluation study conducted by Wang et al. [226] to highlight how the RL and DRL techniques perform relative to well-known meta-heuristic baselines. In this study, the authors have evaluated meta-heuristic methods such as NSGA-II and NSGA-III over Q-Learning, Deep Q-Network (DQN), and Deep Reinforcement Learning-based IoT Scheduler (DRLIS) on the *FogBus2* edge-fog-cloud platform. Four heterogeneous IoT applications (image recognition, face detection, OCR, and traffic monitoring) were used to analyze convergence behavior, response time, and scheduling overhead under realistic conditions.

Analysis of response time during training and evaluation phases: During the training phase, DRLIS consistently achieved lower latency and faster convergence than both meta-heuristic and RL schedulers. This performance advantage persisted in the evaluation phase, maintaining response times below 800 ms for most iterations and demonstrating stable inference efficiency. The average response times for different algorithms during the training and evaluation phase are summarized in Tables 12 and 13 respectively.

Analysis of scheduling overhead: Table 14 summarizes the average scheduling overhead measured in the same setup. DRLIS incurred a moderate overhead of 3.8 ms slightly higher than Q-Learning (1.8 ms) but considerably lower than DQN and meta-heuristic counterparts (6–10 ms). This marginal computational cost is compensated by significantly faster convergence and reduced response latency.

Inference: Across all phases, DRLIS achieved faster convergence and lower response times than both meta-heuristic and RL schedulers.

Although its scheduling overhead is slightly higher than Q-Learning, the overall trade-off between inference cost and performance efficiency strongly favors DRL-based methods for heterogeneous fog/edge systems.

11. Challenges and future directions

Machine learning has emerged as a promising paradigm for optimizing makespan, energy, thermal behavior, and reliability in multi-core platforms. Despite rapid progress, several fundamental challenges must be addressed before these techniques can be fully integrated into practical and certifiable real-time systems. A significant contribution of this survey is the in-depth analysis of both *well-established limitations* and *newly uncovered challenges* that have emerged from our taxonomy and comparative synthesis of 83 learning-based scheduling studies.

To improve clarity, this section explicitly distinguishes between widely recognized challenges in the literature and novel challenges newly identified by this survey that were not distinctly addressed in prior reviews.

11.1. Well-known challenges

- *Scalability with complex systems:* As the number of tasks, cores, and constraints increases, the feature space (for ML models) or the state-action space (for RL models) expands exponentially. This growth makes both training and inference computationally demanding, particularly in heterogeneous architectures where task dependencies and varying core capabilities add complexity. Reinforcement learning agents face convergence issues in high-dimensional spaces, while supervised and unsupervised methods require large volumes of representative data. Distributed or hierarchical learning partially mitigates scalability problems but introduces synchronization overhead. Achieving scalable models that yield real-time decisions without excessive training cost remains an open challenge.
- *High training and inference overheads:* Complex ML and RL models often require significant computational resources during both training and runtime, which can be prohibitive in embedded or resource-constrained platforms. Inference may compete for CPU, GPU, or memory bandwidth with other real-time tasks, leading to unpredictable latency and energy consumption. Although off-line training on powerful servers is possible, transferring large models to lightweight devices without performance degradation is unresolved. Designing lightweight architectures and specialized accelerators remains a critical research need.
- *Generalization and runtime adaptability:* Learning-based schedulers frequently struggle to maintain robustness across both platform changes and runtime variability. Models trained for a specific hardware or workload often fail to generalize to new conditions, and even well-generalized models can degrade when faced with unpredictable runtime dynamics such as workload bursts, temperature shifts, or hardware aging. While transfer learning, domain adaptation, and continual learning techniques offer partial remedies, achieving both cross-platform portability and real-time adaptability within a single lightweight framework remains a significant open challenge.
- *Lack of transparency and interpretability:* Most ML-based scheduling models operate as opaque “black boxes”, providing limited insight into why specific task-core mappings are chosen. This lack of explainability hinders certification and adoption in safety-critical systems such as avionics or automotive control. Without interpretability, debugging or validating ML-based schedulers becomes difficult, undermining confidence and compliance with emerging explainability regulations.

- *Need for robust multi-objective optimization:* Real-world scheduling typically requires simultaneous optimization of multiple objectives such as makespan, energy, temperature, reliability, and cost. Yet, most ML approaches still optimize one or two metrics independently. Reinforcement learning often struggles with reward shaping in non-linear objective spaces, leading to biased or unstable trade-offs. Adaptive Pareto-based optimization remains an active but challenging area.
- *Lack of standardized benchmarks and data availability:* Existing benchmark suites (e.g., SPLASH-2, PARSEC) were not designed with ML scheduling in mind, limiting their representativeness. Consequently, published works use diverse workloads and evaluation metrics, hindering reproducibility. Collecting real-world data from industrial or safety-critical environments is costly and subject to privacy restrictions. *(This point is partly reinforced by our survey: only a small fraction of papers reuse common datasets or benchmarks.)*
- *Opportunity for hybrid and hardware-aware solutions:* Purely heuristic or purely learning-based methods exhibit opposite weaknesses: heuristics are predictable but inflexible, while ML models adapt but may violate real-time constraints or require massive data. Future schedulers must combine heuristic predictability with ML adaptability, while being explicitly hardware-aware (cache, DVFS states, accelerators) to exploit heterogeneous resources effectively. This survey highlights integrated multi-decision learning jointly optimizing task mapping, migration, and DVFS as an emerging hybrid direction.

11.2. Newly identified challenges

The following challenges were not distinctly addressed in earlier surveys but emerged from this survey’s multi-dimensional analysis and cross-comparison of recent ML, DL, and RL-based scheduling studies.

- *Communication delays in heterogeneous multiprocessors:* Existing surveys often treat computation time as the primary scheduling bottleneck, overlooking communication-induced latency across heterogeneous cores. Our synthesis reveals that communication delays due to non-uniform cache hierarchies, interconnect bandwidths, and physical distance significantly impact the feedback loops in RL and federated-learning schedulers. These delays slow convergence, destabilize training, and degrade mapping quality. Incorporating inter-core latency as a state or penalty term in learning formulations represents an underexplored but crucial direction.
- *Need for non-intrusive ML approaches:* Learning-based optimization modules running alongside critical real-time tasks must guarantee deterministic timing behavior. Our review highlights that very few works ensure ML operations do not interfere with task execution. Designing “non-intrusive” learning frameworks that maintain predictable overheads, bounded execution time, and background adaptivity is vital for real-world adoption in safety-critical systems.
- *Verifying the correctness of ML-based schedulers:* Traditional formal verification tools are inadequate for validating the probabilistic, data-driven behavior of ML models. No previous survey explicitly recognized correctness verification as an independent challenge. This survey identifies the urgent need for scalable co-verification frameworks that combine theoretical analysis, simulation, and runtime monitoring to validate ML scheduling decisions.
- *Evolving standards and lack of certification precedent:* Regulatory frameworks such as FAA DO-178C and ISO 26262 were designed for deterministic software, not adaptive ML-based systems. Our analysis finds that existing standards lack provisions for continuous learning, model retraining, or uncertainty quantification creating significant certification and lifecycle challenges. Future efforts must establish formal standards and testing pipelines tailored for learning enabled scheduling.

- *System-level ML for reliability and fault-tolerance:* Existing ML-based fault-tolerance studies focus on localized fault detection. Through cross-study comparison, this survey uncovers the absence of system-level reliability modeling using ML that spans processor, interconnect, and thermal effects. Integrating fault prediction, recovery, and scheduling in a unified ML framework remains a critical open frontier.
- *Instability and convergence failures in RL models:* Reinforcement learning-based schedulers often exhibit oscillatory or unstable convergence due to sparse rewards, large state-action spaces, or shifting workloads. While instability has been observed in prior works, this survey highlights the role of heterogeneous hardware and multi-objective conflicts as key hidden causes requiring targeted algorithmic remedies.
- *High experimentation cost and hyperparameter sensitivity:* DRL schedulers often depend on extensive trial and error hyperparameter tuning (learning rates, exploration factors, reward weights). This survey consolidates evidence that such sensitivity impedes reproducibility. Automated hyperparameter optimization and surrogate modeling are promising but underutilized directions.

11.3. Remedies and emerging directions

To address the aforementioned challenges, recent research has begun to explore a range of remedies that collectively enhance scalability, transparency, adaptability, and reliability of learning-based scheduling frameworks. One of the most promising approaches is the adoption of *curriculum learning*, in which models are gradually trained on workloads of increasing complexity. This progressive training helps reduce the explosion of state-action spaces that often hinders reinforcement learning in large-scale multicore environments. Complementary to this, *hierarchical and federated reinforcement learning* techniques decompose large scheduling problems into smaller, more tractable subproblems, thereby mitigating scalability bottlenecks and enabling distributed model updates across computing nodes.

To reduce runtime and energy overhead, studies increasingly employ *model compression*, *quantization*, and *knowledge distillation*. These methods make deep models lightweight enough for deployment on embedded devices without significant loss of accuracy. Likewise, the use of *transfer learning*, *domain adaptation*, and *meta-learning* has emerged as an effective strategy for improving generalizability and reducing the need for extensive retraining across heterogeneous platforms or workloads. Another important direction focuses on enhancing interpretability and trustworthiness through *Explainable AI (XAI)* techniques. Attention-based visualization, saliency mapping, and symbolic surrogate models are increasingly integrated to provide insights into task-to-core allocation decisions, which is crucial for certification in safety- and mission-critical systems. Meanwhile, *continual learning* enables adaptive updates during runtime, allowing schedulers to evolve with workload changes while *safety wrappers* ensure fallback stability when learned policies behave unpredictably. Hybridization between heuristic and learning-based scheduling methods is gaining traction as a means of combining analytical predictability with adaptive flexibility. Hardware-awareness is also becoming essential; models that explicitly incorporate hardware characteristics such as DVFS states, cache hierarchies, and accelerator availability can make more informed scheduling decisions that reflect real platform constraints. Finally, researchers are beginning to integrate *formal verification techniques* with runtime monitoring to ensure correctness and safety, while emphasizing the importance of open-source benchmarks, synthetic workload generators, and standardized evaluation metrics to promote reproducibility and fair comparison across studies.

Collectively, these remedies mark a transition from isolated algorithmic innovations toward a cohesive ecosystem of scalable, interpretable, and certifiable ML-based scheduling solutions for next-generation heterogeneous multi-core systems. In summary, this survey

contributes beyond prior works by exposing previously overlooked dimensions such as communication latency, non-intrusiveness, correctness verification, certification gaps, and cross-layer learning integration. These newly identified challenges emphasize that transitioning ML-based scheduling from theoretical promise to deployable reality requires addressing not only algorithmic performance but also verifiability, transparency, and compliance. Future research must therefore move toward unified, explainable, and certifiable learning frameworks for next-generation heterogeneous multi-core platforms.

12. Conclusion

Learning-based scheduling methods for multiprocessor systems demonstrate distinct strengths and limitations. Traditional ML approaches are lightweight, interpretable, and relatively easy to deploy, but they often rely on offline training and struggle with adaptability under dynamic workloads. DL extends these capabilities by automatically extracting complex features and capturing task dependencies, though at the expense of higher computational and inference costs. RL introduces adaptability through sequential decision-making and online learning but faces challenges in reward design, convergence, and scalability. DRL, which combines the representational power of DL with the adaptability of RL, delivers strong performance across objectives such as makespan, energy, and reliability. However, its practical deployment is hindered by model complexity, inference overhead, and limited generalization to unseen workloads and architectures.

The choice of scheduling method depends on application demands, resource constraints, and performance objectives. For resource-constrained or real-time environments, ML may be preferable due to its low overhead. DL becomes advantageous in scenarios requiring accurate modeling of task dependencies or non-linear workload patterns. RL methods are better suited for adaptive scheduling in dynamic or uncertain environments, while DRL is most effective for large-scale heterogeneous platforms that require multi-objective adaptability. This diversity highlights that no single approach is universally optimal, and schedulers must be carefully matched to system context and performance goals.

This survey provides a unified taxonomy that classifies 83 studies based on learning paradigm, task model, platform heterogeneity, and optimization objective. It offers comparative insights across ML, DL, RL, and DRL methods, clarifying their respective strengths, weaknesses, and application domains. This analysis also reveals a clear methodological shift from predictive ML/DL approaches, which focus on modeling and estimation, toward adaptive, policy-driven RL/DRL frameworks that enable online learning and dynamic decision-making in complex multi-core environments. Structured tabular summaries, comparative framework, and standardized evaluation guidelines together form a coherent reference for future work. Looking ahead, promising research directions include developing lightweight DRL frameworks that minimize training and inference overhead, exploring cross-platform and federated learning to enhance generalization across heterogeneous hardware, and advancing explainable ML/RL based scheduling to improve transparency and support industrial adoption. These targeted directions provide a practical roadmap for building next-generation schedulers that are efficient, interpretable, and transferable across diverse computing environments. Reliability and thermal aware scheduling remain underexplored yet essential for sustainable and fault-tolerant operation. This survey also underscores the importance of experimental standardization, as the lack of unified benchmarks, consistent metric definitions, and transparent reporting practices continues to limit reproducibility and cross-paper comparability. Systematic evaluation and benchmarking of ML, RL, DL, and DRL approaches against heuristic and meta-heuristic methods can help quantify real gains in adaptability, scalability, and energy efficiency, while revealing trade-offs in convergence, interpretability, and implementation complexity. Such unified assessments will bridge the methodological gap between algorithmic

and data-driven paradigms, guiding the design of hybrid frameworks that combine the strengths of both.

In summary, this survey synthesizes the state-of-the-art across ML, DL, RL, and DRL paradigms and establishes a structured foundation for understanding their interplay with task and platform characteristics. By clarifying trade-offs, trends, and actionable research directions, it serves as a comprehensive reference for future exploration and innovation in learning-based scheduling for heterogeneous multicore systems.

CRedit authorship contribution statement

Shaima Rahim: Writing – original draft, Resources, Methodology, Investigation, Formal analysis, Data curation. **Piyooosh Purushothaman Nair:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Investigation, Conceptualization. **Rajesh Devaraj:** Writing – review & editing, Supervision, Project administration, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- [1] B. Hu, S. Xu, Z. Cao, M. Zhou, Safety-guaranteed and development cost-minimized scheduling of dag functionality in an automotive system, *IEEE Trans. Intell. Transp. Syst.* 23 (4) (2022) 3074–3086, <http://dx.doi.org/10.1109/TITS.2020.3030607>.
- [2] A. Zhang, Y. Chen, L. Chen, G. Chen, On the NP-hardness of scheduling with time restrictions, 2017, [arXiv:1703.00575](https://arxiv.org/abs/1703.00575) URL: <https://arxiv.org/abs/1703.00575>.
- [3] D.G.R. O.S. Abdul Qadir, A novel heuristic based task scheduling algorithm to minimize the makespan in cloud environment, *Int. J. Adv. Sci. Technol.* 29 (05) (2020) 3737–3746, URL: <https://sersc.org/journals/index.php/IJAST/article/view/12276>.
- [4] S. Ahamad, Ratneshwer, Some studies on performability analysis of safety critical systems, *Comput. Sci. Rev.* 39 (2021) 100319, <http://dx.doi.org/10.1016/j.cosrev.2020.100319>, URL: <https://www.sciencedirect.com/science/article/pii/S1574013720304196>.
- [5] S.K. Roy, R. Devaraj, A. Sarkar, K. Maji, S. Sinha, Contention aware optimal scheduling of real-time precedence-constrained task graphs on heterogeneous distributed systems, *J. Syst. Archit.* 105 (2020) 101706, <http://dx.doi.org/10.1016/j.sysarc.2019.101706>, URL: <https://www.sciencedirect.com/science/article/pii/S1383762119305132>.
- [6] P.P. Nair, A. Sarkar, N. Harsha, M. Gandhi, P. Chakrabarti, S. Ghose, ERfair scheduler with processor suspension for real-time multiprocessor embedded systems, *ACM Trans. Des. Autom. Electron. Syst. (TODAES)* 22 (1) (2016) 1–25.
- [7] P.P. Nair, A. Sarkar, S. Biswas, Fault-tolerant real-time fair scheduling on multiprocessor systems with cold-standby, *IEEE Trans. Dependable Secur. Comput.* 18 (4) (2019) 1718–1732.
- [8] J. Liu, J. Guo, Energy efficient scheduling of real-time tasks on multicore processors with voltage islands, *Future Gener. Comput. Syst.* 56 (2016) 202–210, <http://dx.doi.org/10.1016/j.future.2015.06.003>, URL: <https://www.sciencedirect.com/science/article/pii/S0167739X15002046>.
- [9] A.Z. Jooya, A. Baniyadi, M. Analoui, History-aware, resource-based dynamic scheduling for heterogeneous multi-core processors, *IET Comput. Digit. Tech.* 5 (4) (2009) <http://dx.doi.org/10.1049/iet-cdt.2009.0045>.
- [10] S. Fujita, A branch-and-bound algorithm for solving the multiprocessor scheduling problem with improved lower bounding techniques, *IEEE Trans. Comput.* 60 (7) (2011) 1006–1016, <http://dx.doi.org/10.1109/TC.2010.120>.
- [11] T. Hikida, H. Nishikawa, H. Tomiyama, Heuristic algorithms for dynamic scheduling of moldable tasks in multicore embedded systems, *Int. J. Reconfigurable Embed. Syst. (IJRES)* 10 (2021) 157, <http://dx.doi.org/10.11591/ijres.v10i3.pp157-167>.
- [12] A. Pathania, V. Venkatramani, M. Shafique, T. Mitra, J. Henkel, Optimal greedy algorithm for many-core scheduling, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 36 (6) (2017) 1054–1058, <http://dx.doi.org/10.1109/TCAD.2016.2618880>.
- [13] S. Kim, H. Lee, Efficient multi-task scheduling with nearest neighbor algorithm for multi-robot systems in antarctic environments, *J. Inst. Control. Robot. Syst.* 29 (4) (2023) 325–332, <http://dx.doi.org/10.5302/J.ICROS.2023.22.8012>.
- [14] S.K. Mishra, R. Manjula, A meta-heuristic based multi objective optimization for load distribution in cloud data center under varying workloads, *Clust. Comput.* 23 (4) (2020) 3079–3093.
- [15] J.K. Konjaang, L. Xu, Meta-heuristic approaches for effective scheduling in infrastructure as a service cloud: A systematic review, *J. Netw. Syst. Manage.* 29 (2) (2021) 15.
- [16] J. Zhang, L. Cheng, C. Liu, Z. Zhao, Y. Mao, Cost-aware scheduling systems for real-time workflows in cloud: An approach based on genetic algorithm and deep reinforcement learning, *Expert Syst. Appl.* 234 (2023) 120972, <http://dx.doi.org/10.1016/j.eswa.2023.120972>, URL: <https://www.sciencedirect.com/science/article/pii/S0957417423014744>.
- [17] S. Kirkpatrick, C.D. Gelatt, M.P. Vecchi, Optimization by simulated annealing, *Science* 220 (4598) (1983) 671–680, <http://dx.doi.org/10.1126/science.220.4598.671>, URL: <https://www.science.org/doi/abs/10.1126/science.220.4598.671>.
- [18] F. Wang, H. Zhang, K. Li, Z. Lin, J. Yang, X.-L. Shen, A hybrid particle swarm optimization algorithm using adaptive learning strategy, *Inform. Sci.* 436 (2018) 162–177.
- [19] M. Dorigo, V. Maniezzo, A. Colomi, Ant system: optimization by a colony of cooperating agents, *IEEE Trans. Syst. Man Cybern. B* 26 (1) (1996) 29–41, <http://dx.doi.org/10.1109/3477.484436>.
- [20] A. Choudhury, M. Ghose, A. Islam, et al., Machine learning-based computation offloading in multi-access edge computing: A survey, *J. Syst. Archit.* 148 (2024) 103090.
- [21] R.I. Davis, A. Burns, A survey of hard real-time scheduling for multiprocessor systems, *ACM Comput. Surv.* 43 (4) (2011) <http://dx.doi.org/10.1145/1978802.1978814>.
- [22] C.M. Krishna, Fault-tolerant scheduling in homogeneous real-time systems, *ACM Comput. Surv.* 46 (4) (2014) <http://dx.doi.org/10.1145/2534028>.
- [23] S. Zhuravlev, J.C. Saez, S. Blagodurov, A. Fedorova, M. Prieto, Survey of energy-cognizant scheduling techniques, *IEEE Trans. Parallel Distrib. Syst.* 24 (7) (2013) 1447–1464, <http://dx.doi.org/10.1109/TPDS.2012.20>.
- [24] M. Bambagini, M. Marinoni, H. Aydin, G. Buttazzo, Energy-aware scheduling for real-time systems: A survey, *ACM Trans. Embed. Comput. Syst.* 15 (1) (2016) <http://dx.doi.org/10.1145/2808231>.
- [25] S.Z. Sheikh, M.A. Pasha, Energy-efficient multicore scheduling for hard real-time systems: A survey, *ACM Trans. Embed. Comput. Syst.* 17 (6) (2018) <http://dx.doi.org/10.1145/3291387>.
- [26] S. Pagani, P.S. Manoj, A. Jantsch, J. Henkel, Machine learning for power, energy, and thermal management on multicore processors: A survey, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 39 (1) (2018) 101–116.
- [27] Y.-W. Zhang, R.-K. Chen, A survey of energy-aware scheduling in mixed-criticality systems, *J. Syst. Archit.* 127 (2022) 102524, <http://dx.doi.org/10.1016/j.sysarc.2022.102524>, URL: <https://www.sciencedirect.com/science/article/pii/S1383762122000881>.
- [28] G. Xie, X. Xiao, H. Peng, R. Li, K. Li, A survey of low-energy parallel scheduling algorithms, *IEEE Trans. Sustain. Comput.* 7 (1) (2022) 27–46, <http://dx.doi.org/10.1109/TSUSC.2021.3057983>.
- [29] S. Safari, M. Ansari, H. Khdr, P. Gohari-Nazari, S. Yari-Karin, A. Yeganeh-Khaksar, S. Hessabi, A. Ejlali, J. Henkel, A survey of fault-tolerance techniques for embedded systems from the perspective of power, energy, and thermal issues, *IEEE Access* 10 (2022) 12229–12251, <http://dx.doi.org/10.1109/ACCESS.2022.3144217>.
- [30] F. Reghenzani, Z. Guo, W. Fornaciari, Software fault tolerance in real-time systems: Identifying the future research questions, *ACM Comput. Surv.* 55 (14s) (2023) <http://dx.doi.org/10.1145/3589950>.
- [31] E.H. Houssein, A.G. Gad, Y.M. Wazery, P.N. Suganthan, Task scheduling in cloud computing based on meta-heuristics: review, taxonomy, open challenges, and future trends, *Swarm Evol. Comput.* 62 (2021) 100841.
- [32] X. Yang, N. Rahmani, Task scheduling mechanisms in fog computing: review, trends, and perspectives, *Kybernetes* 50 (1) (2021) 22–38.
- [33] R. Medara, R.S. Singh, A review on energy-aware scheduling techniques for workflows in iaaS clouds, *Wirel. Pers. Commun.* 125 (2) (2022) 1545–1584.
- [34] R. Ghafari, F.H. Kabutarkhani, N. Mansouri, Task scheduling algorithms for energy optimization in cloud environment: a comprehensive review, *Clust. Comput.* 25 (2) (2022) 1035–1093.
- [35] Z.J.K. Abadi, N. Mansouri, M. Khalouie, Task scheduling in fog environment—Challenges, tools & methodologies: A review, *Comput. Sci. Rev.* 48 (2023) 100550.
- [36] Z. Jalali Khalil Abadi, N. Mansouri, A comprehensive survey on scheduling algorithms using fuzzy systems in distributed environments, *Artif. Intell. Rev.* 57 (1) (2024) 4.

- [37] Z. Jalali Khalil Abadi, N. Mansouri, M.M. Javidi, Deep reinforcement learning-based scheduling in distributed systems: a critical review, *Knowl. Inf. Syst.* 66 (10) (2024) 5709–5782.
- [38] H. Hou, S.N.A. Jawaddi, A. Ismail, Energy efficient task scheduling based on deep reinforcement learning in cloud environment: A specialized review, *Future Gener. Comput. Syst.* 151 (2024) 214–231.
- [39] M. Mahdizadeh, A. Montazerolghaem, K. Jamshidi, Task scheduling and load balancing in SDN-based cloud computing: A review of relevant research, *J. Eng. Res.* (2024).
- [40] A. Jayanetti, S. Halgamuge, R. Buyya, Reinforcement learning based workflow scheduling in cloud and edge computing environments: A taxonomy, review and future directions, 2024, arXiv preprint [arXiv:2408.02938](https://arxiv.org/abs/2408.02938).
- [41] E.H. Houssein, A.G. Gad, Y.M. Wazery, P.N. Suganthan, Task scheduling in cloud computing based on meta-heuristics: Review, taxonomy, open challenges, and future trends, *Swarm Evol. Comput.* 62 (2021) 100841, <https://doi.org/10.1016/j.swevo.2021.100841>, URL: <https://www.sciencedirect.com/science/article/pii/S221065022100002X>.
- [42] N. Kandasamy, J.P. Hayes, B.T. Murray, Dependable communication synthesis for distributed embedded systems, *Reliab. Eng. Syst. Saf.* 89 (1) (2005) 81–92, <https://doi.org/10.1016/j.res.2004.08.008>, URL: <https://www.sciencedirect.com/science/article/pii/S0951832004001784> Safety, Reliability and Security of Industrial Computer Systems.
- [43] T. Abdelzaher, E. Atkins, K. Shin, QoS negotiation in real-time systems and its application to automated flight control, in: *Proceedings Third IEEE Real-Time Technology and Applications Symposium*, 1997, pp. 228–238, <https://doi.org/10.1109/RTTAS.1997.601361>.
- [44] F. Oberti, A. Savino, E. Sanchez, P. Casasso, F. Parisi, S. Di Carlo, CAN-MM: multiplexed message authentication code for controller area network message authentication in road vehicles, *IEEE Trans. Veh. Technol.* (2024).
- [45] R.M. Vaz, K.N. Hodel, M.M. Santos, B.A. Arruda, M.L. Netto, J.F. Justo, An efficient formulation for optimization of FlexRay frame scheduling, *Veh. Commun.* 24 (2020) 100234, <https://doi.org/10.1016/j.vehcom.2020.100234>, URL: <https://www.sciencedirect.com/science/article/pii/S221420962030005X>.
- [46] W. Steiner, TTEthernet Specification, Vol. Nov 39, TTEch Computertechnik, 2008, p. 40.
- [47] H. Topcuoglu, S. Hariri, M.-Y. Wu, Performance-effective and low-complexity task scheduling for heterogeneous computing, *IEEE Trans. Parallel Distrib. Syst.* 13 (3) (2002) 260–274, <https://doi.org/10.1109/71.993206>.
- [48] P. Pop, P. Eles, Z. Peng, Schedulability analysis for systems with data and control dependencies, in: *Proceedings 12th Euromicro Conference on Real-Time Systems. Euromicro RTS 2000*, 2000, pp. 201–208, <https://doi.org/10.1109/EMRTS.2000.854008>.
- [49] S.M. Abdulhamid, M.S. Abd Latiff, G. Abdul-Salaam, S.H. Hussain Madni, Secure scientific applications scheduling technique for cloud computing environment using global league championship algorithm, *PLOS ONE* 11 (7) (2016) 1–18, <https://doi.org/10.1371/journal.pone.0158102>.
- [50] Z. Tong, X. Deng, J. Mei, B. Liu, K. Li, Response time and energy consumption co-offloading with SLRTA algorithm in cloud-edge collaborative computing, *Future Gener. Comput. Syst.* 129 (2021) <https://doi.org/10.1016/j.future.2021.11.014>.
- [51] S. Mazzola, T. Benz, B. Forsberg, L. Benini, A data-driven approach to lightweight DVFS-aware counter-based power modeling for heterogeneous platforms, 2023, arXiv preprint [arXiv:2305.06782](https://arxiv.org/abs/2305.06782) URL: <https://arxiv.org/abs/2305.06782>.
- [52] G. Bambini, A. Ottaviano, C. Conficoni, A. Tilli, L. Benini, A. Bartolini, Modeling and controlling many-core HPC processors: An alternative to PID and moving average algorithms, 2024, ArXiv Preprint [arXiv:2405.18030](https://arxiv.org/abs/2405.18030) URL: <https://arxiv.org/abs/2405.18030>.
- [53] Z. Liu, Y. Li, J. Hu, X. Yu, S. Shiau, X. Ai, Z. Zeng, Z. Zhang, DeepOHeat: Operator learning-based ultra-fast thermal simulation in 3D-IC design, 2023, arXiv preprint [arXiv:2302.12949](https://arxiv.org/abs/2302.12949) URL: <https://arxiv.org/abs/2302.12949>.
- [54] W. Huang, S. Ghosh, S. Velusamy, K. Sankaranarayanan, K. Skadron, M. Stan, Real-time temperature prediction for large-scale multi-core chips based on graph convolutional neural networks, *Electronics* 14 (6) (2023) 1223, <https://doi.org/10.3390/electronics14061223>.
- [55] G. Xie, Y. Chen, Y. Liu, R. Li, K. Li, Minimizing development cost with reliability goal for automotive functional safety during design phase, *IEEE Trans. Reliab.* 67 (1) (2017) 196–211.
- [56] S.K. Roy, R. Devaraj, A. Sarkar, K. Maji, S. Sinha, Contention-aware optimal scheduling of real-time precedence-constrained task graphs on heterogeneous distributed systems, *J. Syst. Archit.* 105 (2020) 101706.
- [57] S.K. Roy, R. Devaraj, A. Sarkar, Contention cognizant scheduling of task graphs on shared bus-based heterogeneous platforms, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 41 (2) (2021) 281–293.
- [58] P. Purushothaman Nair, H. Reddi, R. Devaraj, et al., Hybrid-sched: a QoS adaptive offline-online scheduler for real-time tasks on multi-cores, *J. Sched.* (2024) <https://doi.org/10.1007/s10951-024-00831-y>.
- [59] A. Zhadan, A. Allahverdyan, I. Kondratov, V. Mikheev, O. Petrosian, A. Romanovskii, Multi-agent reinforcement learning based adaptive heterogeneous dag scheduling, 2023, <https://doi.org/10.13140/RG.2.2.29025.28008>.
- [60] L.A. Wolsey, *Integer Programming*, Wiley-Interscience, 1998, p. 276.
- [61] G.B. Dantzig, Maximization of a linear function of variables subject to linear inequalities, *Act. Anal. Prod. Alloc.* (1951) 339–347.
- [62] M.A. Iverson, F. Özgüner, G.J. Follen, Parallelizing existing applications in a distributed heterogeneous environment, 1995, URL: <https://api.semanticscholar.org/CorpusID:1948900>.
- [63] G. Sih, E. Lee, A compile-time scheduling heuristic for interconnection-constrained heterogeneous processor architectures, *IEEE Trans. Parallel Distrib. Syst.* 4 (2) (1993) 175–187, <https://doi.org/10.1109/71.207593>.
- [64] H. Djigal, J. Feng, J. Lu, J. Ge, IPPTS: An efficient algorithm for scientific workflow scheduling in heterogeneous computing systems, *IEEE Trans. Parallel Distrib. Syst.* 32 (5) (2020) 1057–1071.
- [65] H. Kanemitsu, M. Hanada, H. Nakazato, Clustering-based task scheduling in a large number of heterogeneous processors, *IEEE Trans. Parallel Distrib. Syst.* 27 (11) (2016) 3144–3157, <https://doi.org/10.1109/TPDS.2016.2526682>.
- [66] B. Jedari, M. Dehghan, Efficient DAG scheduling with resource-aware clustering for heterogeneous systems, in: *Computer and Information Science*, 2009, URL: <https://api.semanticscholar.org/CorpusID:15574087>.
- [67] A. Kaur, P. Gupta, M. Singh, Hybrid balanced task clustering algorithm for scientific workflows in cloud computing, *Scalable Comput.: Prac. Exp.* 20 (2019) 237–258, <https://doi.org/10.12694/scpe.v20i2.1515>.
- [68] W. Ahmad, B. Alam, An efficient list scheduling algorithm with task duplication for scientific big data workflow in heterogeneous computing environments, *Concurr. Comput.: Pract. Exp.* 33 (5) (2021) e5987, <https://doi.org/10.1002/cpe.5987>.
- [69] K. He, X. Meng, Z. Pan, L. Yuan, P. Zhou, A novel task-duplication based clustering algorithm for heterogeneous computing environments, *IEEE Trans. Parallel Distrib. Syst.* 30 (1) (2019) 2–14, <https://doi.org/10.1109/TPDS.2018.2851221>.
- [70] J.H. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control and Artificial Intelligence*, MIT Press, Cambridge, MA, USA, 1992.
- [71] L. Breiman, Random forests, *Mach. Learn.* 45 (1) (2001) 5–32, <https://doi.org/10.1023/A:1010933404324>.
- [72] L. Breiman, J.H. Friedman, R.A. Olshen, C.J. Stone, *Classification and regression trees*, in: *Wadsworth Statistics/Probability, Vol. X*, 1984.
- [73] L.A.P. Da Rosa, L.L. Zanon, L. Carro, Using machine learning techniques to evaluate multicore soft error reliability, *IEEE Trans. Circuits Syst. I. Regul. Pap.* 66 (6) (2019) 2151–2164, <https://doi.org/10.1109/TCSI.2019.2906155>.
- [74] C. Cortes, V. Vapnik, Support-vector networks, *Mach. Learn.* 20 (3) (1995) 273–297, <https://doi.org/10.1007/BF00994018>.
- [75] A. Gamatié, X. An, Y. Zhang, A. Kang, G. Sassatelli, Empirical model-based performance prediction for application mapping on multicore architectures, *J. Syst. Archit.* 98 (2019) 1–16, <https://doi.org/10.1016/j.sysarc.2019.06.001>, URL: <https://www.sciencedirect.com/science/article/pii/S1383762118306465>.
- [76] T. Cover, P. Hart, Nearest neighbor pattern classification, *IEEE Trans. Inform. Theory* 13 (1) (1967) 21–27, <https://doi.org/10.1109/TIT.1967.1053964>.
- [77] Z. Wang, W. Zhan, H. Duan, G. Min, H. Huang, Deep-reinforcement-learning-based continuous workflows scheduling in heterogeneous environments, *IEEE Internet Things J.* 12 (10) (2025) 14036–14050, <https://doi.org/10.1109/JIOT.2024.3524506>.
- [78] S. Ahmad, I. Shakeel, S. Mehruz, J. Ahmad, Deep learning models for cloud, edge, fog, and IoT computing paradigms: Survey, recent advances, and future directions, *Comput. Sci. Rev.* 49 (2023) 100568, <https://doi.org/10.1016/j.cosrev.2023.100568>, URL: <https://www.sciencedirect.com/science/article/pii/S1574013723000357>.
- [79] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, Learning phrase representations using RNN Encoder-Decoder for statistical machine translation, 2014, arXiv preprint [arXiv:1406.1078](https://arxiv.org/abs/1406.1078).
- [80] J. Zhou, G. Cui, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, M. Sun, Graph neural networks: A review of methods and applications, *AI Open* 1 (2020) 57–81, <https://doi.org/10.1016/j.aiopen.2021.01.001>.
- [81] B. Johnston, T. de Boissière, M.A.H. Monil, N.R. Miniskar, A. Young, S. Lee, J.S. Vetter, IRIS-GNN: Leveraging graph neural networks for scheduling on truly heterogeneous runtime systems, in: *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024, pp. 1071–1080, <https://doi.org/10.1109/SCW63240.2024.00147>.
- [82] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS '17*, Curran Associates Inc., 2017, pp. 6000–6010, <https://doi.org/10.48550/arXiv.1706.03762>.
- [83] Y. Youssef, P.R.M. de Araujo, A. Noureldin, S. Givigi, TRATSS: Transformer-based task scheduling system for autonomous vehicles, 2025, arXiv preprint [arXiv:2504.05407](https://arxiv.org/abs/2504.05407).
- [84] H. Mao, M. Schwarzkopf, S.B. Venkatakrishnan, Z. Meng, M. Alizadeh, Learning scheduling algorithms for data processing clusters, 2018, CoRR, [abs/1810.01963](https://arxiv.org/abs/1810.01963) [arXiv:1810.01963](https://arxiv.org/abs/1810.01963) URL: <https://arxiv.org/abs/1810.01963>.

- [85] P. Sun, Z. Guo, J. Wang, J. Li, J. Lan, Y. Hu, DeepWeave: Accelerating job completion time with deep reinforcement learning-based coflow scheduling, in: C. Bessiere (Ed.), *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20, International Joint Conferences on Artificial Intelligence Organization*, 2020, pp. 3314–3320, <http://dx.doi.org/10.24963/ijcai.2020/458>, Main track.
- [86] X. Ni, J. Li, M. Yu, W. Zhou, K. Wu, Generalizable resource allocation in stream processing via deep reinforcement learning, 2019, N/A CoRR [abs/1911.08517](https://arxiv.org/abs/1911.08517) URL: <http://arxiv.org/abs/1911.08517>.
- [87] Ü. Ulusoy, M.S. Güzel, E. Bostanci, A Q-learning-based approach for simple and multi-agent systems, in: R.L. Ruiz (Ed.), *Multi Agent Systems*, IntechOpen, Rijeka, 2020, <http://dx.doi.org/10.5772/intechopen.88484>.
- [88] M. Zhao, L. Mo, J. Liu, J. Han, D. Niu, GAT-based deep reinforcement learning algorithm for real-time task scheduling on multicore platform, in: 2024 36th Chinese Control and Decision Conference, CCDC, 2024, pp. 5674–5679, <http://dx.doi.org/10.1109/CCDC62350.2024.10587381>.
- [89] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, arXiv preprint [arXiv:1707.06347](https://arxiv.org/abs/1707.06347).
- [90] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M.A. Riedmiller, A.K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, D. Hassabis, Human-level control through deep reinforcement learning, *Nature* 518 (2015) 529–533, URL: <https://api.semanticscholar.org/CorpusID:205242740>.
- [91] S.S. Mangalampalli, G.R. Karri, S.N. Mohanty, S. Ali, M. Ijaz Khan, S. Abdullaev, S.A. Alqahtani, Multi-objective prioritized task scheduler using improved asynchronous advantage actor critic (a3c) algorithm in multi cloud environment, *IEEE Access* 12 (2024) 11354–11377, <http://dx.doi.org/10.1109/ACCESS.2024.3355092>.
- [92] B.P. Gan, S.Y. Huang, The modified mapping heuristic algorithm, in: *Proceedings of 1996 IEEE Second International Conference on Algorithms and Architectures for Parallel Processing, ICA/Sup 3/PP '96*, 1996, pp. 456–463, <http://dx.doi.org/10.1109/ICAPP.1996.562909>.
- [93] T. Hagras, J. Janecek, A simple scheduling heuristic for heterogeneous computing environments, in: *Proceedings of the Second International Conference on Parallel and Distributed Computing, ISPD '03*, IEEE Computer Society, USA, 2003, pp. 104–110.
- [94] L.F. Bittencourt, R. Sakellariou, E.R.M. Madeira, DAG scheduling using a lookahead variant of the heterogeneous earliest finish time algorithm, in: 2010 18th Euromicro Conference on Parallel, Distributed and Network-Based Processing, 2010, pp. 27–34, <http://dx.doi.org/10.1109/PDP.2010.56>.
- [95] J.D.C. Little, K.G. Murty, D.W. Sweeney, C. Karel, An algorithm for the traveling salesman problem, *Oper. Res.* 11 (6) (1963) 972–989.
- [96] S. Zhou, Q. Guo, Z. Du, D. Liu, T. Chen, L. Li, S. Liu, J. Zhou, O. Temam, X. Feng, X. Zhou, Y. Chen, ParaML: A polyvalent multicore accelerator for machine learning, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 39 (9) (2020) 1764–1777, <http://dx.doi.org/10.1109/TCAD.2019.2927523>.
- [97] B. Gomatheeshwari, J. Selvakumar, Appropriate allocation of workloads on performance asymmetric multicore architectures via deep learning algorithms, *Microprocess. Microsyst.* 73 (2020) 102996, <http://dx.doi.org/10.1016/j.micpro.2020.102996>.
- [98] P. Neelakantan, N.S. Yadav, Proficient job scheduling in cloud computation using an optimized machine learning strategy, *Int. J. Inf. Technol.* 15 (12) (2023) 2409–2421, <http://dx.doi.org/10.1007/s41870-023-01278-8>.
- [99] S. Allaqband, M. Nazish, S. Allaqband, J. Bashir, M.T. Banday, An efficient machine learning based CPU scheduler for heterogeneous multicore processors, *Int. J. Inf. Technol.* 17 (2024) <http://dx.doi.org/10.1007/s41870-024-01936-5>.
- [100] Z. Wang, W. Zhan, H. Duan, H. Huang, Multiobjective optimization deep reinforcement learning for dependent task scheduling based on spatio-temporal fusion graph neural network, *Eng. Appl. Artif. Intell.* 148 (2025) 110337, <http://dx.doi.org/10.1016/j.engappai.2025.110337>, URL: <https://www.sciencedirect.com/science/article/pii/S0952197625003379>.
- [101] R.S. Sutton, Introduction: The challenge of reinforcement learning, in: *Reinforcement Learning*, Springer US, Boston, MA, 1992, pp. 1–3, http://dx.doi.org/10.1007/978-1-4615-3618-5_1.
- [102] A. Asghari, M.K. Sohrabi, F. Yaghmaee, Online scheduling of dependent tasks of cloud's workflows to enhance resource utilization and reduce the makespan using multiple reinforcement learning-based agents, *Soft Comput.* 24 (21) (2020) 16177–16199.
- [103] Z. Tian, Z. Wang, J. Xu, H. Li, P. Yang, R.K.V. Maeda, Collaborative power management through knowledge sharing among multiple devices, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 38 (7) (2019) 1203–1215, <http://dx.doi.org/10.1109/TCAD.2018.2837131>.
- [104] H. Li, Z. Tian, J. Xu, R.K.V. Maeda, Z. Wang, Z. Wang, Chip-specific power delivery and consumption co-management for process-variation-aware many-core systems using reinforcement learning, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 28 (5) (2020) 1150–1163, <http://dx.doi.org/10.1109/TVLSI.2020.2966866>.
- [105] C. Peng, M. Wang, J. Liu, L. Mo, D. Niu, Energy-efficient real-time DAG task scheduling on multicore platform by deep reinforcement learning, in: 2024 IEEE/CIC International Conference on Communications in China, ICC, 2024, pp. 597–602, <http://dx.doi.org/10.1109/ICCC62479.2024.10681954>.
- [106] M. Khaleel, M.M. Zhu, Energy-efficient task scheduling and consolidation algorithm for workflow jobs in cloud, *Int. J. Comput. Sci. Eng.* 13 (3) (2016) 268–284.
- [107] L. Li, J. Dong, D. Zuo, J. Wu, SLA-aware and energy-efficient VM consolidation in cloud data centers using robust linear regression prediction model, *IEEE Access* 7 (2019) 9490–9500.
- [108] Y. Huang, Deep Q-networks, in: *Deep Reinforcement Learning: Fundamentals, Research and Applications*, Springer Singapore, Singapore, 2020, pp. 135–160, http://dx.doi.org/10.1007/978-981-15-4095-0_4.
- [109] T. Zhou, M. Lin, CPU frequency scheduling of real-time applications on embedded devices with temporal encoding-based deep reinforcement learning, *J. Syst. Archit.* 142 (2023) 102955.
- [110] T. Aslanidis, A. Chouliaras, D. Chatzopoulos, Reinforcement learning techniques for optimizing system configuration on the cloud: A taxonomy and open problems, in: *Proceedings of the 2023 International Conference on Embedded Wireless Systems and Networks, EWSN '23*, Association for Computing Machinery, New York, NY, USA, 2023, pp. 345–350.
- [111] Z. Tong, X. Deng, H. Chen, J. Mei, DDMTS: A novel dynamic load balancing scheduling scheme under sla constraints in cloud computing, *J. Parallel Distrib. Comput.* 149 (2021) 138–148, <http://dx.doi.org/10.1016/j.jpdc.2020.11.007>, URL: <https://www.sciencedirect.com/science/article/pii/S0743731520304147>.
- [112] Y. Garí, D.A. Monge, C. Mateos, A Q-learning approach for the autoscaling of scientific workflows in the cloud, *Future Gener. Comput. Syst.* 127 (2022) 168–180, <http://dx.doi.org/10.1016/j.future.2021.09.007>, URL: <https://www.sciencedirect.com/science/article/pii/S0167739X21003538>.
- [113] H. Topcuoglu, C. Sevilimis, Task scheduling with conflicting objectives, in: *International Conference on Advances in Information Systems*, Springer, 2002, pp. 346–355.
- [114] S. Alshaer, C. Lua, P. Muoka, D. Onwuchekwa, R. Obermaier, Graph neural networks based meta-scheduling in adaptive time-triggered systems, in: 2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation, ETFA, 2022, pp. 1–6, <http://dx.doi.org/10.1109/ETFA52439.2022.9921580>.
- [115] F. Maurer, M. Meidinger, Y. Lu, T. Hallermeier, A. Surhonne, T. Wild, A. Herkersdorf, Qos-aware dynamic frequency scaling for mixed-critical systems based on shielded reinforcement learning, in: 2024 IEEE Nordic Circuits and Systems Conference, NORCAS, IEEE, 2024, <http://dx.doi.org/10.1109/NorCAS64408.2024.10752457>.
- [116] A. Pivezhandi, H. Ma, C. Yan, A. Anwar, H. Ghasemzadeh, Statistical learning for feature-aware task-to-core allocation, 2025, arXiv preprint [arXiv:2502.15716](https://arxiv.org/abs/2502.15716) URL: <https://arxiv.org/abs/2502.15716>.
- [117] N. Ayub, M. Imtiaz, E. Ali, A. Alqahtani, A. Ali, M. Ashurov, S. Albouq, F. Law, A decision framework for intra task fixed priority INTEL PXA270 distributed architecture for soft RT-applications based on deep learning, *Eng. Technol. Appl. Sci. Res.* 15 (2025) 23553–23558, <http://dx.doi.org/10.48084/etasr.10006>.
- [118] Z. Tong, X. Deng, H. Chen, J. Mei, H. Liu, QL-HEFT: a novel machine learning scheduling scheme base on cloud computing environment, *Neural Comput. Appl.* 32 (10) (2020) 5553–5570, <http://dx.doi.org/10.1007/s00521-019-04118-8>.
- [119] J. Pan, Y. Wei, A deep reinforcement learning-based scheduling framework for real-time workflows in the cloud environment, *Expert Syst. Appl.* 255 (2024) 124845, <http://dx.doi.org/10.1016/j.eswa.2024.124845>, URL: <https://www.sciencedirect.com/science/article/pii/S0957417424017123>.
- [120] H. Wu, W. Shen, W. Lin, W. Li, K. Li, End-edge-cloud heterogeneous resources scheduling method based on rnn and particle swarm optimization, *IEEE Trans. Netw. Serv. Manag.* 22 (2) (2025) 1664–1676, <http://dx.doi.org/10.1109/TNSM.2024.3507017>.
- [121] J. An, Q. Wu, C. Dong, Z. Zhang, A dynamic task mapping scheme based on machine learning, in: 2024 Asia-Pacific Conference on Image Processing, Electronics and Computers, IPEC, 2024, pp. 389–397, <http://dx.doi.org/10.1109/IPEC61310.2024.00074>.
- [122] J.L.C. Hoffmann, A.A. Fröhlich, Online machine learning for energy-aware multicore real-time embedded systems, *IEEE Trans. Comput.* 71 (2) (2022) 493–505, <http://dx.doi.org/10.1109/TC.2021.3056070>.
- [123] J.L.C. Hoffmann, J.L. De Souza, A.A. Fröhlich, Lightweight and robust online learning for energy-aware multicore real-time schedulers, in: 2024 XIV Brazilian Symposium on Computing Systems Engineering, SBESC, 2024, pp. 1–6, <http://dx.doi.org/10.1109/SBESC65055.2024.10771912>.
- [124] X. Zhan, J. Chen, E. Sánchez-Sinencio, P. Li, Power management for multicore processors via heterogeneous voltage regulation and machine learning enabled adaptation, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 27 (11) (2019) 2641–2654, <http://dx.doi.org/10.1109/TVLSI.2019.2923911>.
- [125] S.K. Mandal, G. Bhat, C.A. Patil, J.R. Doppa, P.P. Pande, U.Y. Ogras, Dynamic resource management of heterogeneous mobile platforms via imitation learning, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 27 (12) (2019) 2842–2854, <http://dx.doi.org/10.1109/TVLSI.2019.2926106>.
- [126] D. Kim, Y.-B. Ko, S.-H. Lim, Energy-efficient real-time multi-core assignment scheme for asymmetric multi-core mobile devices, *IEEE Access* 8 (2020) 117324–117334, <http://dx.doi.org/10.1109/ACCESS.2020.3005235>.

- [127] J. Zhang, S. Sadiqbatcha, S.X.-D. Tan, Hot-trim: Thermal and reliability management for commercial multicore processors considering workload dependent hot spots, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 42 (7) (2023) 2290–2302, <http://dx.doi.org/10.1109/TCAD.2022.3216552>.
- [128] T.A. Rahmani, G. Belalem, S.A. Mahmoudi, O.R. Merad-Boudia, Machine learning-driven energy-efficient load balancing for real-time heterogeneous systems, *Clust. Comput.* (2024) <http://dx.doi.org/10.1007/s10586-023-04215-3>.
- [129] M. Lavanya, B. Shanthi, S. Saravanan, Multi objective task scheduling algorithm based on SLA and processing time suitable for cloud environment, *Comput. Commun.* 151 (2020) 183–195.
- [130] P. Neelima, A.R.M. Reddy, An efficient load balancing system using adaptive dragonfly algorithm in cloud computing, *Clust. Comput.* 23 (4) (2020) 2891–2899.
- [131] S. Elsherbiny, E. Eldaydamony, M. Alrahmawy, A.E. Reyad, An extended intelligent water drops algorithm for workflow scheduling in cloud computing environment, *Egypt. Inform. J.* 19 (1) (2018) 33–55.
- [132] H. Wu, W. Lin, W. Shen, X. Wang, C.L.P. Chen, K. Li, Prediction of heterogeneous device task runtime based on edge server-oriented deep neuro-fuzzy system, *IEEE Trans. Serv. Comput.* 18 (1) (2025) 372–384, <http://dx.doi.org/10.1109/TSC.2024.3520869>.
- [133] C. Lua, Y. Zhang, O. Hekal, D. Onwuchekwa, R. Obermaier, GNN link prediction for time-triggered systems, in: 2023 International Conference on Artificial Intelligence in Information and Communication, ICAIIC, 2023, pp. 176–181, <http://dx.doi.org/10.1109/ICAIIIC57133.2023.10066960>.
- [134] R. Urnuela, A.-M. Déplanche, Y. Trinet, STORM a simulation tool for real-time multiprocessor scheduling evaluation, in: 2010 IEEE 15th Conference on Emerging Technologies & Factory Automation, ETFA 2010, 2010, pp. 1–8, <http://dx.doi.org/10.1109/ETFA.2010.5641179>.
- [135] G. Balasekaran, S. Jayakumar, R. Pérez de Prado, An intelligent task scheduling mechanism for autonomous vehicles via deep learning, *Energies* 14 (6) (2021) <http://dx.doi.org/10.3390/en14061788>, URL: <https://www.mdpi.com/1996-1073/14/6/1788>.
- [136] W. Shen, W. Lin, W. Wu, H. Wu, K. Li, Reinforcement learning-based task scheduling for heterogeneous computing in end-edge-cloud environment, *Clust. Comput.* (2025) <http://dx.doi.org/10.1007/s10586-024-04828-2>, Online first: Jan 21, 2025.
- [137] F.M.M. ul Islam, M. Lin, L.T. Yang, K.-K.R. Choo, Task aware hybrid DVFS for multi-core real-time systems using machine learning, *Inform. Sci.* 433–434 (2018) 315–332, <http://dx.doi.org/10.1016/j.ins.2017.08.042>, URL: <https://www.sciencedirect.com/science/article/pii/S002002517308897>.
- [138] P. Pillai, K.G. Shin, Real-time dynamic voltage scaling for low-power embedded operating systems, *Proc. the Eighteenth ACM Symp. Oper. Syst. Princ.* 1 (1) (2001) 251–264, URL: <https://api.semanticscholar.org/CorpusID:6783900>.
- [139] H. Aydin, R. Melhem, D. Mosse, P. Mejia-Alvarez, Power-aware scheduling for periodic real-time tasks, *IEEE Trans. Comput.* 53 (5) (2004) 584–600, <http://dx.doi.org/10.1109/TC.2004.1275298>.
- [140] Y. Wang, W. Zhang, M. Hao, Z. Wang, Online power management for multi-cores: A reinforcement learning based approach, *IEEE Trans. Parallel Distrib. Syst.* 33 (4) (2022) 751–764, <http://dx.doi.org/10.1109/TPDS.2021.3092270>.
- [141] M.A. Uddin, M. Lin, L.T. Yang, Dynamic slack-sharing learning technique with DVFS for real-time systems, *IEEE Trans. Sustain. Comput.* 9 (3) (2024) 261–270.
- [142] J. Wang, S. Xiao, X. Guo, D. Jia, W. Wu, Energy-saving scheduling strategy for real-time applications on big.little architectures, *IEEE Internet Things J.* 12 (15) (2025) 29460–29471, <http://dx.doi.org/10.1109/IIOT.2025.3568691>.
- [143] Z. Yang, C. Serafy, T. Lu, A. Srivastava, Enhanced phase-driven Q-learning-based DRM for multicore processors, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 38 (11) (2019) 2022–2031, <http://dx.doi.org/10.1109/TCAD.2018.2877014>.
- [144] F. Hossein-Khani, O. Akbari, A reinforcement learning-based task mapping method to improve the reliability of clustered manycores, *IEEE Access* 13 (2025) 123460–123472, <http://dx.doi.org/10.1109/ACCESS.2025.3585768>.
- [145] S. Mandal, K. Gaurkar, P. Dasgupta, A. Hazra, An RL based approach for thermal-aware energy optimized task scheduling in multi-core processors, in: 2021 34th International Conference on VLSI Design and 2021 20th International Conference on Embedded Systems, VLSID, 2021, pp. 181–186, <http://dx.doi.org/10.1109/VLSID51830.2021.00036>.
- [146] D. Liu, S.-G. Yang, Z. He, M. Zhao, W. Liu, CARTAD: Compiler-assisted reinforcement learning for thermal-aware task scheduling and DVFS on multicores, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 41 (6) (2022) 1813–1826, <http://dx.doi.org/10.1109/TCAD.2021.3095028>.
- [147] R. Siyadatzaheh, M. Ansari, M. Shafique, A. Ejlali, RL-TIME: Reinforcement learning-based task replication in multicore embedded systems, 2025, [arXiv: 2503.12677](https://arxiv.org/abs/2503.12677) URL: <https://arxiv.org/abs/2503.12677>.
- [148] D.A. Velliangiri, D.J. Velusamy, D.M. M. D.R.L. Rose, Task replication based energy management using random-weighted privacy-preserving distributed algorithm for real-time embedded system, *Future Gener. Comput. Syst.* 167 (2025) 107708, <http://dx.doi.org/10.1016/j.future.2025.107708>, URL: <https://www.sciencedirect.com/science/article/pii/S0167739X25000032>.
- [149] A. Yeganeh-Khaksar, M. Ansari, A. Ejlali, ReMap: Reliability management of peak-power-aware real-time embedded systems through task replication, *IEEE Trans. Emerg. Top. Comput.* 10 (1) (2022) 312–323.
- [150] H. Xu, B. Zhang, C. Pan, K. Li, Energy-efficient triple modular redundancy scheduling on heterogeneous multi-core real-time systems, *J. Parallel Distrib. Comput.* 191 (2024) 104915, <http://dx.doi.org/10.1016/j.jpdc.2024.104915>, URL: <https://www.sciencedirect.com/science/article/pii/S0743731524000790>.
- [151] I.I. Weber, V.B. Zanini, F.G. Moraes, Reinforcement learning for thermal and reliability management in manycore systems, *Des. Autom. Embedded Syst.* 29 (2) (2025).
- [152] S. Maity, A. Majumder, R. Roy, A. Hota, S. Dey, Harnessing machine learning in dynamic thermal management in embedded cpu-gpu platforms, *ACM Trans. Des. Autom. Electron. Syst.* 30 (2) (2025) 1–32.
- [153] Y. Xiao, S. Nazarian, P. Bogdan, Self-optimizing and self-programming computing systems: A combined compiler, complex networks, and machine learning approach, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 27 (6) (2019) 1416–1427, <http://dx.doi.org/10.1109/TVLSI.2019.2897650>.
- [154] F. Afshari, A. Abdi, Qsmix: Q-learning-based task scheduling approach for mixed-critical applications on heterogeneous multi-cores, *J. Supercomput.* 80 (2024) 17895–17922, <http://dx.doi.org/10.1007/s11227-024-06096-8>.
- [155] T. Shi, Q. Xu, J. Wang, C. Xu, K. Wu, K. Lu, C. Qiao, Enhancing the safety of autonomous driving systems via aol-Optimized task scheduling, *IEEE Trans. Veh. Technol.* 74 (3) (2025) 3804–3819, <http://dx.doi.org/10.1109/TVT.2024.3488194>.
- [156] M. Chen, J. Xu, W. Zhang, Z. Li, A new customer-oriented multi-task scheduling model for cloud manufacturing considering available periods of services using an improved hyper-heuristic algorithm, *Expert Syst. Appl.* 269 (2025) 126419, <http://dx.doi.org/10.1016/j.eswa.2025.126419>, URL: <https://www.sciencedirect.com/science/article/pii/S0957417425000417>.
- [157] H. Lin, M.-F. Li, C.-F. Jia, Degree-of-node task scheduling of fine-grained parallel programs on heterogeneous systems, *J. Comput. Sci. Tech.* 34 (5) (2019) 1096–1108, <http://dx.doi.org/10.1007/s11390-019-1962-4>.
- [158] E. Ilavarasan, P. Thambidurai, R. Mahilmanan, High performance task scheduling algorithm for heterogeneous computing system, in: M. Hobbs, A.M. Gosinski, W. Zhou (Eds.), *Distributed and Parallel Computing*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 193–203.
- [159] E. Ilavarasan, P. Thambidurai, Low complexity performance effective task scheduling algorithm for heterogeneous computing environments, *J. Comput. Sci.* 3 (2) (2007) 94–103, <http://dx.doi.org/10.3844/jcssp.2007.94.103>, URL: <https://thescpub.com/abstract/jcssp.2007.94.103>.
- [160] Q. Tan, W. Chen, D. Liu, A deep reinforcement learning scheduling algorithm for heterogeneous tasks on heterogeneous multi-core processors, in: 2024 6th International Conference on Electronics and Communication, Network and Computer Technology, ECNCT, 2024, pp. 519–523, <http://dx.doi.org/10.1109/ECNCT63103.2024.10704560>.
- [161] I. Khan, S. Ahmad, S. Neeha, A. Syed, S. Mubeen, A deep reinforcement learning framework for task scheduling for leveraging energy efficiency in cloud computing, *Springer Nature*, 2023, pp. 484–493, http://dx.doi.org/10.2991/978-94-6463-252-1_51.
- [162] U. Gupta, S.K. Mandal, M. Mao, C. Chakrabarti, U.Y. Ogras, A deep Q-learning approach for dynamic management of heterogeneous processors, *IEEE Comput. Archit. Lett.* 18 (1) (2019) 14–17, <http://dx.doi.org/10.1109/LCA.2019.2892151>.
- [163] S. Gupta, G. Agarwal, Hybrid fuzzy-based deep remora reinforcement learning based task scheduling in heterogeneous multicore-processor, *Microprocess. Microsyst.* 92 (2022) 104544, <http://dx.doi.org/10.1016/j.micpro.2022.104544>, URL: <https://www.sciencedirect.com/science/article/pii/S0141933122000990>.
- [164] R. Ayari, I. Hafnaoui, G. Beltrame, G. Nicolescu, ImGA: an improved genetic algorithm for partitioned scheduling on heterogeneous multi-core systems, *Des. Autom. Embedded Syst.* 22 (1) (2018) 183–197.
- [165] M.A. El Sayed, E.S.M. Saad, R.F. Aly, S.M. Habashy, Energy-efficient task partitioning for real-time scheduling on multi-core platforms, *Computers* 10 (1) (2021) <http://dx.doi.org/10.3390/computers10010010>, URL: <https://www.mdpi.com/2073-431X/10/1/10>.
- [166] Z. Tong, F. Ye, B. Liu, J. Cai, J. Mei, DDQN-TS: A novel bi-objective intelligent scheduling algorithm in the cloud environment, *Neurocomputing* 455 (2021) 419–430, <http://dx.doi.org/10.1016/j.neucom.2021.05.070>, URL: <https://www.sciencedirect.com/science/article/pii/S0925231221008237>.
- [167] X. Li, T. Zhou, H. Wang, M. Lin, Energy-efficient computation with DVFS using deep reinforcement learning for multi-task systems in edge computing, *IEEE Trans. Sustain. Comput.* (2025) 1–13, <http://dx.doi.org/10.1109/TSUSC.2025.3593971>.
- [168] J. Li, H. Zhang, Q. Yu, S. Li, L. Cheng, S. Wu, BD-TTS: A blockchain and DRL-based framework for trusted task scheduling in edge computing, *Comput. Netw.* 251 (2024) 110609, <http://dx.doi.org/10.1016/j.comnet.2024.110609>.
- [169] M. Mohammadi, H. Beitollahi, Q-scheduler: A temperature and energy-aware deep Q-learning technique to schedule tasks in real-time multiprocessor embedded systems, *IET Comput. Digit. Tech.* 16 (4) (2022) 125–140.
- [170] T. Tan, G. Cao, Thermal-aware scheduling for deep learning on mobile devices with NPU, *IEEE Trans. Mob. Comput.* 23 (12) (2024) 10706–10719, <http://dx.doi.org/10.1109/TMC.2024.3379501>.

- [171] M. Shiva Rama Krishna, S. Mangalampalli, A novel fault-tolerant aware task scheduler using deep reinforcement learning in cloud computing, *Appl. Sci.* 13 (21) (2023) <http://dx.doi.org/10.3390/app132112015>, URL: <https://www.mdpi.com/2076-3417/13/21/12015>.
- [172] T. Prem Jacob, K. Pradeep, A multi-objective optimal task scheduling in cloud environment using cuckoo particle swarm optimization, *Wirel. Pers. Commun.* 109 (1) (2019) 315–331.
- [173] S. Sudheer Mangalampalli, G. Reddy Karri, S. Nandan Mohanty, S. Ali, A.M. Alamri, S.A. Alqahtani, Efficient hybrid DDPG task scheduler for HPC and HTC in cloud environment, *IEEE Access* 12 (2024) 108897–108920, <http://dx.doi.org/10.1109/ACCESS.2024.3435914>.
- [174] Z. Tong, X. Deng, F. Ye, S. Basodi, X. Xiao, Y. Pan, Adaptive computation offloading and resource allocation strategy in a mobile edge computing environment, *Inform. Sci.* 537 (2020) 116–131, <http://dx.doi.org/10.1016/j.ins.2020.05.057>, URL: <https://www.sciencedirect.com/science/article/pii/S0020025520304655>.
- [175] B. Krishnamurthy, S.G. Shiva, Integral-valued pythagorean fuzzy-set-based dyna q+ framework for task scheduling in cloud computing, *Sensors* 24 (16) (2024) <http://dx.doi.org/10.3390/s24165272>, URL: <https://www.mdpi.com/1424-8220/24/16/5272>.
- [176] M. Farooq, A. Zafar, A. Samad, FR-EAHTS: Federated reinforcement learning for enhanced task scheduling with hierarchical load balancing and dynamic power adjustment in multi-core systems, *Telecommun. Syst.* 88 (2025) 48, <http://dx.doi.org/10.1007/s11235-025-01276-0>.
- [177] M. Ghahari-Bidgoli, M. Ghoabaei-Arani, A. Sharif, An efficient task offloading and auto-scaling approach for IoT applications in edge computing environment, *Computing* 107 (2025) 125, <http://dx.doi.org/10.1007/s00607-025-01471-6>.
- [178] Z. Zhang, C. Xu, K. Liu, S. Xu, L. Huang, A resource optimization scheduling model and algorithm for heterogeneous computing clusters based on GNN and RL, *J. Supercomput.* 80 (2024) 24138–24172, <http://dx.doi.org/10.1007/s11227-024-06383-4>.
- [179] Y. Zhou, H. Qin, G. Chen, Intelligent task scheduling for multi-core processors based on graph neural networks and deep reinforcement learning, in: *Proceedings of the 4th International Conference on Artificial Intelligence and Computer Engineering, ICAICE '23*, Association for Computing Machinery, New York, NY, USA, 2024, pp. 875–880, <http://dx.doi.org/10.1145/3652628.3652774>.
- [180] H. Hou, A. Ismail, EETS: An energy-efficient task scheduler in cloud computing based on improved DQN algorithm, *J. King Saud Univ. - Comput. Inf. Sci.* 36 (8) (2024) 102177, <http://dx.doi.org/10.1016/j.jksuci.2024.102177>, URL: <https://www.sciencedirect.com/science/article/pii/S1319157824002660>.
- [181] J. Li, S. Li, J. Tan, S. Cheng, D. Jin, S. Chen, J. Yang, Graph coding aided deep reinforcement learning for fault-tolerant workflow scheduling, in: *2024 IEEE 24th International Conference on Communication Technology, ICCT, 2024*, pp. 60–65, <http://dx.doi.org/10.1109/ICCT62411.2024.10946490>.
- [182] J. Zou, I. Gray, POMDP-active inference model for hybrid critical multi-core heterogeneous scheduling, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* (2025) <http://dx.doi.org/10.1109/TCAD.2025.3592586>, 1–1.
- [183] M. Guthaus, J. Ringenberg, D. Ernst, T. Austin, T. Mudge, R. Brown, MiBench: A free, commercially representative embedded benchmark suite, in: *Proceedings of the Fourth Annual IEEE International Workshop on Workload Characterization, WWC-4 (Cat. No.01EX538)*, 2001, pp. 3–14, <http://dx.doi.org/10.1109/WWC.2001.990739>.
- [184] R. Dick, D. Rhodes, W. Wolf, TGFF: task graphs for free, in: *Proceedings of the Sixth International Workshop on Hardware/Software Codesign., CODES/CASHE'98*, 1998, pp. 97–101, <http://dx.doi.org/10.1109/HSC.1998.666245>.
- [185] G. Juve, E. Deelman, K. Vahi, G. Mehta, B. Berriman, B.P. Berman, P. Maechling, Characterizing and profiling scientific workflows, *Future Gener. Comput. Syst.* 29 (3) (2013) 682–692, <http://dx.doi.org/10.1016/j.future.2012.08.015>.
- [186] E. Deelman, K. Vahi, G. Juve, M. Rynge, S. Callaghan, P.J. Maechling, R. Mayani, W. Chen, R. Ferreira da Silva, M. Livny, et al., Pegasus and CyberShake: Workflow applications on the TeraGrid, in: *TeraGrid Conference*, 2008, URL: <https://www.isi.edu/~deelman/papers/2008/TeraGrid08.pdf>.
- [187] E. Deelman, G. Singh, M.-H. Su, J. Blythe, Y. Gil, C. Kesselman, G. Mehta, K. Vahi, G.B. Berriman, J. Good, et al., Pegasus: A framework for mapping complex scientific workflows onto distributed systems, in: *Scientific Programming*, 13, IOS Press, 2005, pp. 219–237, <http://dx.doi.org/10.1155/2005/128026>.
- [188] S.K. Roy, R. Devaraj, A. Sarkar, SAFLA: Scheduling multiple real-time periodic task graphs on heterogeneous systems, *IEEE Trans. Comput.* 72 (4) (2023) 1067–1080, <http://dx.doi.org/10.1109/TC.2022.3191970>.
- [189] B. Tiganoaia, G. Iordache, C. Negru, F. Pop, Scheduling in CloudSim of interdependent tasks for SLA design, *Stud. Inf. Control* 28 (2019) 477–484, <http://dx.doi.org/10.24846/v28i4y201911>.
- [190] W. Chen, E. Deelman, WorkflowSim: A toolkit for simulating scientific workflows in distributed environments, in: *2012 IEEE 8th International Conference on E-Science*, 2012, pp. 1–8, <http://dx.doi.org/10.1109/eScience.2012.6404430>.
- [191] Alibaba Group, Alibaba cluster trace v2018, 2018, [Accessed 28 August 2025] <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018>.
- [192] Google Inc., Google cluster workload traces, 2011, 29-day trace from 12, 000 machines; (Accessed: 28 August 2025) <https://github.com/google/cluster-data>.
- [193] A. Hussain, M. Aleem, GoCJ: Google Cloud Jobs Dataset, 2018, <http://dx.doi.org/10.17632/b7bp6xhrcd.1>, URL: <https://data.mendeley.com/datasets/b7bp6xhrcd/1> Synthetic dataset generated from Google cluster trace statistics.
- [194] G. Liu, W. Lin, H. Zhang, J. Lin, S. Peng, K. Li, Public datasets for cloud computing: A comprehensive survey, *ACM Comput. Surv.* 57 (8) (2025) <http://dx.doi.org/10.1145/3719003>.
- [195] Transaction Processing Performance Council, TPC-h benchmark, 2014, (Accessed 28 August 2025) <http://www.tpc.org/tpch/>.
- [196] S. Arda, A. Krishnakumar, A. Goksoy, N. Kumbhare, J. Mack, A. Sartor, A. Akoglu, R. Marculescu, U. Ogras, DS3: A system-level domain-specific system-on-chip simulation framework, *IEEE Trans. Comput.* 69 (8) (2020) 1248–1262, <http://dx.doi.org/10.1109/TC.2020.2986963>.
- [197] D. Bailey, E. Barszcz, B. J.T., B. D.S., C. R.L., D. D., F. R.A., P. Frederickson, L. T.A., R. Schreiber, H. Simon, V. Venkatakrishnan, W. K., The nas parallel benchmarks, *Int. J. High Perform. Comput. Appl.* 5 (1991) 63–73, <http://dx.doi.org/10.1177/109434209100500306>.
- [198] C. Bienia, S. Kumar, J.P. Singh, K. Li, The PARSEC benchmark suite: Characterization and architectural implications, in: *2008 International Conference on Parallel Architectures and Compilation Techniques, PACT, Placeholder Publisher*, New York, NY, USA, 2008, pp. 72–81.
- [199] S. Woo, M. Ohara, E. Torrie, J. Singh, A. Gupta, The SPLASH-2 programs: characterization and methodological considerations, in: *Proceedings 22nd Annual International Symposium on Computer Architecture*, 1995, pp. 24–36, <http://dx.doi.org/10.1109/ISCA.1995.524546>.
- [200] J.J. Dongarra, P. Luszczek, A. Petitet, The LINPACK benchmark: past, present and future, *Concurr. Comput.: Pract. Exp.* 15 (9) (2003) 803–820, <http://dx.doi.org/10.1002/cpe.728>, arXiv: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.728> URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.728>.
- [201] R.K.V. Maeda, Q. Cai, J. Xu, Z. Wang, Z. Tian, Fast and accurate exploration of multi-level caches using hierarchical reuse distance, in: *2017 IEEE International Symposium on High Performance Computer Architecture, HPCA, 2017*, pp. 145–156, URL: <https://api.semanticscholar.org/CorpusID:4342195>.
- [202] M.A. Souza, P.H. Penna, M.M. Queiroz, A.D. Pereira, L.F.W. Góes, H.C. Freitas, M. Castro, P.O. Navaux, J.-F. Méhaut, CAP bench: a benchmark suite for performance and energy evaluation of low-power many-core processors, *Concurr. Comput.: Pract. Exp.* 29 (4) (2017) e3892, <http://dx.doi.org/10.1002/cpe.3892>, arXiv: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.3892> URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.3892> e3892 cpe.3892.
- [203] Y. Chandarli, F. Fauberteau, D. Masson, S. Midonnet, M. Qamhie, YARTISS: A tool to visualize, test, compare and evaluate real-time scheduling algorithms, in: *3 Rd International Workshop on Analysis Tools and Methodologies for Embedded and Real-Time Systems*, 2012, p. 21.
- [204] M. Chéramy, P.-E. Hladik, A.-M. Déplanche, Simso: A simulation tool to evaluate real-time multiprocessor scheduling algorithms, in: *5th International Workshop on Analysis Tools and Methodologies for Embedded and Real-Time Systems, WATERS, 2014*, pp. 6–p.
- [205] R.K. Vivas Maeda, P. Yang, H. Li, Z. Tian, Z. Wang, Z. Wang, X. Chen, J. Feng, J. Xu, A fast joint application-architecture exploration platform for heterogeneous systems, in: *Embedded, Cyber-Physical, and IoT Systems: Essays Dedicated To Marilyn Wolf on the Occasion of Her 60th Birthday*, Springer International Publishing, Cham, 2020, pp. 203–232, http://dx.doi.org/10.1007/978-3-030-16949-7_9.
- [206] T. Willhalm, R. Dementiev, Intel® performance counter monitor – a better way to measure CPU utilization, 2022, URL: <https://www.intel.com/content/www/us/en/developer/articles/tool/performance-counter-monitor.html>.
- [207] N. Binkert, B. Beckmann, G. Black, S.K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D.R. Hower, T. Krishna, S. Sardashti, et al., The gem5 simulator, *ACM SIGARCH Comput. Archit. News* 39 (2) (2011) 1–7.
- [208] J.-H. Han, R.E. West, K. Skadron, M.R. Stan, Thermal simulation of processing-in-memory devices using HotSpot 7.0, in: *2021 27th International Workshop on Thermal Investigations of ICs and Systems, THERMINIC, 2021*, pp. 1–5, <http://dx.doi.org/10.1109/THERMINIC52472.2021.9626520>.
- [209] S. Li, J.H. Ahn, R.D. Strong, J.B. Brockman, D.M. Tullsen, N.P. Jouppi, The McPAT framework for multicore and manycore architectures: Simultaneously modeling power, area, and timing, *ACM Trans. Archit. Code Optim. (TACO)* 10 (1) (2013) 1–29.
- [210] M. Maerz, P. Nance, Thermal modeling of power electronic systems, *PCIM Eur. Mag* 2 (2000).
- [211] J. Yao, L. Mo, J. Han, D. Niu, Task scheduling and temperature optimization co-design for multi-core embedded systems, in: *2025 International Symposium of Electronics Design Automation, ISEDA, 2025*, pp. 683–688, <http://dx.doi.org/10.1109/ISED65950.2025.11100782>.
- [212] L. Deng, The MNIST database of handwritten digit images for machine learning research [best of the web], *IEEE Signal Process. Mag.* 29 (6) (2012) 141–142, <http://dx.doi.org/10.1109/MSP.2012.2211477>.
- [213] A. Geiger, P. Lenz, R. Urtasun, Are we ready for autonomous driving? The KITTI vision benchmark suite, in: *2012 IEEE Conference on Computer Vision and Pattern Recognition*, 2012, pp. 3354–3361, <http://dx.doi.org/10.1109/CVPR.2012.6248074>.

- [214] EEMBC, CoreMark: An EEMBC benchmark, 2023, (Accessed: 13 March 2025) <https://www.eembc.org/coremark/>.
- [215] EEMBC, Application note CMP01: Porting CoreMark-PRO to Bare-Meta, 2021, URL: <https://www.eembc.org/techlit/apnotes/cmp01.pdf> Application Note.
- [216] J.L. Henning, SPEC CPU2006 benchmark descriptions, SIGARCH Comput. Arch. News 34 (4) (2006) 1–17, <http://dx.doi.org/10.1145/1186736.1186737>.
- [217] R.P. Weicker, Dhrystone: A synthetic systems programming benchmark, Commun. ACM 27 (10) (1984) 1013–1030, <http://dx.doi.org/10.1145/358274.358283>.
- [218] H.J. Curnow, B.A. Wichmann, A synthetic benchmark, Comput. J. 19 (1) (1976) 43–49, Central Computer Agency, Riverwalk House, London SW1P 4RT; National Physical Laboratory, Teddington, Middlesex TW11 0LW.
- [219] R. Dick, Embedded system synthesis benchmarks suite (E3S), 2010, 2009, URL: <http://Ziyang.Eecs.Umich.Edu/~Dickrp/E3sdd>.
- [220] S. Stuijk, M. Geilen, T. Basten, Sd³: SDF for free, in: Proceedings of the Sixth International Conference on Application of Concurrency To System Design, ACSD'06, 2006, pp. 276–278, <http://dx.doi.org/10.1109/ACSD.2006.23>.
- [221] I. Almomani, B. Al-Kasasbeh, M. AL-Akhras, WSN-DS: A dataset for intrusion detection systems in wireless sensor networks, J. Sensors 2016 (1) (2016) 4731953, <http://dx.doi.org/10.1155/2016/4731953>, arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1155/2016/4731953> URL: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2016/4731953>.
- [222] R. ZHAO, NSL-KDD, 2022, <http://dx.doi.org/10.21227/8rpg-qf98>.
- [223] D. Arp, M. Spreitzenbarth, M. Hübner, H. Gascon, K. Rieck, DREBIN: Effective and explainable detection of android malware in your pocket, in: Symposium on Network and Distributed System Security, NDSS, 2014, <http://dx.doi.org/10.14722/ndss.2014.23247>,
- [224] M. Dellinger, A. Lindsay, B. Ravindran, An experimental evaluation of the scalability of real-time scheduling algorithms on large-scale multicore platforms, J. Exp. Algorithmics (JEA) 17 (2012) 4–1.
- [225] S. Mirjalili, A. Lewis, The whale optimization algorithm, Adv. Eng. Softw. 95 (2016) 51–67, <http://dx.doi.org/10.1016/j.advengsoft.2016.01.008>, URL: <https://www.sciencedirect.com/science/article/pii/S0965997816300163>.
- [226] Z. Wang, M. Goudarzi, M. Gong, R. Buyya, Deep reinforcement learning-based scheduling for optimizing system load and response time in edge and fog computing environments, Future Gener. Comput. Syst. 152 (2024) 55–69, <http://dx.doi.org/10.1016/j.future.2023.10.012>, URL: <https://www.sciencedirect.com/science/article/pii/S0167739X23003862>.
- [227] Z. Tong, H. Chen, X. Deng, K. Li, K. Li, A scheduling scheme in the cloud computing environment using deep Q-learning, Inform. Sci. 512 (2020) 1170–1191, <http://dx.doi.org/10.1016/j.ins.2019.10.035>, URL: <https://www.sciencedirect.com/science/article/pii/S0020025519309971>.
- [228] A. Asghari, M.K. Sohrabi, F. Yaghmaee, A cloud resource management framework for multiple online scientific workflows using cooperative reinforcement learning agents, Comput. Netw. 179 (2020) 107340, <http://dx.doi.org/10.1016/j.comnet.2020.107340>, URL: <https://www.sciencedirect.com/science/article/pii/S1389128619305791>.
- [229] A. Senior, G. Heigold, M. Ranzato, K. Yang, An empirical study of learning rates in deep neural networks for speech recognition, in: 2013 IEEE International Conference on Acoustics, Speech and Signal Processing, IEEE, 2013, pp. 6724–6728.
- [230] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533.